

U

O

W

CSIT314

Software Development

Methodologies

Subject Admin Introduction



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Content Introduction



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Contact details

- Dr. Yudi Zhang (yudi@uow.edu.au)
- Tutor: Mr. Terence Chew (tchew@uow.edu.au)
Chun Huei Liaw (chliaw@uow.edu.au)

Subject objectives (CSIT314)

- On successful completion of this subject, students will be able to, to varying degrees:
 - demonstrate an in-depth understanding of the stages involved in software development and the issues to be considered at each stage.
 - compare and contrast different software development methodologies and process models, and assess their suitability in different development contexts.
 - deploy appropriate theory, practices, and tools for the specification, design, implementation and evaluation of computer-based systems.
 - function effectively as part of a team to apply state-of-the-art software development methodologies to the development of a software system.
 - apply different strategies for assessing and improving software development processes.
 - apply professional standards in software development



Topics

- Introduction and Software Development Lifecycle
- Overview of software process models
- Advanced Unified Modelling Language
- Test driven software development
- Principles and practices of continuous integration and delivery
- DevOps software development practices
- Unified software development process
- Extreme programming
- Kanban software development method
- Data-driven software development
- *And a few others*



Resources

- Lectures
 - PDF files with slides from lectures
- Labs and the project
- Supplementary materials
- One-stop shop: Moodle



Textbook and references

- David Avison, Guy Fitzgerald, Information Systems Development: Methodologies, Techniques and Tools (4th Edition), McGraw-Hill Education, 2006.
- Roger S. Pressman, Software Engineering: A Practitioner's Approach (8th Edition), McGraw-Hill Education, 2014.
- Paul VII, Randal Schaffer, Scrum: A Cleverly Concise and Agile Guide, Pashun Consulting Ltd., 2016
- Craig Larman, Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd Edition), Prentice Hall, 2004
- Philippe Kruchten, The rational unified process: an introduction. Addison-Wesley Professional, 2000
- Mary Beth Chrissis, Michael D. Konrad, Sandra Shrum. CMMI: Guidelines for Process Integration and Product Improvement (2nd Edition), Addison-Wesley Professional, 2006
- Paul Ammann and Jeff Offutt, Introduction to Software Testing (2nd Edition), Cambridge University Press, 2016
- Ian Sommerville, Software Engineering (9th Edition), Cambridge University Press, 2016, <https://cs.gmu.edu/~offutt/softwaretest/>
- IBM DevOps For Dummies®, 3rd IBM Limited Edition, <https://www.ibm.com/downloads/cas/P9NYOK3B>
- Gene Kim, Jez Humble, Patrick Debois, John Willis, The DevOps Handbook: How to Create WorldClass Agility, Reliability, and Security in Technology Organizations, IT Revolution Press, 2016



Assessment

- Lab exercises (10%)
 - Will be assessed in the 4th lab session.
- Group project (40%)
 - Final deliverables
 - Project presentation Q&A - last lab session.
- Final examination
 - Technical Fail
 - To be eligible for a Pass in this subject a student must achieve a mark of at least 40% in the Final Examination.
 - Students who fail to achieve this minimum mark & would have otherwise passed may be given a TF (Technical Fail) for this subject.



Tutorial/Lab

- Each tutorial/lab:
 - First half: an exercise
 - Second half: project
 - Work on the project.
 - Meet “the client” session.
 - Tutor will note your group’s attendance, progress, interactions with “client”, etc. which are the factors considered for the final marking of the project.



Group project

- Up to 7 students per group
- Formation of groups is your responsibility
- You will have to form a group (better within the same lab)
- The group leader needs to submit your group details via Moodle by the end of next week.
- Contact your tutor if you need assistance in forming a group.
- Penalties may be applied if submitting this late.



Academic misconduct

- The Academic Integrity Policy, available at:
<http://www.uow.edu.au/about/policy/UOW058648.html>
- Plagiarism
 - Using another person' s ideas, designs, words or any other work without appropriate acknowledgement ;
 - Re-using one' s own work without appropriate acknowledgement.



Academic misconduct

- copying without appropriate referencing
 - at the very least comment it, but if the code is mostly the work of others you are going to get zero.
- copying from each other
 - Github or sharing codes



Introduction to Software Development and its Lifecycle



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Software Engineering

- The definition (IEEE standard 610.12-1990)
 - “the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software”.
 - Tasks: requirements=> design => construction => testing => analysis => maintenance



Software Engineering

- Popularity: more and more major businesses and industries are being run on software and delivered online services.
 - But “With great power there must also come great responsibility” ...
- Quality:
 - Jobs
 - Money
 - Lives
 - ...
- Software Engineering plays a critical part in the software development, in particular with a complex software.



Components of Software Engineering

- PRODUCT
 - The actual software product or system that is built and put into operation
- PROCESS
 - A framework for the tasks that are required to build high-quality software.



What is Engineering

- A body of knowledge used when building things
 - Scheduling
 - Costing
 - Building
 - Testing
 - Communicating
 - Organising
 - ...

It is easy to build something if you have unlimited money and time. A professional differs from an amateur in that they can contain costs and time.



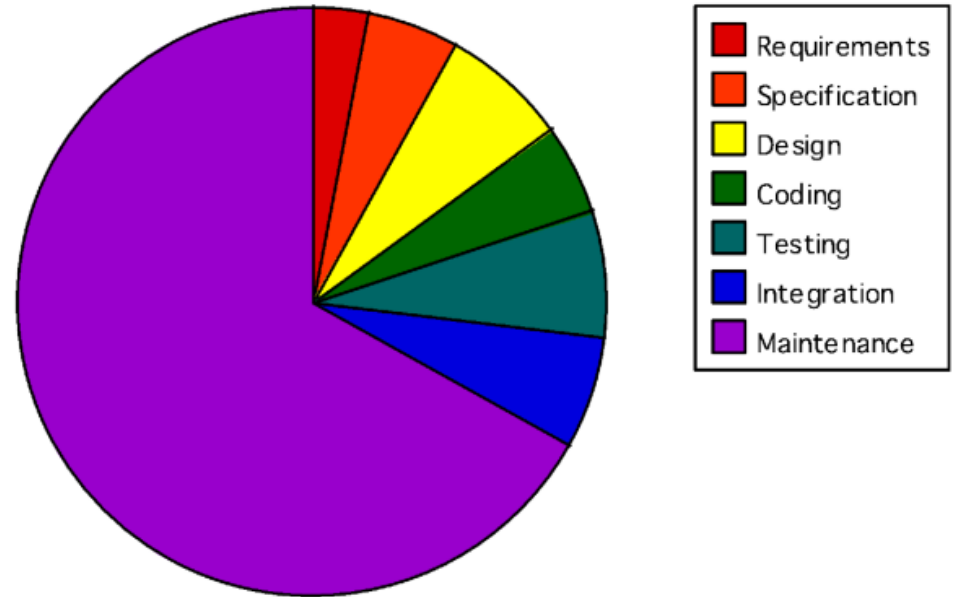
How software is different?

- Software is soft and intangible
- There are no physical laws underlying software behavior
- Software hard to wear out
 - traditional reliability measures don' t apply
- The specification for software continuously changes



Software Development Activities

- Software Engineering plays a critical part in the software development
 - Planning
 - Requirements analysis
 - Design
 - Implementation
 - Verification & Validation
 - Maintenance and evolution



Planning

- Identify business requirement or value
- Analyze feasibility
- Develop work plan
- Staff
- Estimate
- Identify risk



Requirements analysis

- Identify what services are required and the constraints on the system's operation and development.
 - Who are the stakeholders
 - What do they expect
 - Interviews, questionnaires, meeting, etc.
 - Defining the requirements in detail
 - Use cases are the major part of a requirements specification



Design

- Design is concerned with HOW we build a system (not WHAT, more details later)
 - Architectural design
 - Information architecture, data dictionaries, workflow
 - Sub system design
 - Detailed design
 - Persistent data design
 - GUI design



Architecture design

- Concerned with major structure of the software
 - Similar to building architecture (e.g. roof, walls, foundations)
 - Contains sub-systems (air-conditioning unit, water pump)
 - Uses external services (sewerage, water, gas, electric)
 - The most important part of design - if the architecture is wrong, the house will fall down (or software will fall down!)
 - Example: Data-centric, Client-server, Service-oriented



Sub system design

- Describes interfaces/protocols of each subsystem
- Structure of each sub-system
- Structure can describe how each sub-system sub-divides into packages, components



Detailed and Persistent data design

- Detailed design
 - Describes each class, methods, attributes/datatypes
- Persistent data design
 - Describes choice of database, tables, primary and foreign keys



User interface design

- how the GUI will look like
- what tools will be used to build it
- what design (layout) guidelines will be followed
- what style guidelines will be followed



Implementation

- Construction
 - Write the code for the project.
- Installation (or deployment)

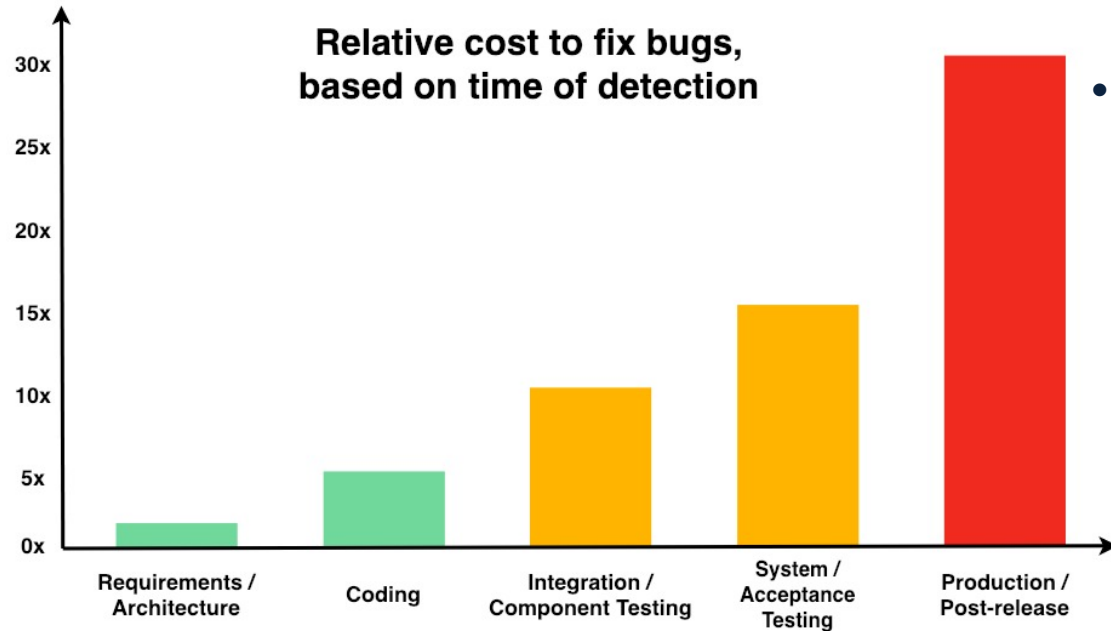


Verification & Validation

- Verification and validation (V & V)
 - The software should conform to its specification (in a correct way)
 - The software should do what the user really requires
- Testing is the most commonly used V & V activity.
- System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.



Why need testing



- Cost of correcting an error in requirement specifications increases as we move through lifecycle phases

Maintenance and evolution

- Changes are inevitable in software development
 - New functionalities requirements emerged at any time in the software development lifecycle
 - Changes in business environments
 - competition, laws, new markets, new customers, etc.
 - Changes in infrastructure environments
 - new servers, new equipment, etc.
 - New soft technology arriving
 - New version of OS, new standards, etc.
 - Bugs need fixing
 - Performance needs improvement



Maintenance and evolution

- Adaptive maintenance
 - Changing the system in response to changes in its environment so it continues to function
- Corrective maintenance
 - Fixing errors & bugs
- Perfective maintenance
 - Changing the system' s functionality to meet changing needs



The eight laws of software evolution

- Law 1: Continuing change
 - system must be continually adapted or it becomes progressively less satisfactory
- Law 2: Increasing complexity
 - system complexity increases unless work is done to maintain or reduce
- Law 3: Self regulation
 - a system to adjust and maintain itself without external intervention, such as feedback loops to adapt to changing conditions, correct errors, and maintain stability over time

The eight laws of software evolution

- Law 4: Conservation of organisational stability
 - organizational stability is preserved despite ongoing changes and improvements to the system
- Law 5: Conservation of familiarity
 - all association (developers, sales personnel and users) must maintain a high level of expertise or proficiency
- Law 6: Continuing growth
 - the functional content must be continually increased



The eight laws of software evolution

- Law 7: Declining quality
 - the quality will appear to be declining unless it is rigorously maintained and adapted to operational environment changes
- Law 8: Feedback system
 - multi-level, multi-loop, multi-agent feedback systems and must be treated as such to achieve significant improvement



U

O

W

CSIT314

Software Development

Methodologies

Software process models



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Outline

- Software process models
 - Waterfall model
 - Agile methods (Scrum in particular)
 - Iterative/incremental model
 - Prototyping model
 - Spiral model



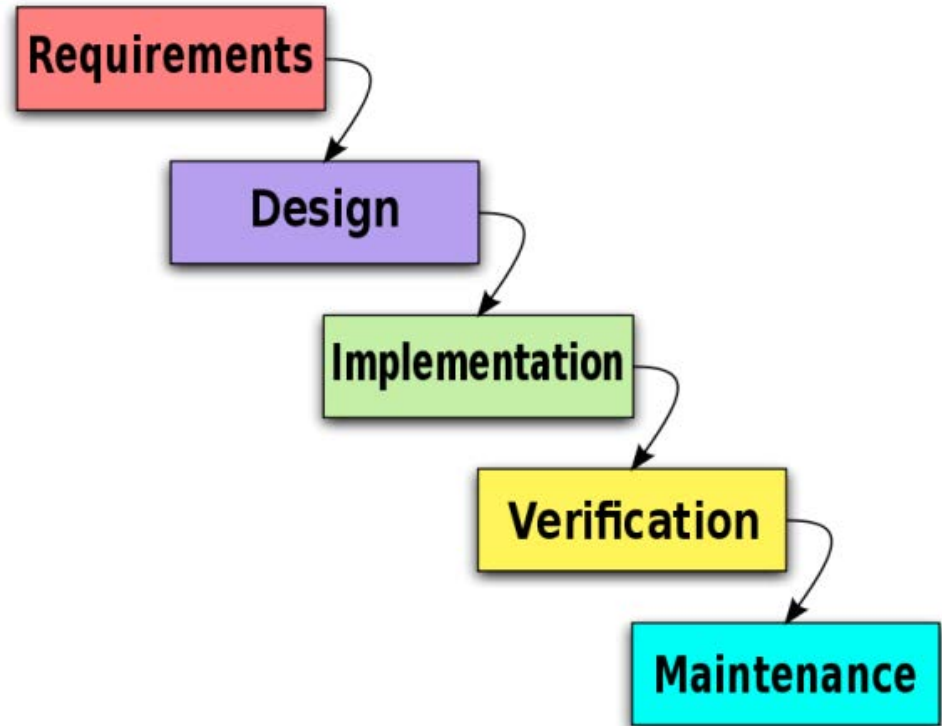
The software process

- A structured set of activities required to develop a software system, which involve:
 - Planning - feasibility, staff, identify risk, etc
 - Requirements analysis - defining what the system should do
 - Design & Implementation - defining how to build the system
 - Verification & Validation (V & V) - do the right thing in a right way
 - Maintenance and Evolution - changing the system in response to changing
- A software process model is an abstract representation of a process.



Waterfall model

- The waterfall model (adapted for software development since 1970s) is a sequential software development process.
 - flows from the top to the bottom
 - a phase has to be complete and absolutely correct before moving onto the next phase.



Waterfall model

- Waterfall project development process works well when:
 - Short term, small scope, less uncertainty
 - Requirements are very well understood and fixed at the outset of the project.
 - Product definition is stable and not subject to changes.
 - Technology is understood.
 - Ample resources with required expertise are available freely.
 - The project is of short duration.
- What happen if?
 - Customer tastes/ requirements changing
 - Disruptive technologies
 - Complex process
 - Such as producing a correct design before implementation and predicting problems
 - Long time frames



Agile methods

- In the 1990s, the IT industry faces substantial frustration
 - Most project followed the waterfall model
 - Big time gap between business requirements and the delivery of software to address those needs
 - Slow plan and/or design
 - Business environments and requirements quickly changed and the final product did not meet the then current needs.
- The waterfall model did not take advantage of how quickly software could be altered or developed



Agile methods

- Project planning
 - Emphasize on flexibility (resource constrain) and evolving customer requirements throughout the development process.
 - Allow flexibility react quickly to opportunities/risks
 - Planning all the time
 - Importance of evolving customer needs leads to incremental, iterative planning process
 - Stay connected with clients



Agile methods

- Agile refers to a set of methods and practices
 - requirements and solutions through the collaborative effort of teams and customers.
 - adaptive planning
 - iterative and incremental development
 - early delivery
 - continuous improvement
 - rapid and flexible response to change is encouraged



Agile principles

- Early and continuous delivery
- Welcome changing requirements
- Deliver working software frequently, with a preference to the shorter timescale
- Business people and developers must work together daily
- Support motivated individuals, and trust them to get the job done
- face-to-face conversation



Agile principles

- Working software (with specific functions)
- Sustainable development between developers and users to maintain a constant pace
- Attention to technical excellence and good design
- Simplicity--simplify their processes and elimination of unnecessary tasks, features--is essential
- Self-organizing teams, not relying on top-down management or extensive upfront planning
- The team learns to become more effective, then tunes and adjusts its behavior accordingly



Agile software development methods

- Scrum
- Extreme Programming
- Kanban
- Dynamic Systems Development Methods
- Lean software development
- Etc.

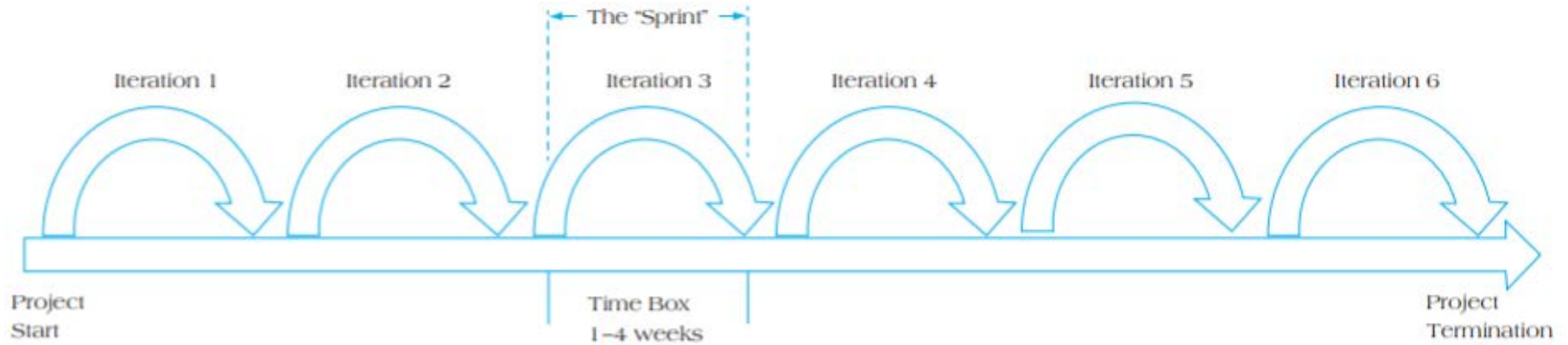


Scrum

- Flexible, iterative system designed for change and uncertainty
- Initial requirement => workable product => collect feedback
- “Rolling wave” process of continuous plan - execute - evaluate cycle
 - first come (request) first serve
 - recognise mistakes of assuming initial project
 - recognise the constraints (different from original specifications)
 - software projects are prone to constant changes
 - recognise customers’ requirement



Scrum



- Sprint—one iteration of the planning and executing cycle.
- Time-box—the length of any particular sprint, fixed in advance, during the Scrum meeting.



Scrum

- User stories—short explanation of the end user that captures what they do or what they need from the project under development (more later)
- Sprint backlog—the set of product backlog items selected for the Sprint, plus a plan for delivering the Sprint Goal. (from the develop team mainly)
- Burndown chart—remaining work in the Sprint backlog.
- Product backlog—a prioritized list of everything that might be needed in completed product and source of requirements for any changes (more later)
- Work backlog—evolving, prioritized queue of business and technical functionality that needs to be developed into a system



User stories

- User stories are short, simple descriptions of a feature told from the perspective of a user or customer
- User stories typically follow a simple template:
 - As a < type of user >, I want < some goal > so that < some reason >.
- Examples:
 - As a site visitor, I can read current news on the home page.
 - As a trainer, I can create a new course or event. This includes the following information: name, description (HTML), trainer names (multiple selection from a list), start date, end date, venue name (HTML) and address, contact name, contact phone, contact email, a link for more information, and a link to register. For a certification course the name of the class is a dropdown list; for others, it is free text.



Product backlog

- comprises the following different types of items:
 - Features
 - Bugs
 - Technical work (e.g. “Upgrade all developers’ workstations to Windows 11”)
 - Knowledge acquisition (e.g. “researching various JavaScript libraries and making a selection.”)
 - Prioritized by the product owner
 - Reprioritized at the start of each sprint



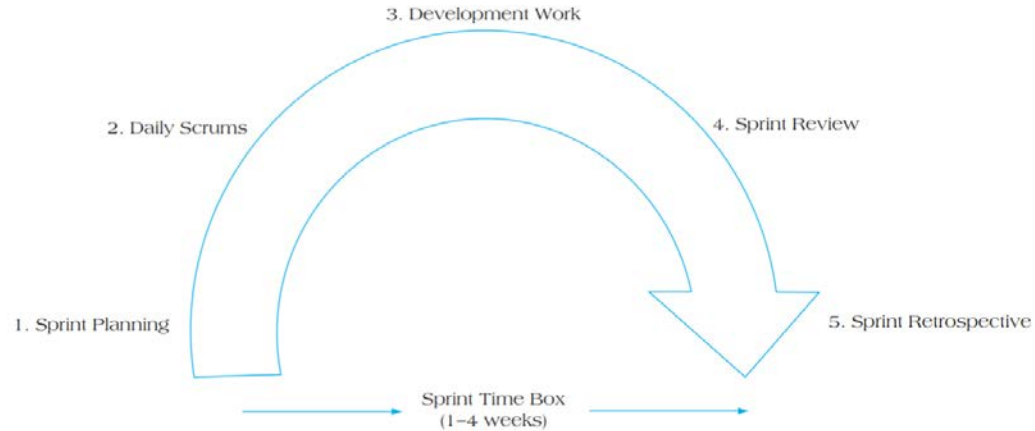
Scrum

- Product owner:
 - Define the features of the product
 - Decide on release date and content
 - Prioritize features according to market value
 - Adjust features and priority every iteration, as needed
 - Accept or reject work results
- Scrum Master
 - Represents management to the project, ensure that the team is fully functional and productive.
 - Responsible for enacting Scrum values and practices
- The team
 - Self-organizing and cross-functional: Programmers, testers, user experience designers, etc.



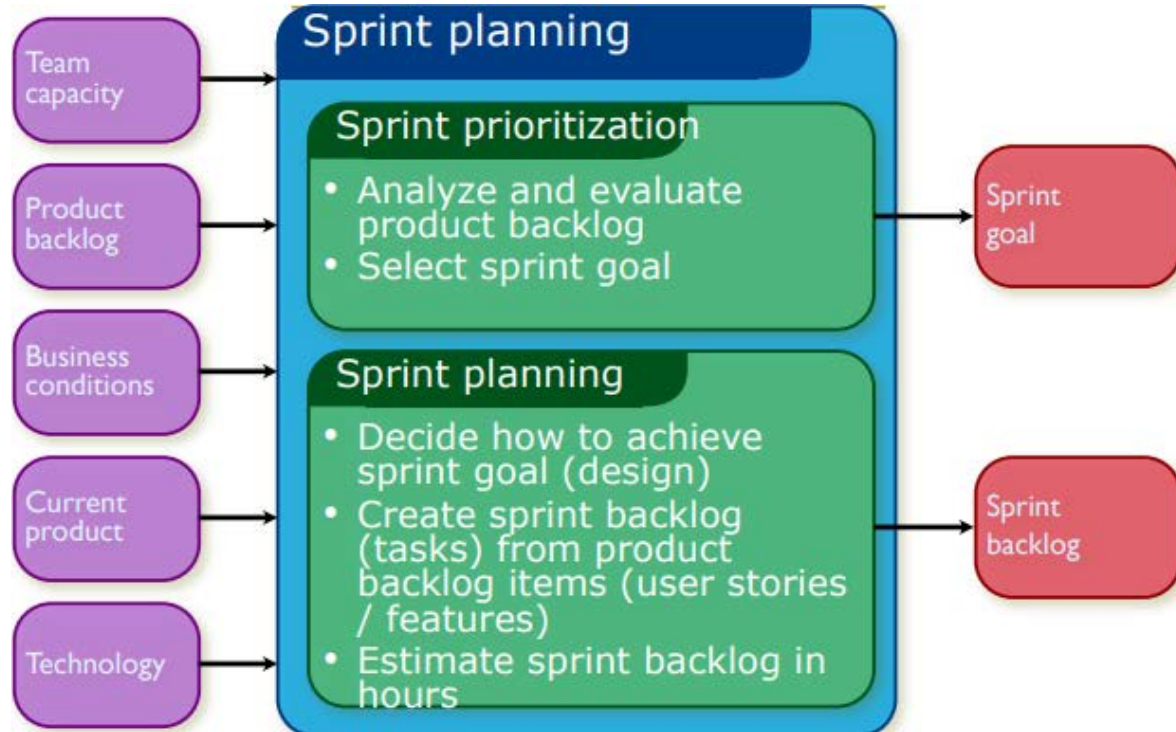
Steps

- Sprint Planning
- Daily Scrums
- Development Work
- Sprint Review
- Sprint Retrospective



Sprint Planning

- collaborative work of the entire Scrum team
- a maximum of one full day (eight hours) for a one-month Sprint
- Generate Sprint Backlog
 - What
 - How



Sprint planning

As a site visitor, I can read current news on the home page.

Develop UML design (4 hours)
Code the middle tier (8 hours)
Code the user interface (4)
Write test fixtures (4)
Code the foo class (6)

- Team decides on the goal for a particular sprint, and then selects items from the product backlog they commit to complete to achieve the sprint's goal.
- Sprint backlog is created
 - Tasks are identified and each is estimated (e.g. 1-16 hours)
 - Collaboratively, not done alone by the Scrum Master
 - High-level design is considered



Daily Scrum

- a short (15 minutes) event for information purposes
- synchronize their activities and create a plan for the next 24-hour time
- Leads to
 - Dev work
 - Sprint duration - 15 days
 - Team size - 5 members
 - Hours/day - 8
 - Total capacity - 600 hours

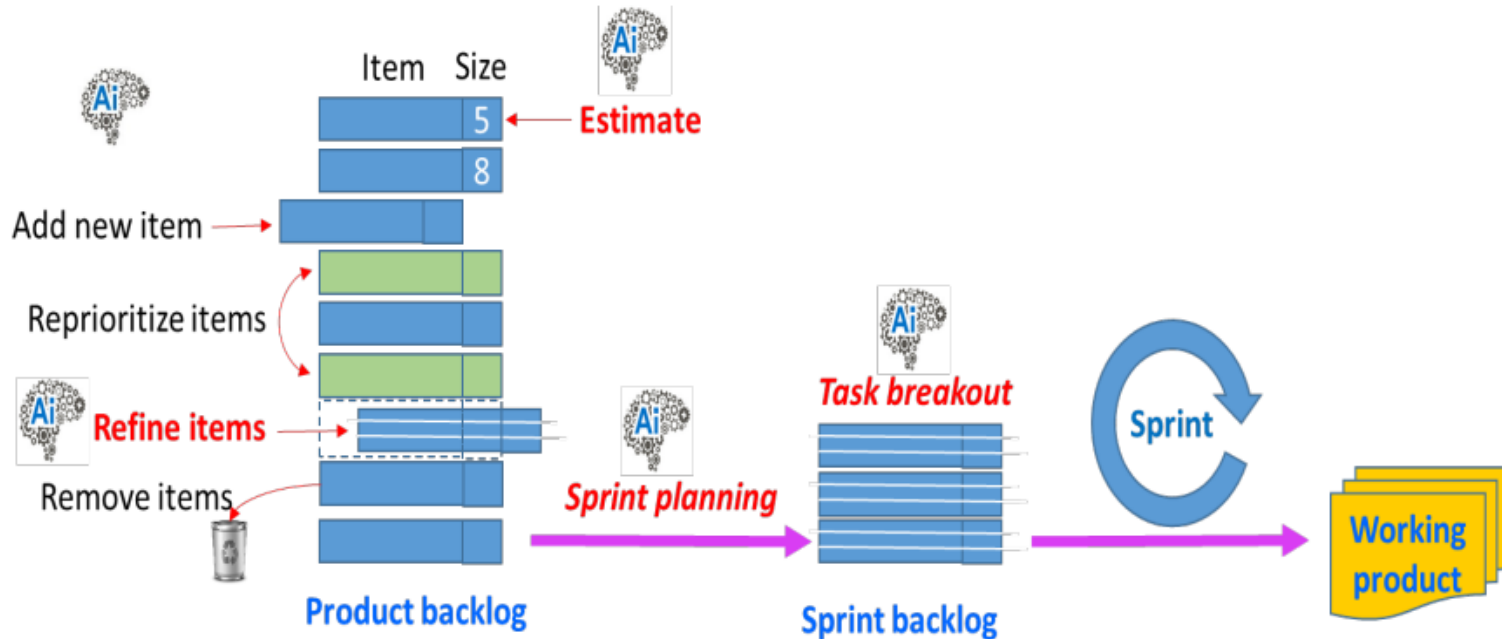


Sprint Review

- (informal) end of the Sprint to encourage feedback and collaboration
- the Scrum team and other key stakeholders
- leads to
 - Modified product backlog
 - Done vs not yet
 - impact
 - Next sprint backlog
 - Timeline, budget, etc
 - Problems, marketplace and/or potential uses



AI for Scrum Development

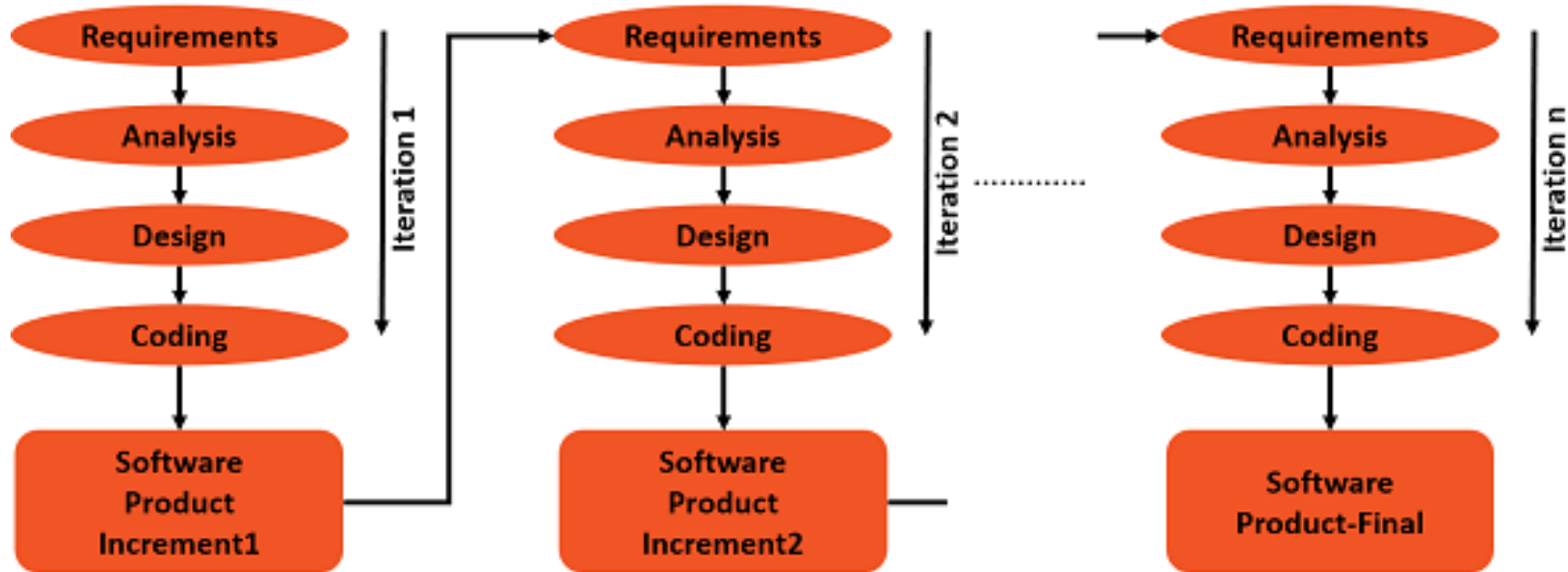


Incremental and Iterative model

- Repeated cycles (iterative)
 - User requirements are prioritised and the highest priority requirements are focus in early iteration
 - At each iteration, design modifications are made and new functional capabilities might be added
- Smaller portions (incremental)
 - Not a single delivery, but to break down
 - Each small one focuses on parts of required functionality



Incremental and Iterative model



Incremental and Iterative model benefits

- Feedback from early stages might still be used in developing latter stages
- The cost of accommodating requirement change is reduced
- More rapid delivery and deployment of useful software to the customer is possible.
 - Customers are able to use and gain value from the software earlier than is possible with a waterfall process



Incremental and Iterative model benefits

- Early increments act as a prototype to help improve requirements for later increments
- Lower risk of overall project failure (due to the break down)
- The highest priority system services tend to receive the most testing

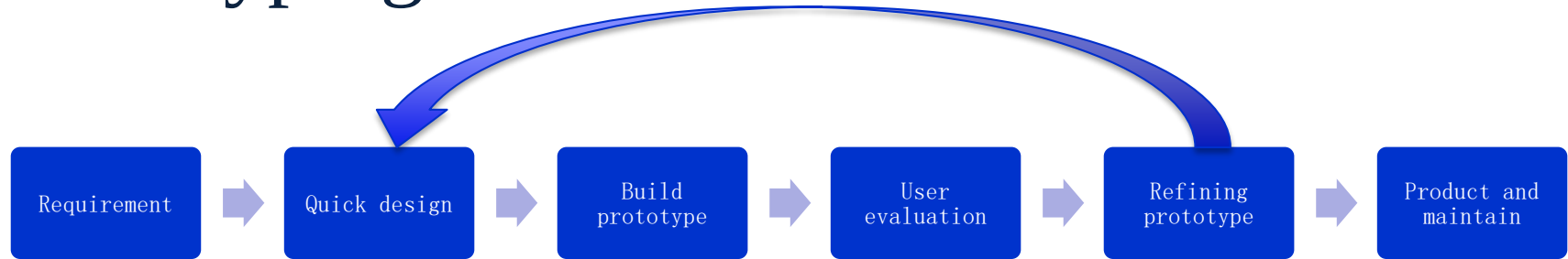


Possible disadvantages

- Loss of economy of scale
 - some costs will be repeated
- System structure tends to degrade as new increments are added.
 - regular change tends to corrupt its structure
 - refactoring is required (change to existing features)
 - incorporating further software changes becomes increasingly difficult and costly



Prototyping model



- gather requirements (user interview)
- quick design focusing on what will be visible to user - input & output formats
- build a prototype, i.e. a working version of the system
- prototype evaluated and requirements refined (user suggestion or comments)
- process iterated until customer & developer satisfied
 - then throw away prototype and “rebuild” system to high quality
- a key property to prototyping is the fact that it is supposed to be done quickly



Throw-away prototypes

- Prototypes should be discarded after development as they are not a good basis for a production system:
 - prototypes are normally undocumented
 - the prototype structure is usually degraded due to rapid change



Prototyping model benefits

- Users are actively involved in the development
 - exploring ideas and getting instant feedback from customers
- Reduced time and costs
 - a working model of the system is provided
 - the users get a better understanding of what developed
 - quicker user feedback
 - improve the quality of requirements and specifications provided to developers.
- Example projects to use prototyping
 - Human-Computer Interface



Prototyping model limitations

- User confusion of prototype and completed system
 - Users can begin to think that a prototype, intended to be thrown away, is the final system that can be delivered soon
 - Customers may not be willing to participate in a long-iteration cycle
- Developer attachment to prototype
 - Prototyping may encourage excessive change requests
 - Too many variations in software requirements



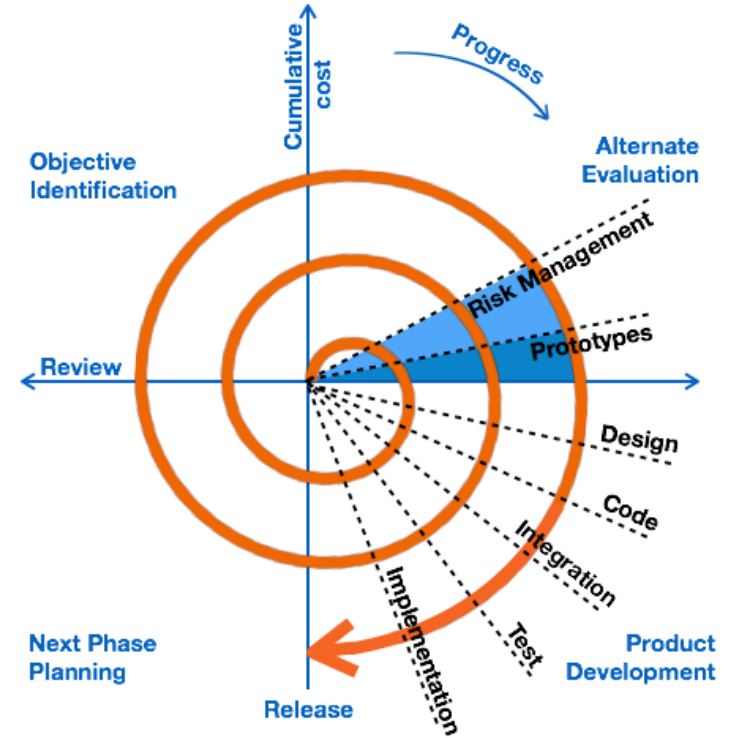
Prototyping model limitations

- Excessive development time of the prototype:
 - A key property to prototyping is the fact that it is supposed to be done quickly
 - the developers may try to develop a prototype that is too complex
- Expense of implementing prototyping:
 - A slow and time taking process, and to be thrown away
 - The costs for building a development team focused on prototyping may be high



Spiral Model

- A combination of waterfall with iterative model
 - Process is represented as a spiral rather than as a sequence of activities
- Workload is less at the beginning
- Allows incremental releases or refinement of the product through each iteration



Spiral Model

- helping people think about iteration in software processes and introducing the risk-driven approach
- mostly used in large projects:
 - Game development is a main area where the spiral model is used and needed
 - Military projects with a 2 year iteration



Limitations

- Complexity
 - multiple stages of planning, risk analysis, prototyping, and evaluation
 - requires extensive documentation at each stage of the process
 - iterative nature can lead to increased costs and time
 - may not be well-suited for small projects with well-defined requirements and limited scope
 - more beneficial for large-scale projects with complex requirements and significant uncertainties



Software Ethics

- Software ethics refers to the moral values and principles that guide the behavior and actions of individuals and organizations involved in software development.
 - Values: good things we strive for, desire and seek to protect.
 - Principles: outlining how we may or may not achieve our values (telling us what's right or wrong).
 - Purpose: reason for upholding our values and principles.



Why are ethics important in software development?

- Software are everywhere in people' s lives, across almost all industry sectors in our society
- Software can do good or cause harm
- Any software development activities (requirements, design, implementation, testing and maintenance) can lead to those consequences



ACM Code of Ethics

- Principle 1: PUBLIC – act consistently with the public interest, e.g. :
 - Accept full responsibility for their own work, and consider the public interest when they use our work.
 - Approve software only if they have a well-founded belief that it is safe, meets specifications, passes appropriate tests, and does not diminish quality of life, diminish privacy or harm the environment.
 - Disclose any actual or potential danger to the user, the public, or the environment.
 - Consider issues of physical disabilities, allocation of resources, economic disadvantage and other factors that can diminish access to the benefits of software.



ACM Code of Ethics

- Principle 2: CLIENT AND EMPLOYER – in the best interests of their client and employer, consistent with the public interest, e. g. :
 - Provide service in their areas of competence, being honest about any limitations of their experience and education.
 - Not knowingly use illegal or unethical software.
 - Keep private any confidential information gained in their professional work, where such confidentiality is consistent with the public interest and consistent with the law.



ACM Code of Ethics

- Principle 3: PRODUCT – ensure that their products and related modifications meet the highest professional standards possible, e.g. :
 - Strive for high quality, acceptable cost and a reasonable schedule, ensuring significant tradeoffs are clear to and accepted by the employer and the client, and are available for consideration by the user and the public.
 - Identify, define and address economic, cultural, legal and environmental issues related to work projects.
 - Ensure adequate testing, debugging, and review of software and related documents on which they work.



ACM Code of Ethics

- Principle 4: JUDGMENT – maintain integrity and independence in their professional judgment, e.g.:
 - Only endorse documents either prepared under their supervision or within their areas of competence and with which they are in agreement.
 - Maintain professional objectivity with respect to any software or related documents they are asked to evaluate.
 - Disclose to all concerned parties those conflicts of interest that cannot reasonably be avoided or escaped.



ACM Code of Ethics

- Principle 5: MANAGEMENT – subscribe to and promote an ethical approach to the management of software development and maintenance, e.g. :
 - Ensure good management for any project on which they work, including effective procedures for promotion of quality and reduction of risk.
 - Ensure that there is a fair agreement concerning ownership of any software, processes, research, writing, or other intellectual property to which a software engineer has contributed.



ACM Code of Ethics

- Principle 6: PROFESSION – advance the integrity and reputation of the profession consistent with the public interest, e.g. :
 - Take responsibility for detecting, correcting, and reporting errors in software and associated documents on which they work.



ACM Code of Ethics

- Principle 7: COLLEAGUES – be fair to and supportive of their colleagues, e.g.:
 - Credit fully the work of others and refrain from taking undue credit.
 - Review the work of others in an objective, candid, and properly-documented way.



ACM Code of Ethics

- Principle 8: SELF – shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession, e.g.:
 - Further their knowledge of developments in the analysis, specification, design, development, maintenance and testing of software and related documents, together with the management of the development process.
 - Improve their ability to create safe, reliable, and useful quality software at reasonable cost and within a reasonable time.



U

O

W

CSIT314

Software Development

Methodologies

Advanced Unified Modelling Language



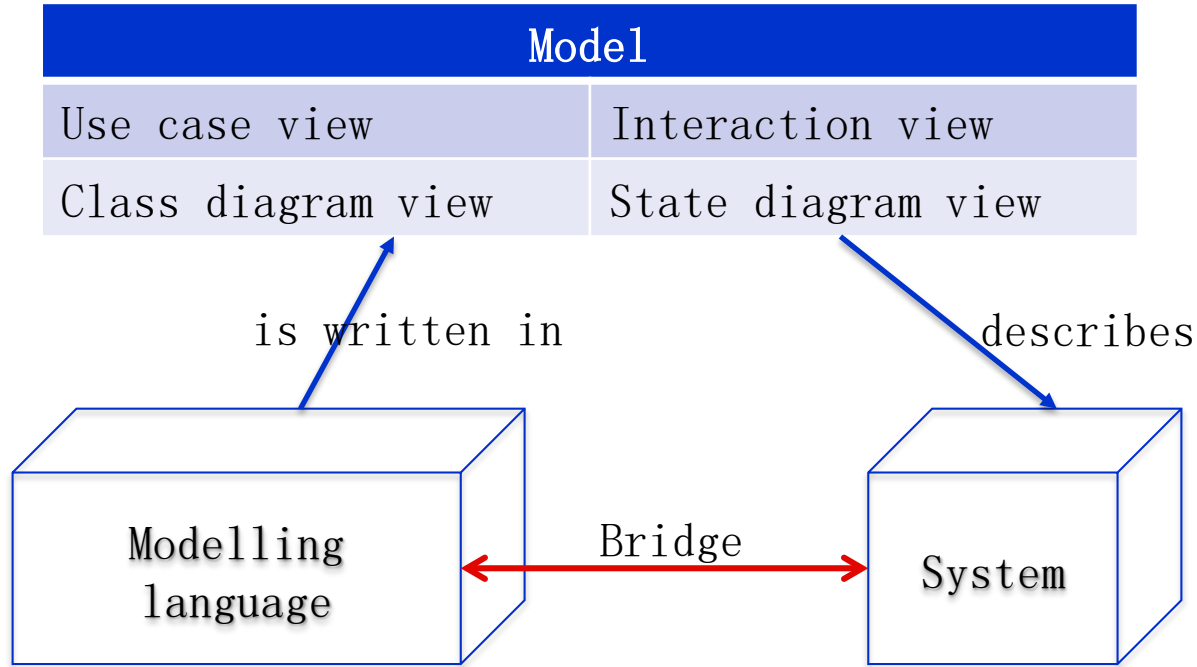
UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Modelling Language

- Process (discussed in the previous lecture)
- How to implement?
 - Modelling language
 - a well defined (modelling) language describes a system, which has clear syntax and semantics suitable for both human and computer interpretation (standard)
 - model design (connected with actual coding)



Modelling Language



UML

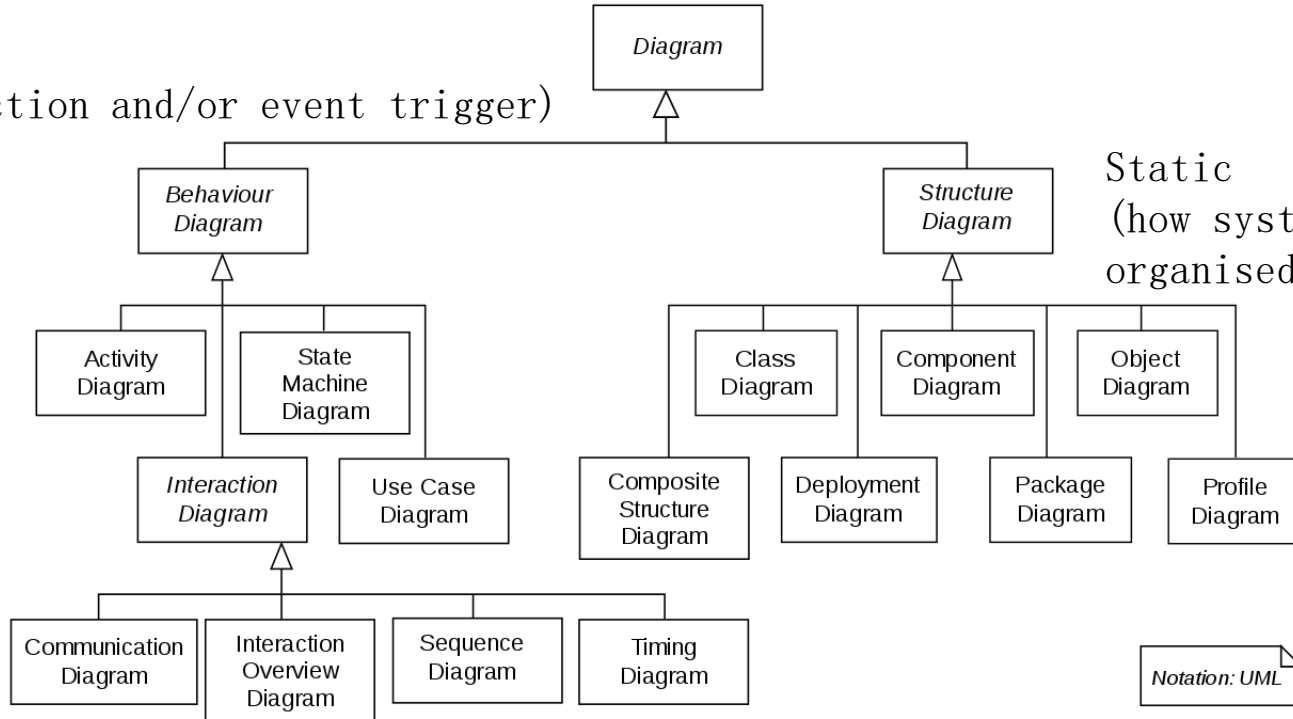
- Unified modelling language (UML) is a general-purpose modelling language that is used to visualize the design of a system
 - activities
 - individual components and interaction with other components
 - how the system will run
 - external user interface



UML

Dynamic

(user interaction and/or event trigger)



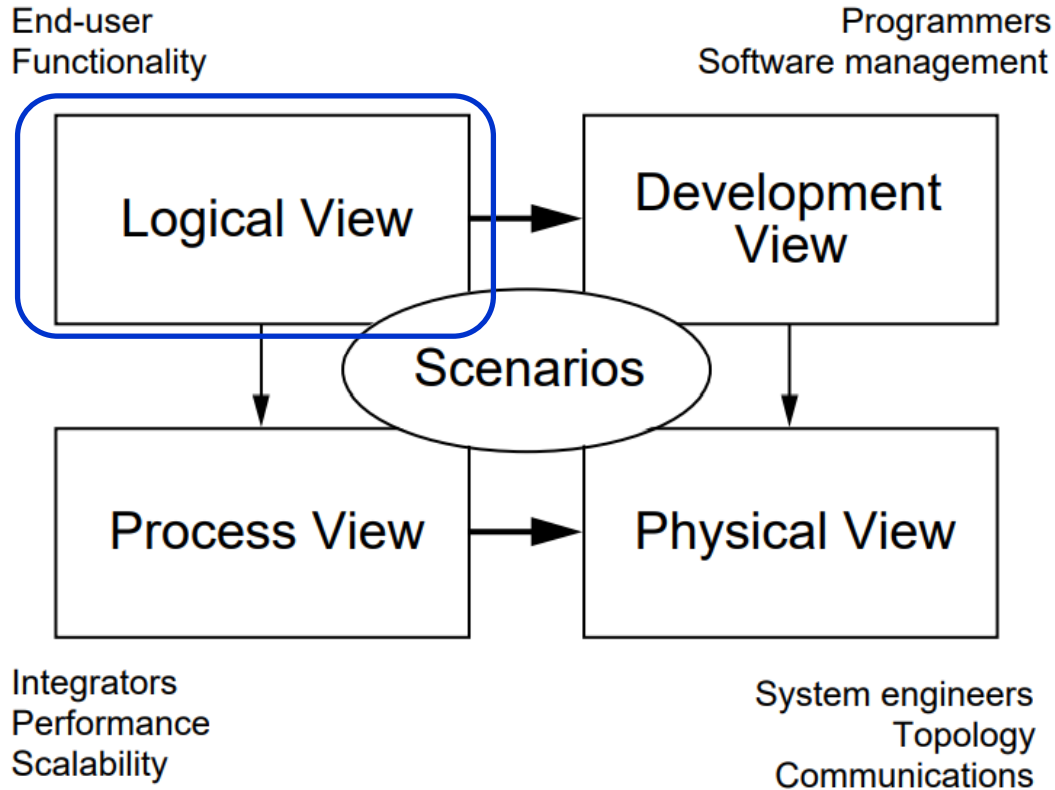
Static
(how systems being organised)

Notation: UML



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

“4+1” View



- Various ‘stakeholders’
- Functional and non functional requirements
- Conceptual and Physical
- 1 is for use case view, or known as scenario (user requirement)

Diagrams and views

- Logical:
 - Class, object, package, composite structure, state
- Process:
 - Communication, activity, interaction overview
- Development:
 - Component
- Physical:
 - Deployment



Conceptual and physical

- Logical View and Process View
 - conceptual level
 - used from analysis to design
 - the logical view is concerned with the functionality that the system provides to end-users
 - the process view deals with the dynamic aspects of the system, how they communicate, and focuses on the runtime behavior of the system
- Development and Physical View
 - physical level (software and hardware)
 - represent the application components as built and deployed



Use case view

- Use Cases model the non/functional requirements of a system
 - one or more Actors interacting with one or more Use Cases
 - Actor is a user or external system
 - Use Case is a typical interaction between an actor and the system
 - Use Cases specify the behavior from user' s point of view,
 - As well as non-functional requirements (Performance, Security, ...)



Example

Use Case: User Authentication

Actor: Bank Customer

Description: The user wants to log in to the mobile banking application using their credentials.

Basic Flow:

The user opens the mobile banking application.

The user enters their username and password.

The application validates the user's credentials against the bank's authentication server.

If the credentials are valid, the application grants access to the user's account.

The user is redirected to the main dashboard of the mobile banking application.

Alternative Flows:

If the user enters invalid credentials:

Then blabla.

If the user forgets their password:

Then blabla

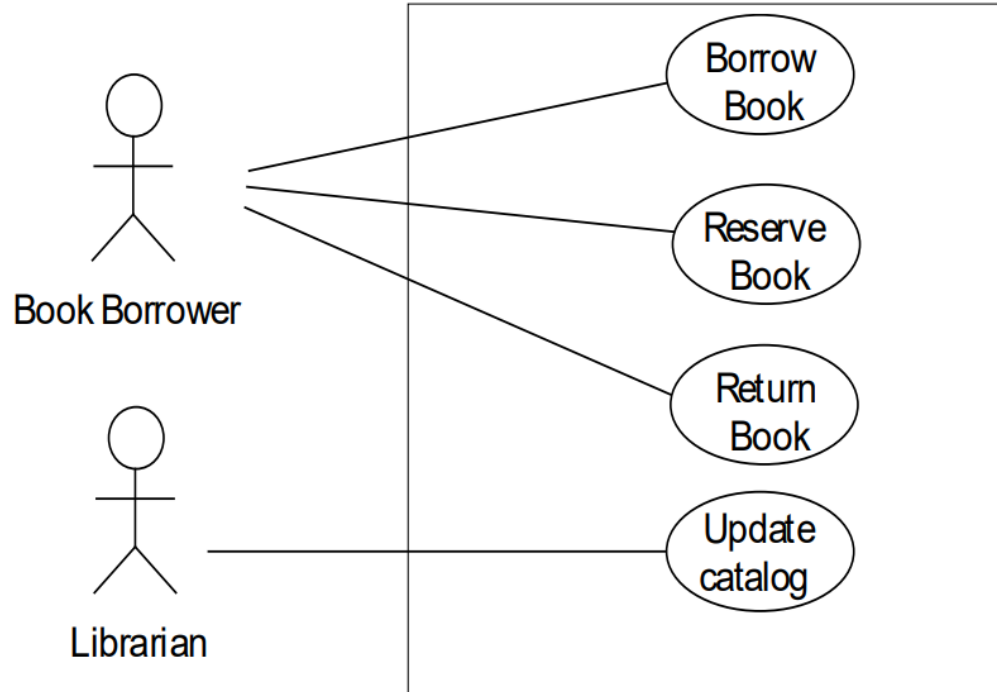


Use case or story

- **Level of Detail:** Use cases are typically more detailed and structured, providing a comprehensive view of system behavior. User stories, on the other hand, are concise and focus on the user's needs.
- **Format:** Use cases follow a standard format with specific sections, while user stories are often written in a more informal, conversational style.
- **Scope:** Use cases may cover multiple interactions and scenarios related to a particular feature or functionality, while user stories focus on individual user needs or goals.



Use cases



Actors and Use Cases

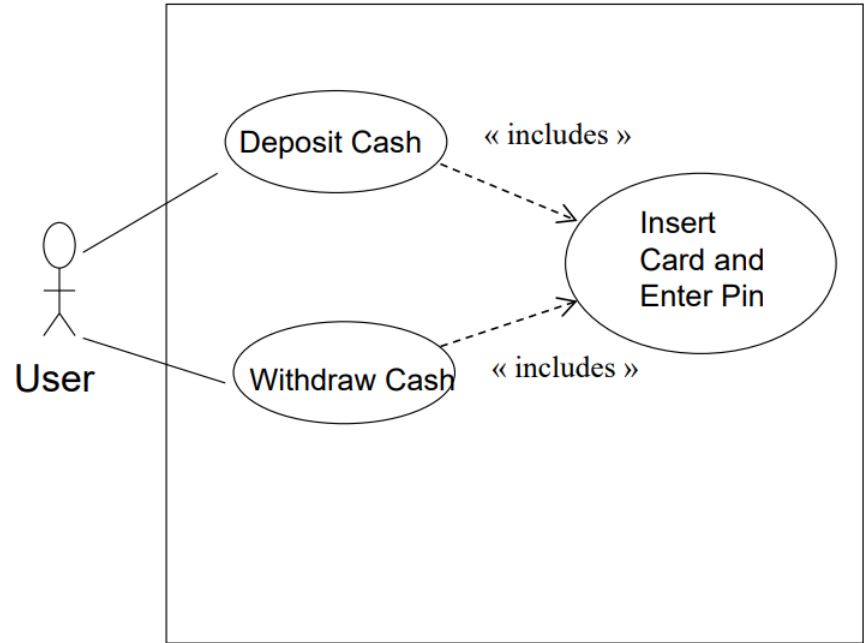
- 1:1
- 1:m
- m:m

A boundary box indicating the scope of the system

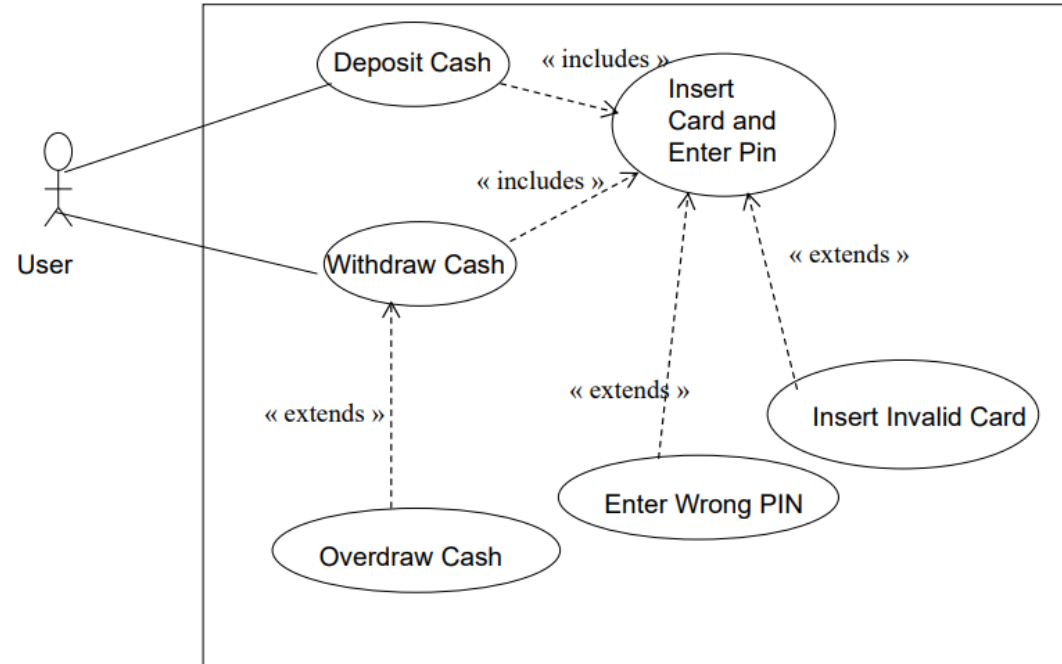


Use Case Relationship

- Use case depends on each other (such as sequence)
- Includes Relationship
 - compulsory of Use Cases



Use Case Relationship

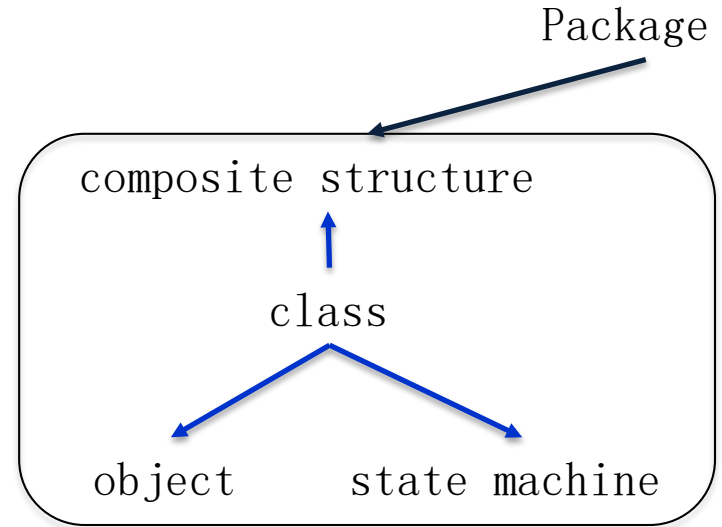


- Extends Relationship
 - optional of Use Cases
 - alternative courses of the same interaction
 - only occur under certain conditions



Logical view

- the organization of software components, their relationships, and the flow of data or control between them
- Main diagrams:
 - class diagrams: model the system
 - package diagrams: logically group different class diagrams
 - Optional diagrams :
 - Object: explain the class relationship via instances
 - State Machine: explain internal states of a specific class
 - Composite structure: model the relationship between parts of a class



UML class symbol

Person (class name)

empty

- Class symbols are the building blocks of class diagrams
- The most basic class symbol is a box with the class name
- Adding in more details may be needed at later stages
 - Attributes, or data types
 - Methods
 - The level of details depends on the related class diagram



UML class symbol

Person

```
- name:String
- age:int
- height:int
- rate:int {read only}

+ getName():String
+ setName():void
+ getHeight():int
+ getAge():int
+ setAge():void
+ main():void
```

Prefix attributeName: Data type Prefix
methodName(): Return data type

Prefixes:

+ : public attributes and methods

- : private attributes and methods

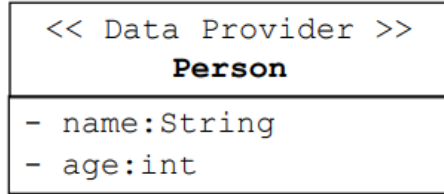
: protected attributes and methods

{read only}: constant

____ (underline): static attributes and methods



UML object symbol



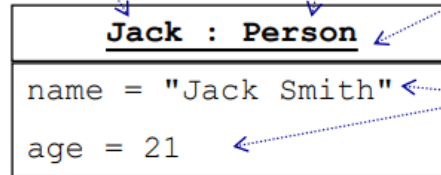
A class



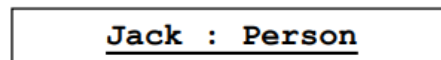
object name

class name

underline



Specific values for
each attribute
(object state)



A simplified UML representation
when state is not important



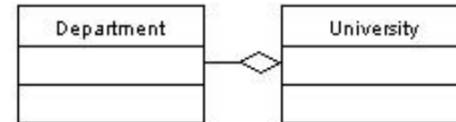
Logical View: class relationships

- Relationships between objects:
 - Aggregation: one object is a part of another object.
 - Composition: cannot exist independently
 - Inheritance: one object is a more specialized version of another object. This is also called specialization.
 - Association: The objects need to communicate (basically everything else – the default option)



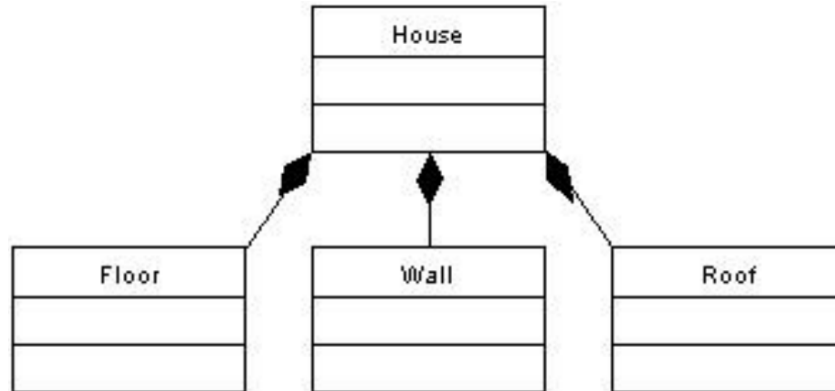
Aggregation

- an 'assembly-component' (i.e. an "is-a-part-of") relationship
- assembly object is a single object made up of component objects.
 - a department IS PART OF a university
- These things can also exist independently



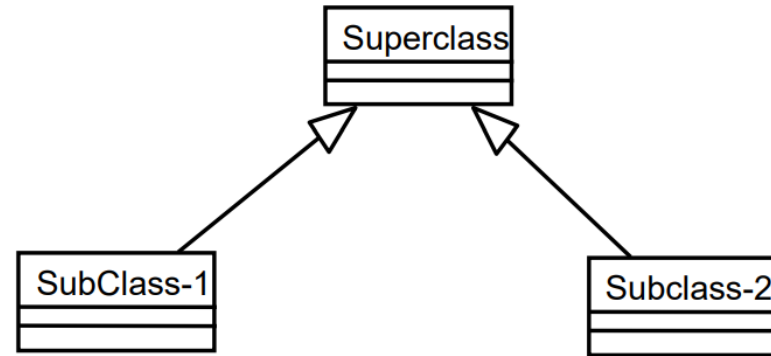
Composition

- ____ is composed of ____
 - E.g. a house is composed of a floor, roof and 4 walls
 - if the house is destroyed, then the other things are also destroyed



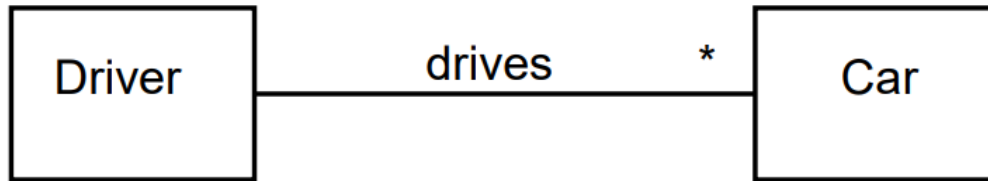
Inheritance

- Generalization/specialization
 - subclasses describe properties that are specializations of a superclass
 - an “is-a” relationship
 - all properties of superclass are inherited by subclass (attributes, operations and associations)



Association

- Associations are the most generic relationships.
 - Any relationships which do not fit into the other categories are association.
 - These relationships often are of the type ‘has-a’ , ‘uses’ , ‘communicates with’ or ‘makes requests of



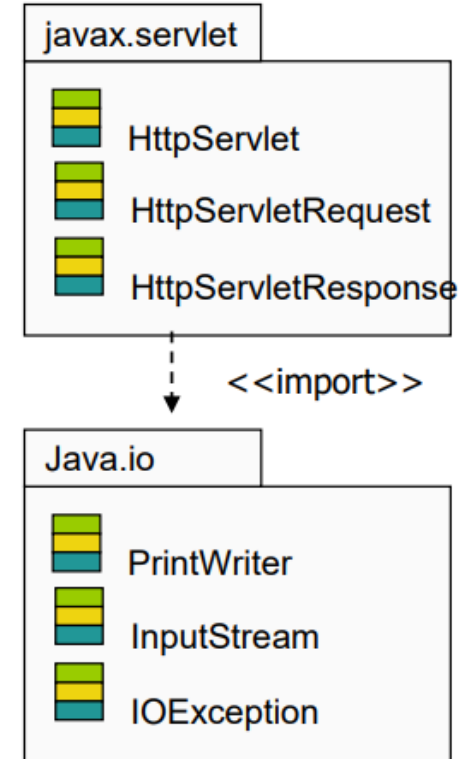
Logical View: Package diagram

- Similar to Java packages (or their equivalents in other language)
- It simply identifies a group of classes with a namespace
- Typically shows the public classes only

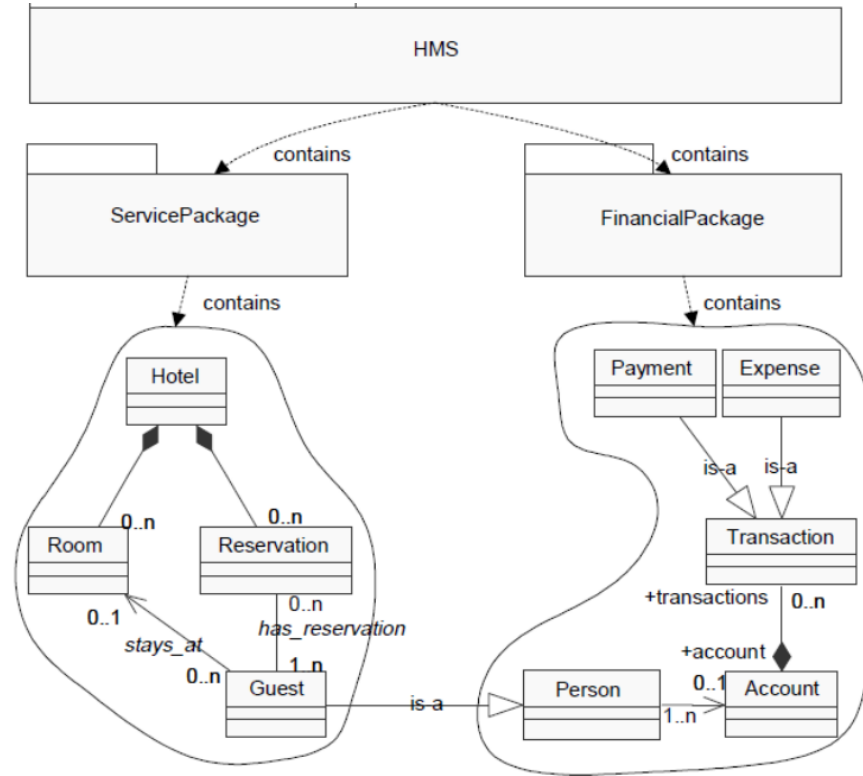


Package diagram

- Simplify UML diagrams by grouping related elements into a single higher-level element
 - such as diagrams, classes, or even other packages
- A dependency relationship
 - if a package is changed, the dependent package also could have to be modified



Package diagram benefit



Package diagram benefit

- **Modularization:** Package diagrams help in breaking down a system into manageable modules based on related functionality.
- **Hierarchical Representation:** They visually represent the nesting of packages, showing the system's architecture at different levels of abstraction.
- **Dependency Management:** Package diagrams illustrate dependencies between packages, aiding in effective management and reducing coupling.
- **Reusability:** Packages promotes reusability and enforcing modularity and separation of concerns.
- **Documentation and Communication:** They serve as visual documentation, facilitating communication and collaboration among stakeholders.



Logical View: State diagram

- A dynamic model that shows the different states (status) that a single object during its lifetime in response to events (triggers).
- What is a state?
 - It is a set of values (of attributes) that describe an object at a specific point in an object' s life
 - student' s program, student' s semester, etc. determine a state of the Student object
 - Some of the objects in the system are quite dynamic in that they pass through a variety of states over their existence
 - A student can change from “new” to “current” to “former

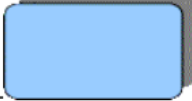


State Diagram: Event

- What is an Event (trigger)?
 - something takes place at a certain point and changes a value(s) that describe an object (the state of an object)
 - the student graduates from the university is an event (that changes the Student' s state from “current” to “former”)
 - concurrent events: two events that are related only by coincident and have no effect on each other
 - 2 planes take off at the same time



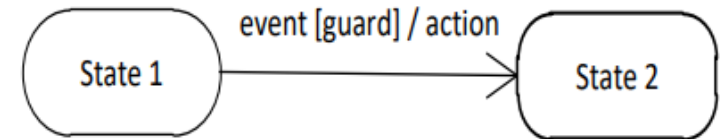
State Diagram

Term and Definition	Symbol
A State Is shown as a rectangle with rounded corners.	
An Initial State Is shown with a small filled in circle	
A Final State Is shown with as a circle surrounding a small solid filled-in circle. This represents the completion of activity.	
An Event Is an occurrence that triggers a change in state. It is used to label a transition.	Event Name
A Transition Indicates that an object in the first state will enter the second state. Is triggered by the occurrence of the event labeling the transition. Is shown as a solid line.	



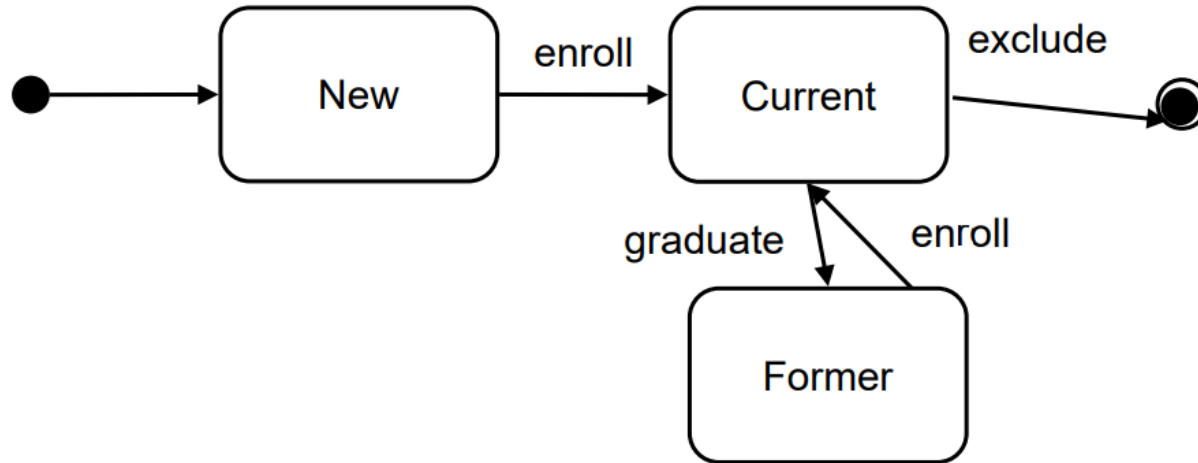
Transitions

- A relationship between 2 states
 - an object (from one state) will perform certain actions and enter the other state when a specific event occurs
- Source State
 - object receives the event and then it may “fire”
- Event Trigger
 - the event whose reception by the object makes it eligible to “fire”
- Guard Condition – An optional expression to see if it should “fire” .
 - “when you go out in the morning (event), if the temperature is below freezing (condition), then put on your gloves (actions)” and move to next state.
- Action
 - an executable atomic computation
- Target State
 - the state that is active after the completion of the transition
- Guard Condition and Action are optional



Example

- Different states of a student



U

O

W

CSIT314

Software Development

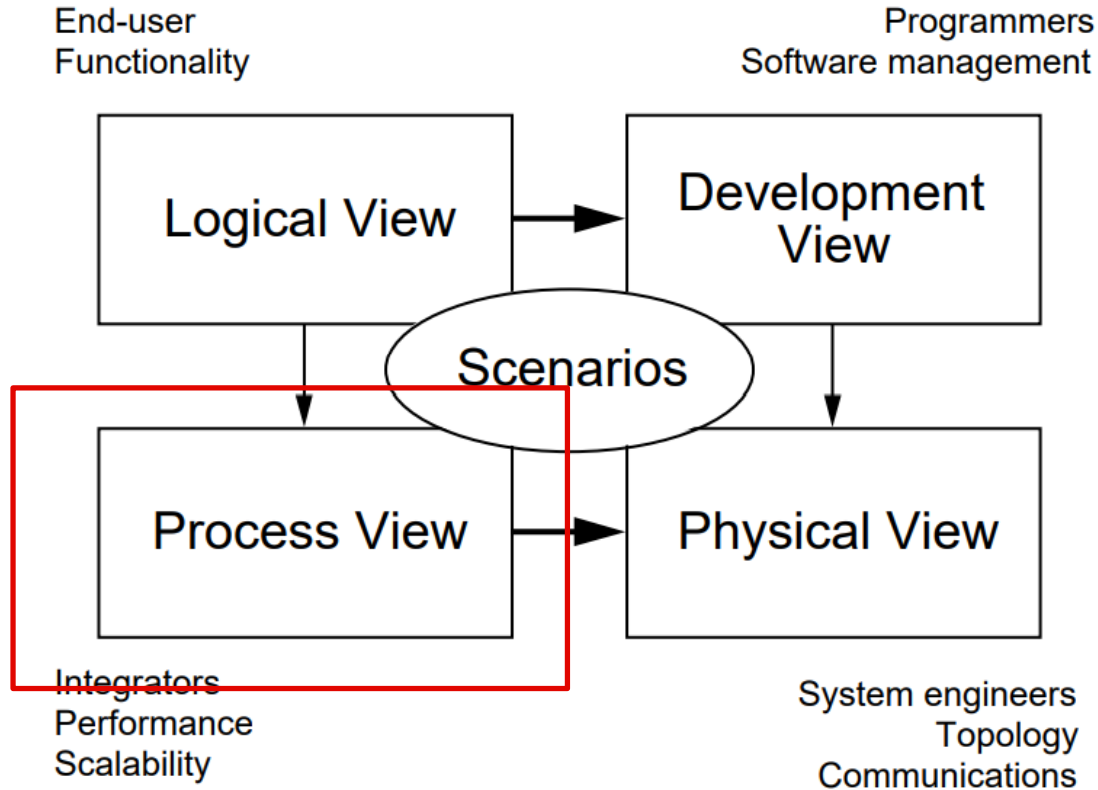
Methodologies

Advanced Unified Modelling Language



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

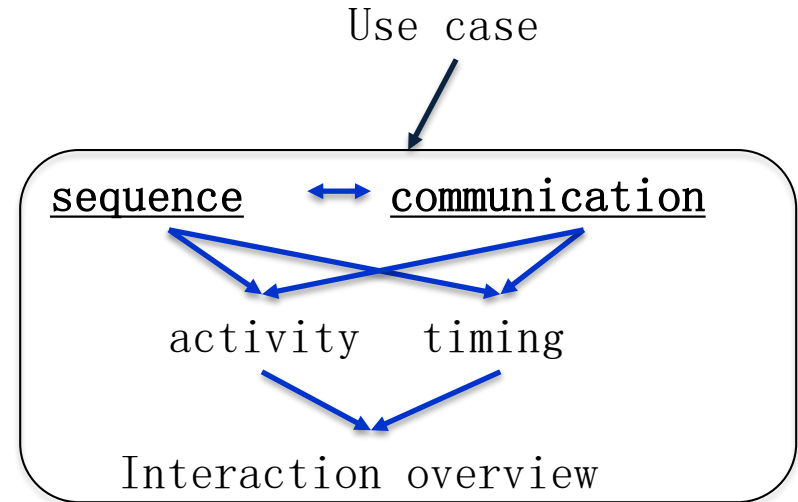
“4+1” View



- Conceptual and Physical
- We discussed logical view so now process view

Process View

- Model the dynamic aspects, communication, and runtime behavior of the system
- Use either sequence or communication diagrams to model interactions in use cases
- Optional:
 - Activity (branching or parallel)
 - Timing
 - Interaction overview (simply other scenarios)



Sequence diagram

- Sequence Diagrams show how classes/objects (or groups of objects) collaborate in some sequential behavior.
 - Which means class diagram is done!
- Used to model how objects interact together.
- Generally a Sequence Diagram describes one Use Case.



Sequence diagram

- Message exchange
 - Communication between objects is achieved by exchanging messages
 - The objects send messages to each other
 - These messages lead to the operations, which means that an object understands or responses precisely those messages for which it has operations

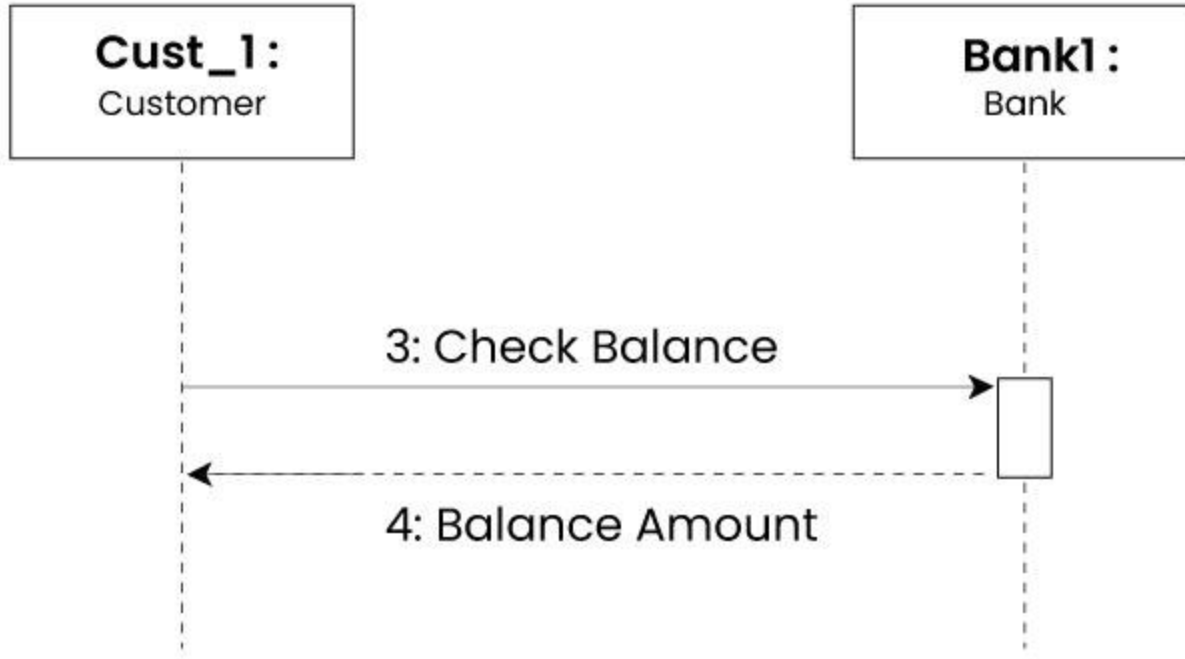


Sequence diagram

- Graphically, a sequence diagram is a table that:
 - shows **classes/objects** arranged along the X axis
 - messages, ordered in increasing time, along the Y axis.
- The object that initiates the action is placed on the left and other increasingly subordinate objects to the right.



Sequence diagram



Sequence diagram

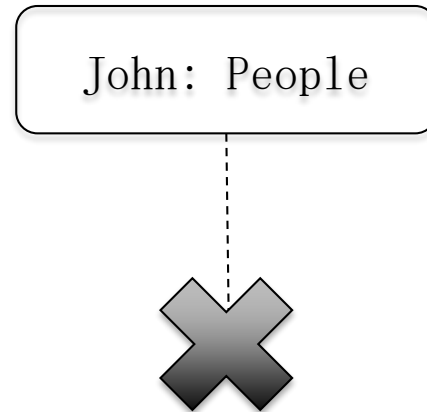
- Class/Objects
 - Shows instance name and/or class name
(instance_name:class_name)

John: People



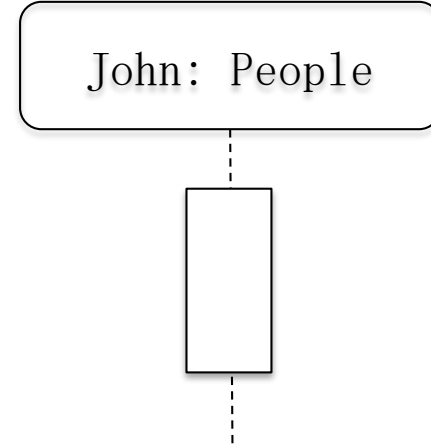
Sequence diagram

- A lifeline shows the life of an object
- The X(cross) shows the point at which the object is deleted

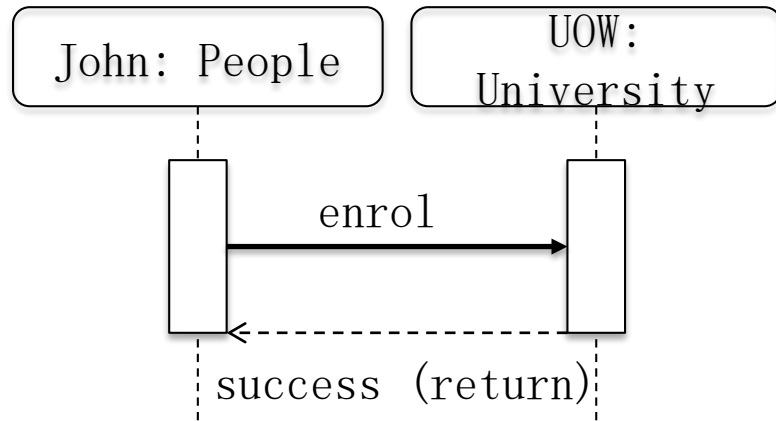


Sequence diagram

- Focus of control
 - a long, hollow, narrow rectangle placed over a lifeline
 - shows object has control (sending or receiving messages)



Sequence diagram



- Message
 - Shown by a line with a filled in arrow head
 - Message name as the class method
 - Private methods as messages
- Message return (optional)
 - Shown by dotted line with empty arrow head



ATM Example

ATM

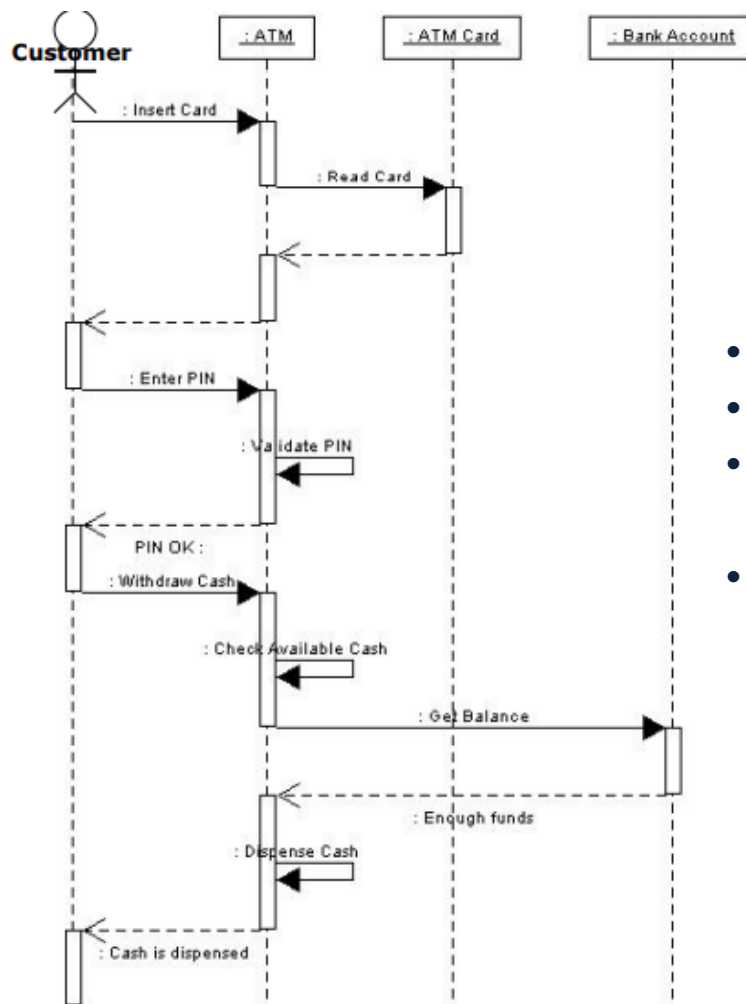
```
+InsertCard()  
+EnterPIN()  
+ValidatePIN()  
+CheckAvailableCash()  
+WithdrawCash()  
+DispenseCash()
```

ATMCard

```
+ReadCard()
```

BankAccount

```
+GetBalance()
```



- Use case + class
- Sequence
- Coding work
- Notes:
 - add new sequence diagram
 - change existing



So far...

- Use case: system functionality or user requirement
- Class/object: implement use case
- State: events change object status
- Sequence: model objects' sequential behavior

- How to design?

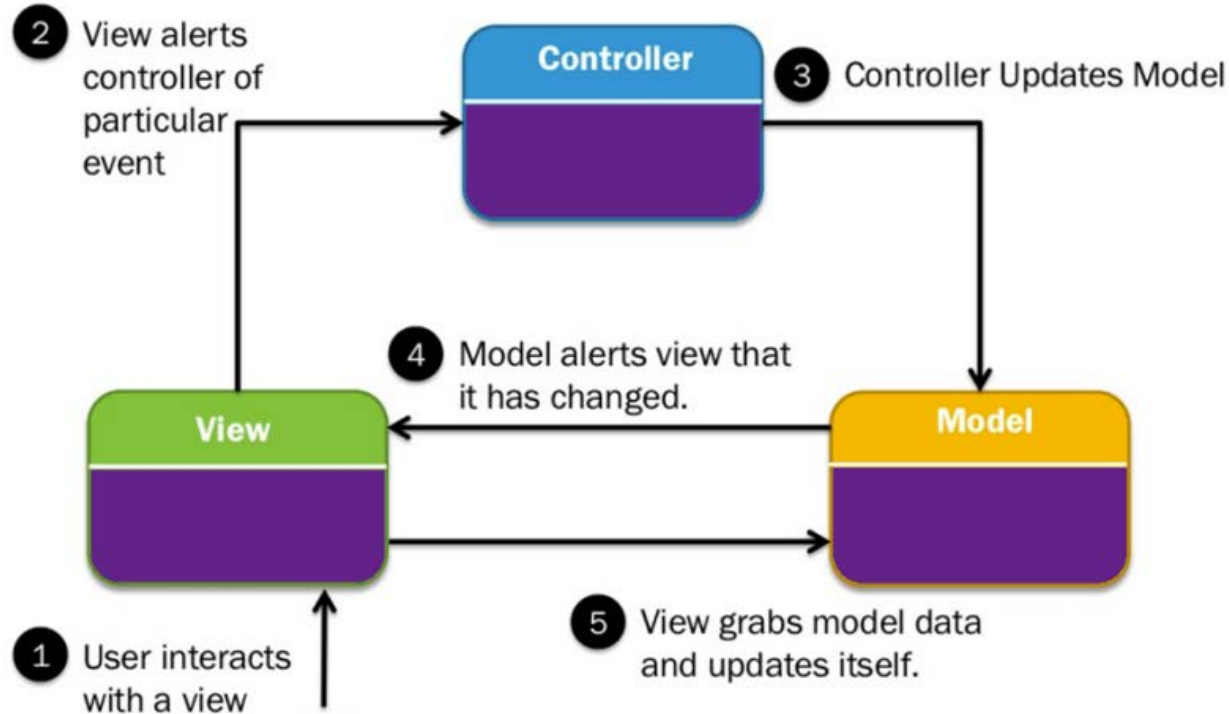


B-C-E design

- Consider applying the B-C-E model (also known as MVC pattern)
 - Entity layer (Model)
 - Holds the data and is responsible for its persistence.
 - Class, attributes, methods
 - Control layer (Control)
 - Contains the rules about how to combine info and how to deal with interaction
 - A third-party coordinator(sequence diagram)
 - Boundary layer (View)
 - Responsible for representing info to user and to receive user interactions.
 - User interface (like front-page, mobile app)



B-C-E design



B-C-E design

- Boundary layer (View)
 - The interaction between the user and the system
 - A boundary class does not provide the actual behavior of the use case – it typically represents the GUI – e.g. dialog, menu, screen.
 - Actors interact only with boundary classes, to perform the use case



B-C-E design

- Control layer (Control)
 - The main *use case* controller class to coordinates all the messaging between the entity classes
 - The use case controller implements the sequence diagram messaging.
 - They are the “glue” between boundary and entity classes - i.e. they connect the user with the data and logic
 - They contain the application-specific business rules
 - A student should enroll in 4 subjects per semester; a customer cannot withdraw more than \$800 per day.
 - One control class per use case (some use cases may not need one)
 - They are created when the system performs the use case and usually die when the use case is performed



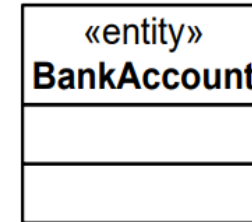
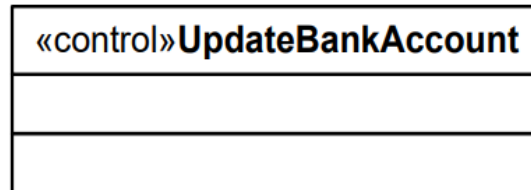
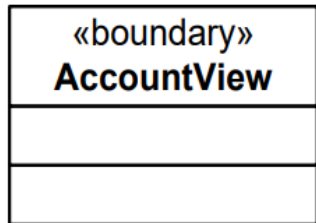
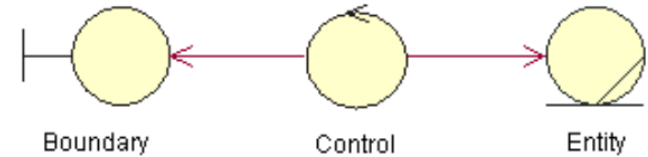
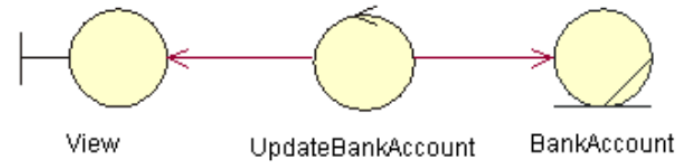
B-C-E design

- Entity layer (Model)
 - Coding work
 - Represent business domain concepts - you will usually have identified many in the domain class diagram
 - Entity classes used to store and manage information in the system - they map directly to database tables
 - Some entity classes may be transient - i.e. they do not get stored in DB but are only temporary



B-C-E design

- 2 representations:
 - Circle for class, line for association between class
 - Add prefix

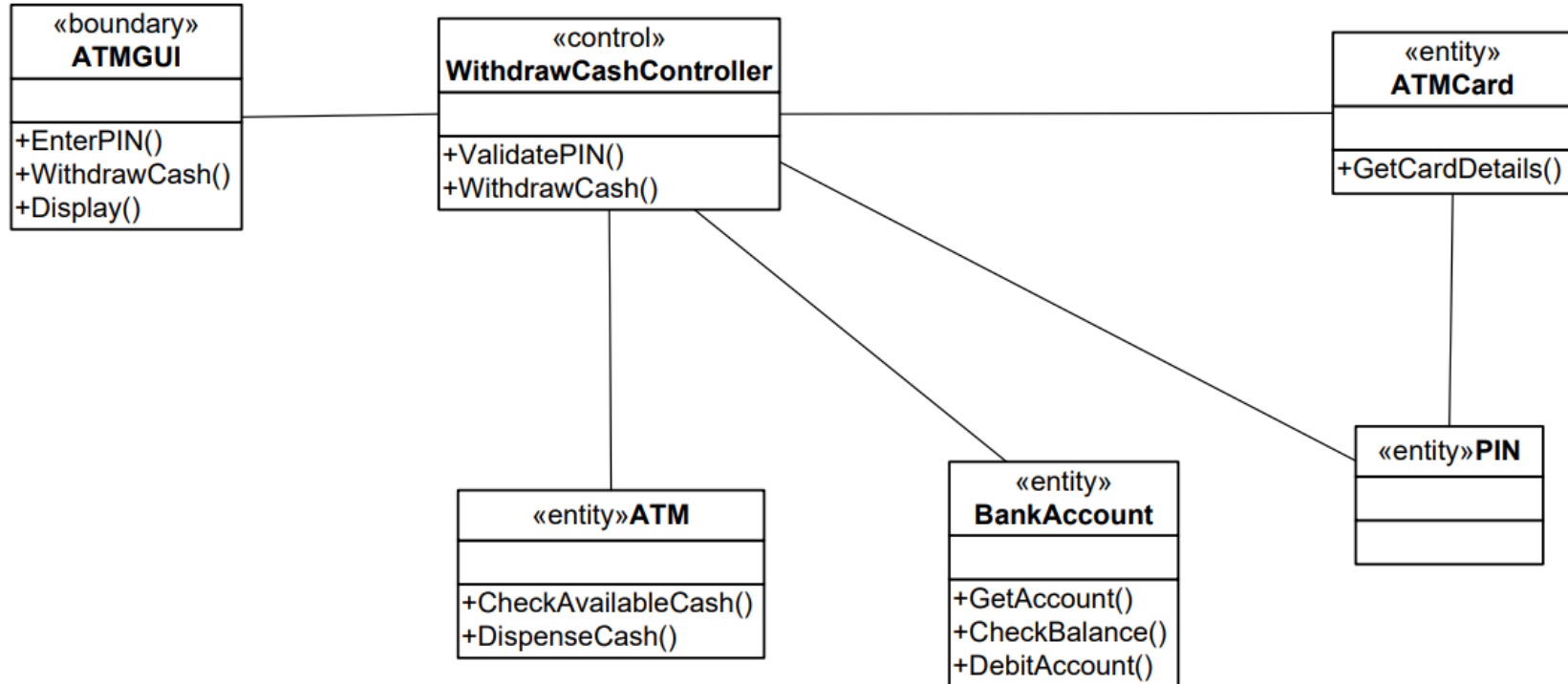


B-C-E design

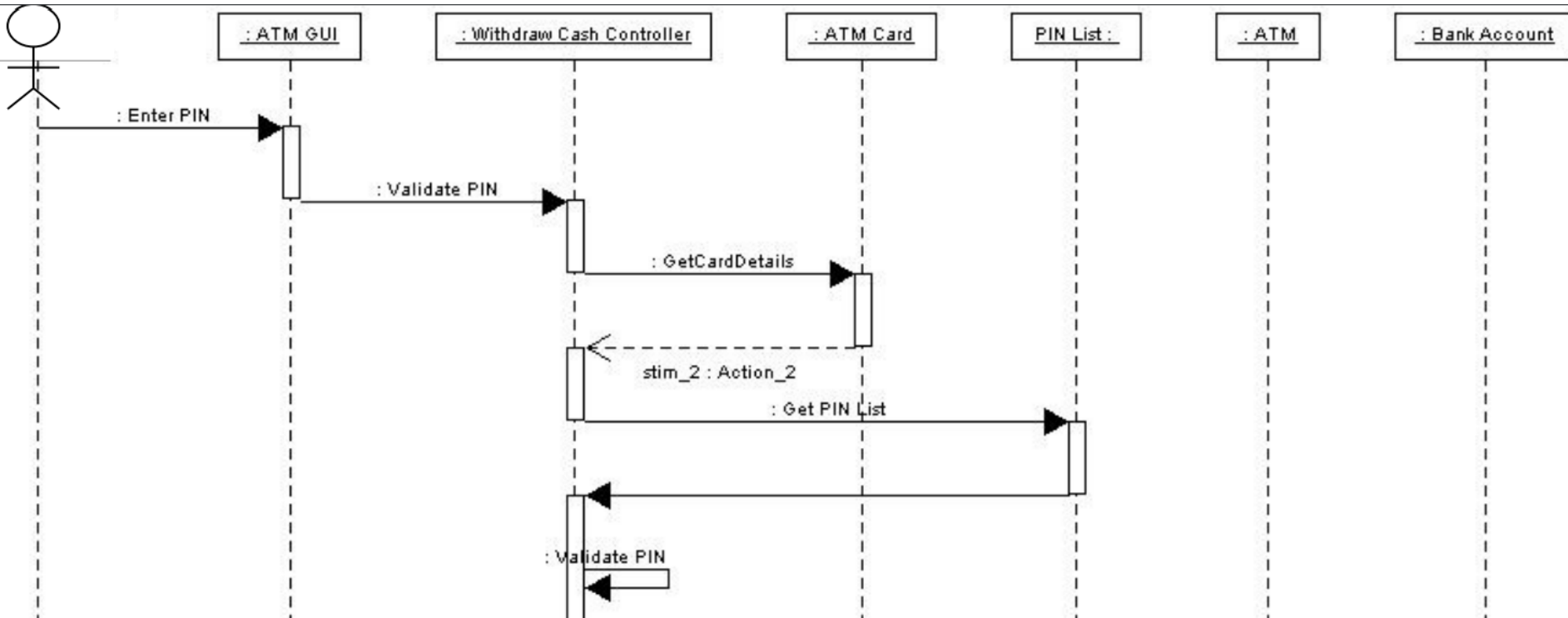
- Applying the B-C-E framework
 - Take an ATM system. A use case has been identified called Withdraw Cash.
 - A class diagram models this system. The diagram is not well designed.



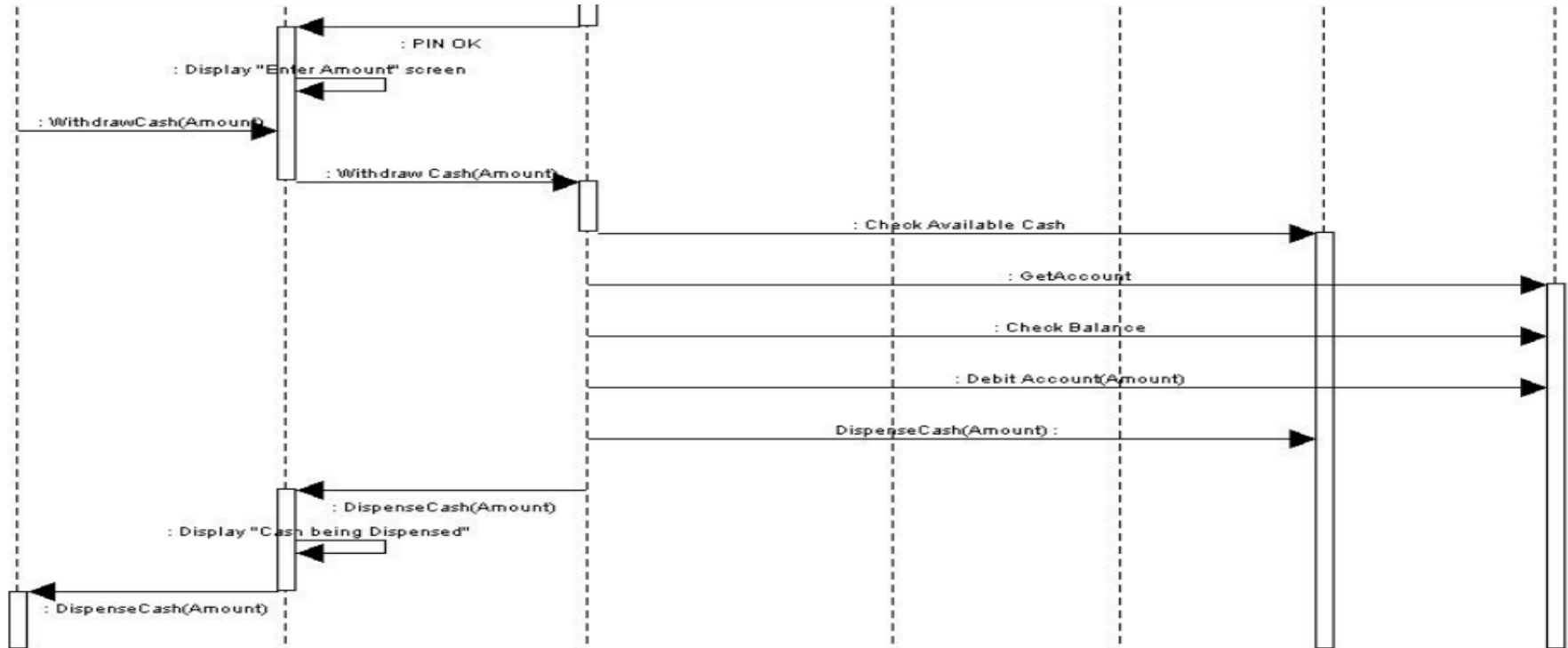
ATM B-C-E design



ATM B-C-E design



ATM B-C-E design

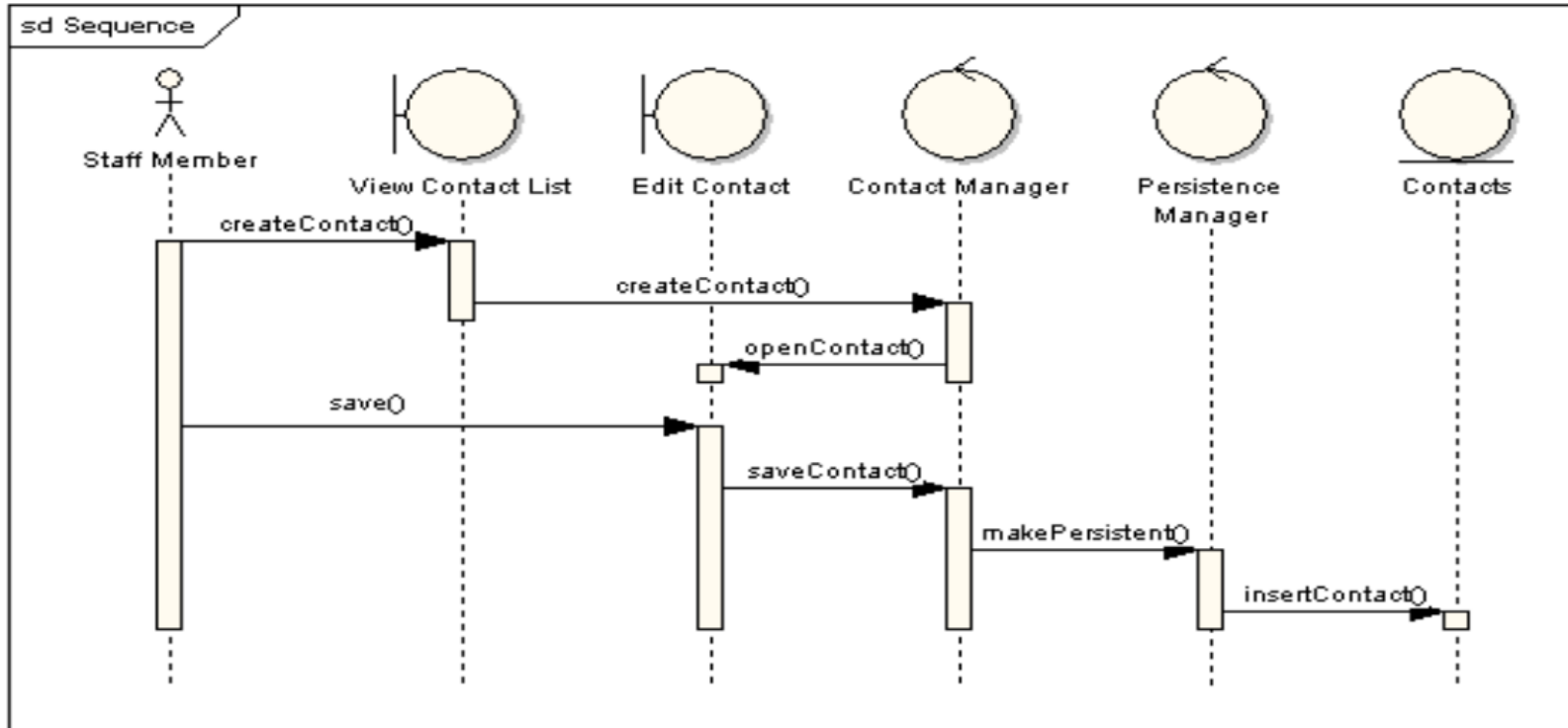


Process View: Communications diagrams

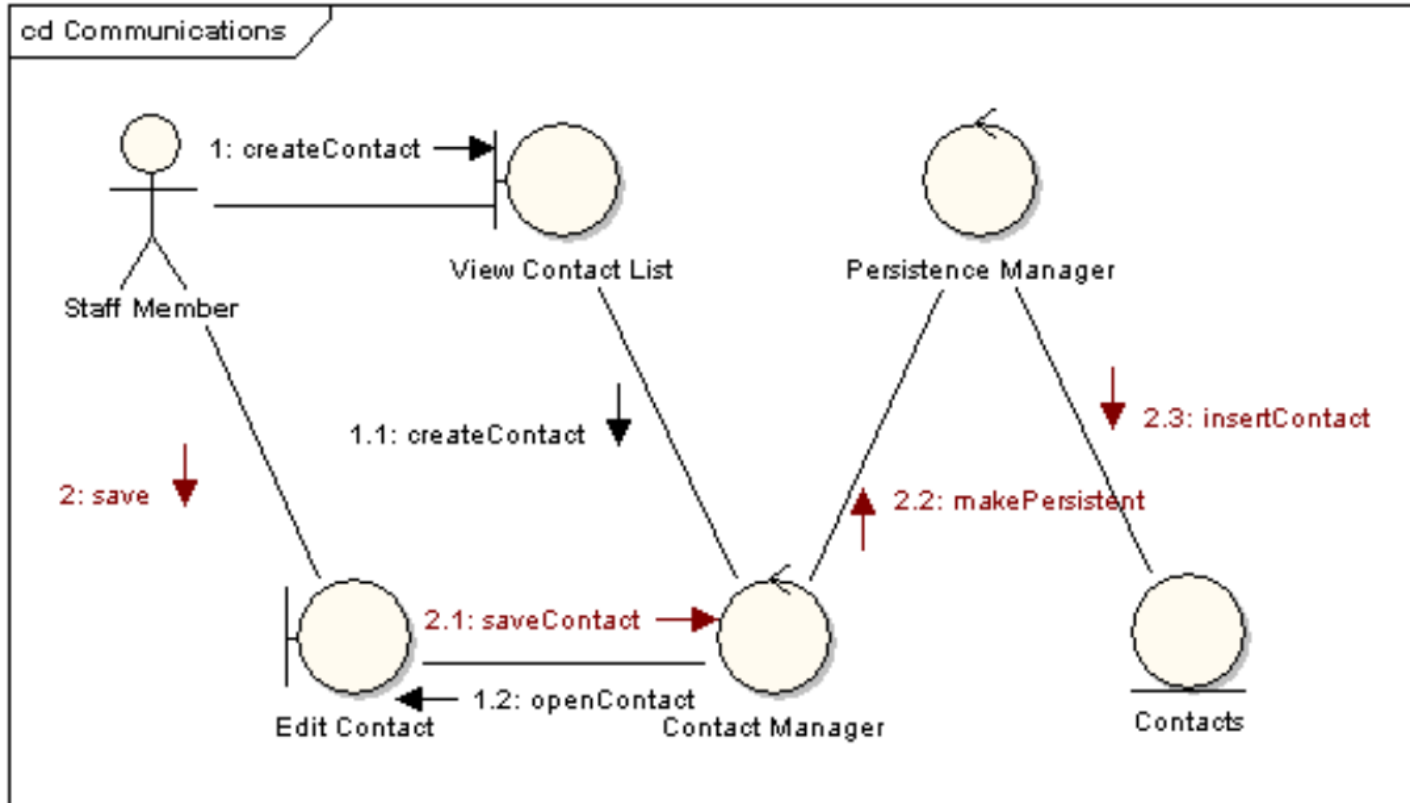
- Convey the same information as a sequence diagram - a difference in style.
 - Sequence diagrams focus on messages occurring over a timeline
 - Communication diagrams focus more on links between participating objects
 - Sequence diagrams focus on the order of messages, while communication diagrams make object relationships clearer.



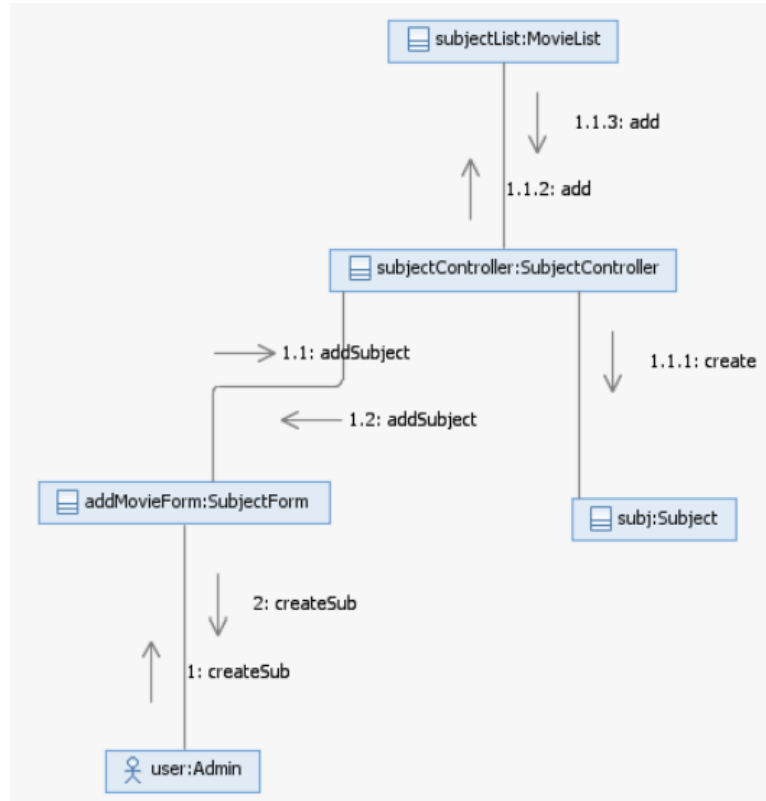
Sequence vs. Communications



Sequence vs. Communications



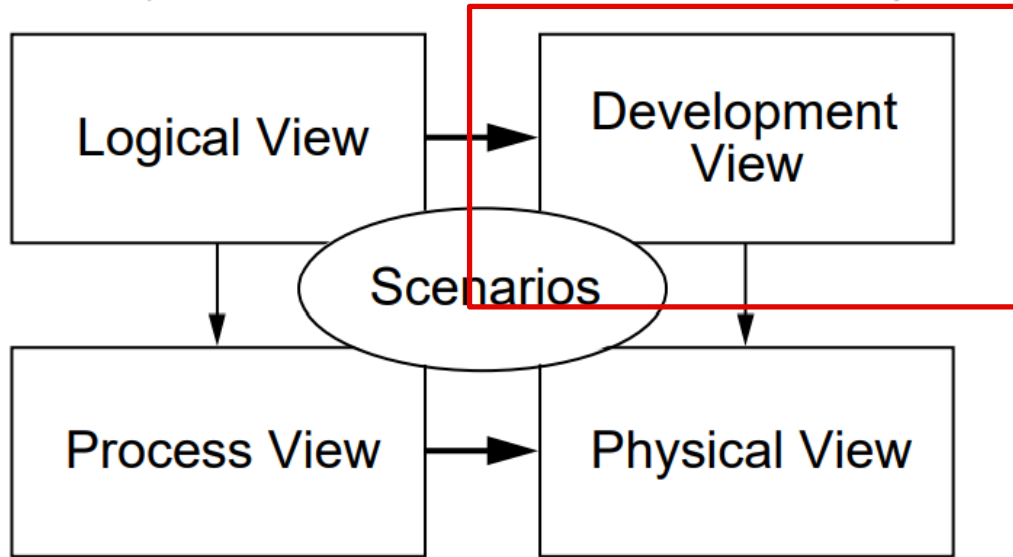
Communications diagrams



“4+1” View

End-user
Functionality

Programmers
Software management



Integrators
Performance
Scalability

System engineers
Topology
Communications

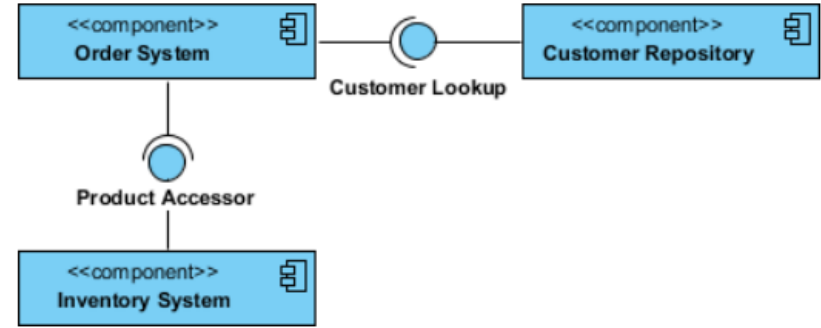
- Conceptual and Physical
- We discussed conceptual view so now physical view

Development View

- Component diagrams illustrate the pieces of software, embedded controllers, etc., that will make up a system.
 - show the relationship between different parts
 - organise the system (show subsystems)
- The term “component” refers to a module of classes that represent independent systems or subsystems with the ability to interface with the rest of the system.
- A component is usually implemented by one or more classes (or objects) at runtime.



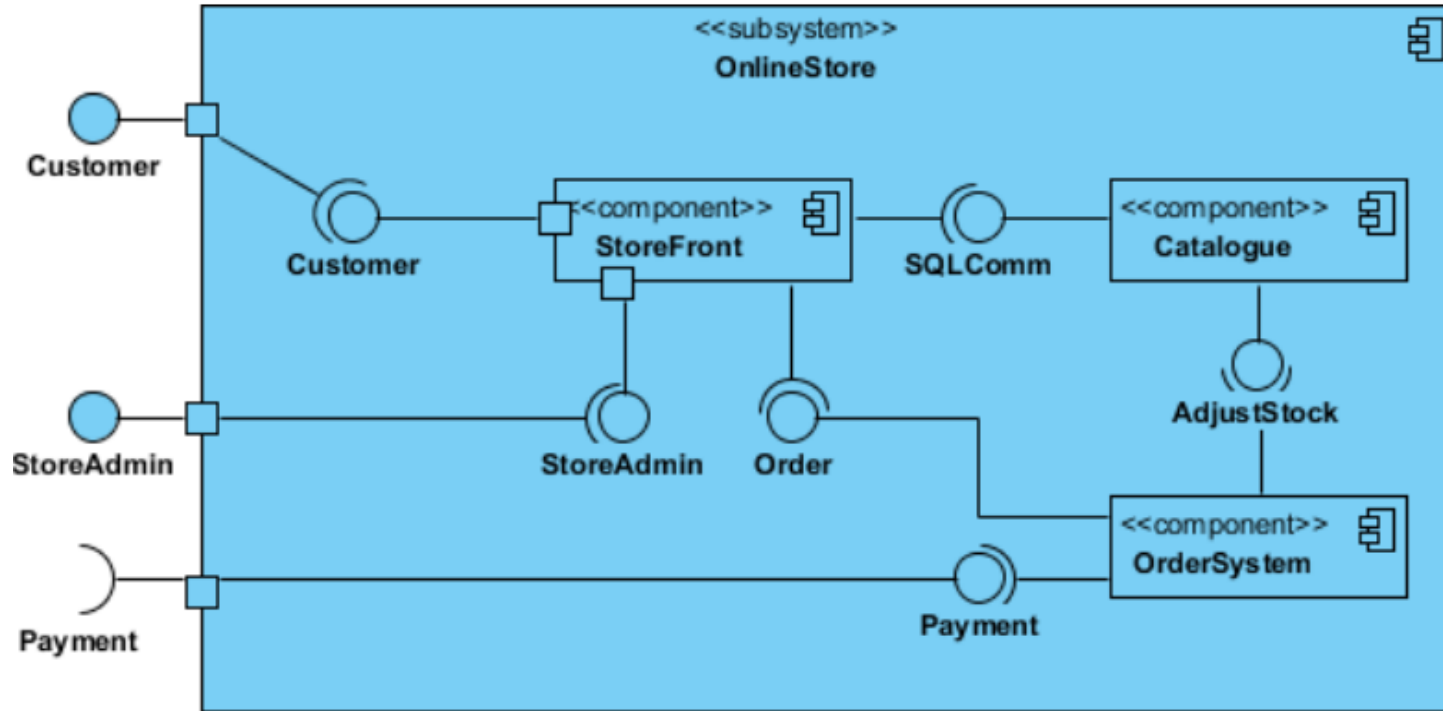
Component diagrams



- Component: rectangles
 - Class or (sub)system needs to communicate with others to perform something (provide services)
 - Assembly Connector
- Provided interface symbols with a complete circle at their end represent an interface that the component provides
- Required Interface symbols with only a half circle at their end represent an interface that the component requires
 - In both cases, the interface's name is placed near the interface symbol itself.



Component diagrams



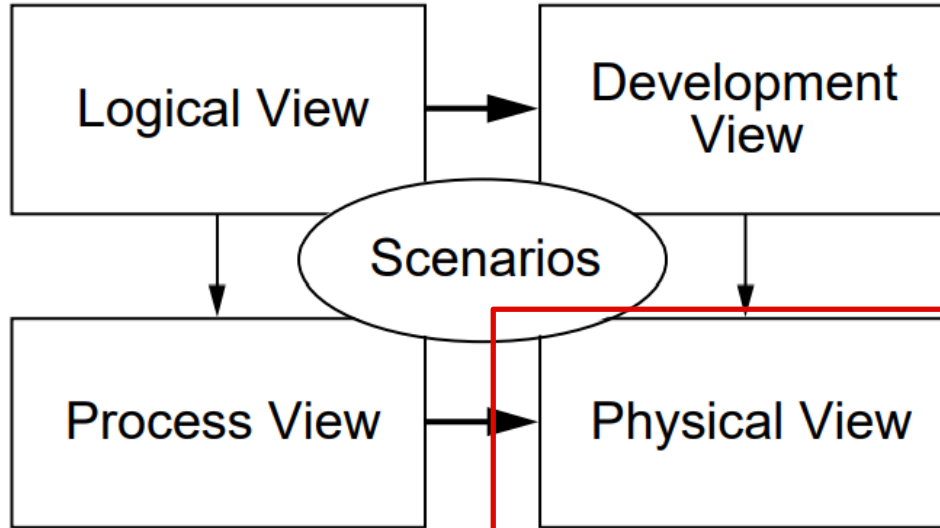
Package: physical location of codes vs component: functionality



“4+1” View

End-user
Functionality

Programmers
Software management



- Last view

Integrators
Performance
Scalability

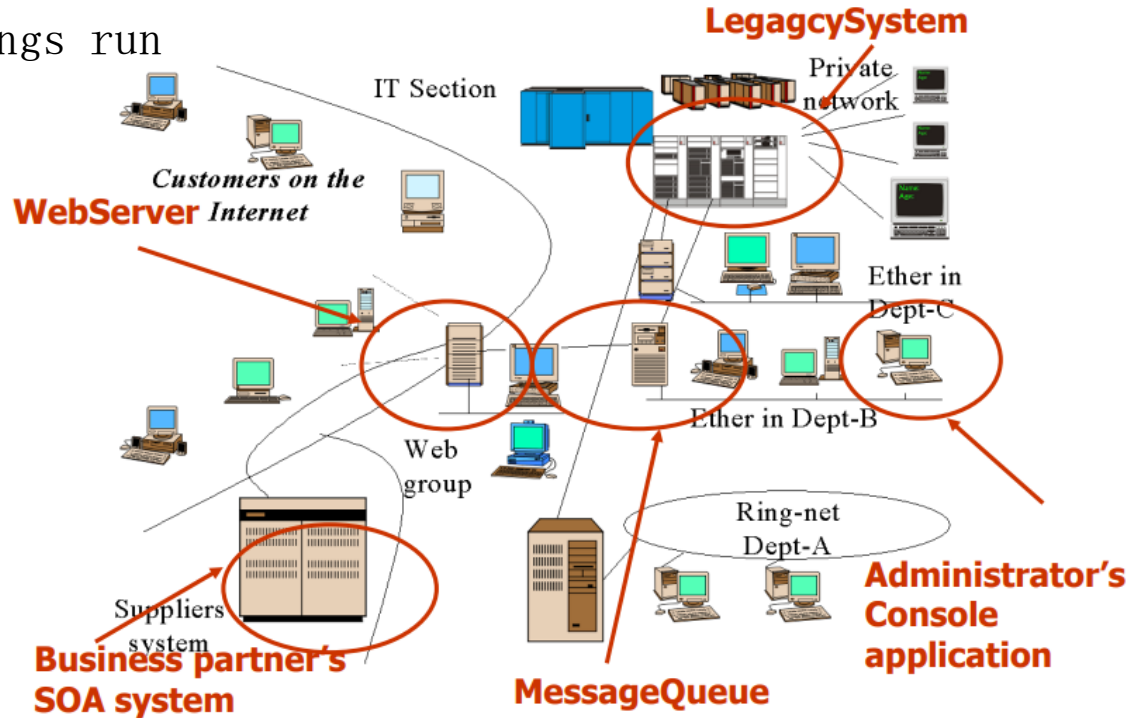
System engineers
Topology
Communications



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Physical view

Where do things run



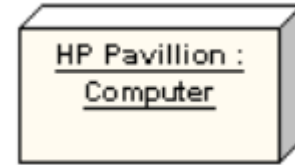
Deployment Diagrams

- Deployment Diagrams
 - models the run-time architecture of a system
 - shows the configuration of the hardware elements (nodes) and shows how software elements and artifacts are mapped onto those nodes
- A Node is a hardware or software element, shown as a three-dimensional box shape



Deployment diagram

- Node Instance: Name and base node type
- Node types
 - standard stereotypes are provided for nodes, «cdrom», «cd-rom», «computer», «disk array», «pc», «pc client», «pc server», «secure», «server», «storage», «unix server», «user pc».



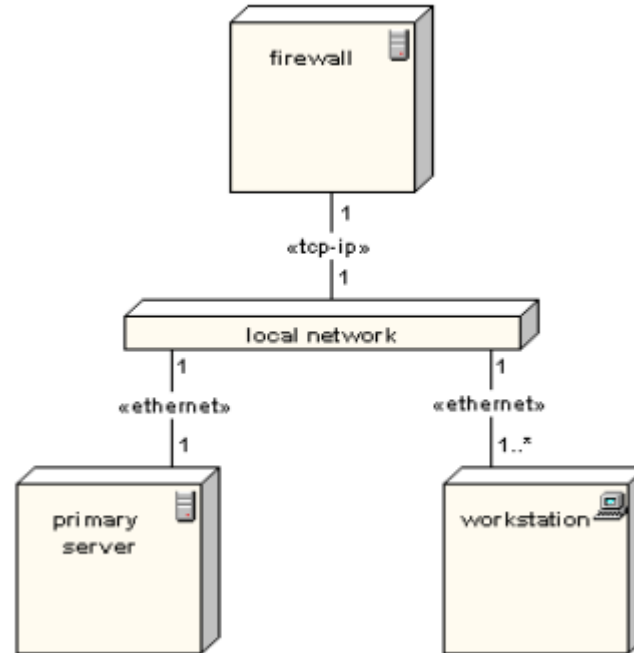
Deployment diagram

- Artifact (soft parts)
 - Any product of the software development: source files, executables, design documents, test reports, prototypes, user manuals, etc.
 - An artifact is denoted by a rectangle showing the artifact name, the «artifact» keyword and a document icon.



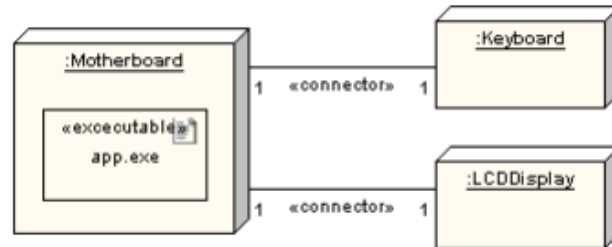
Deployment diagram

- Association
 - An association represents a communication path between nodes

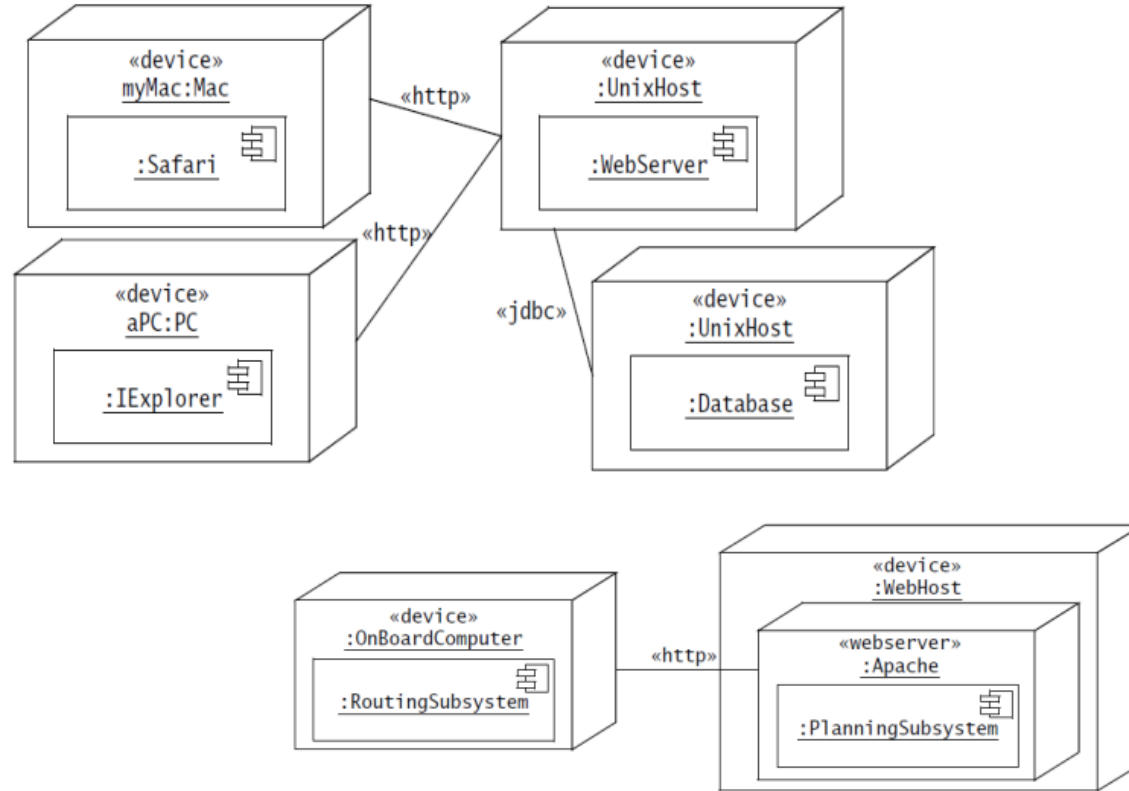


Deployment diagram

- Node as Container
 - A node can contain other elements, such as components or artifacts.



Deployment diagram



U

O

W

CSIT314

Software Development

Methodologies

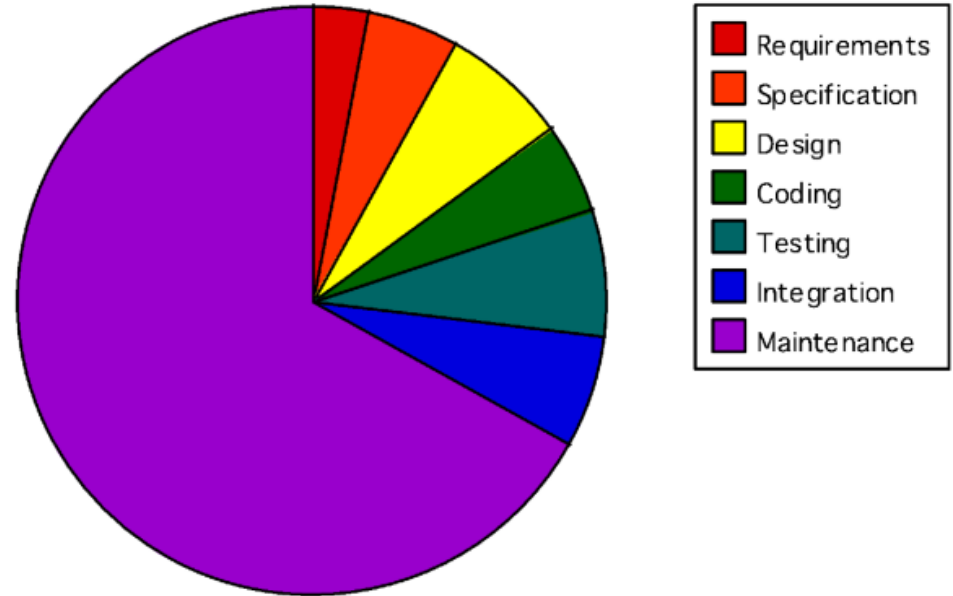
Verification & Validation and Test-driven development



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Software Development Activities

- Software Engineering plays a critical part in the software development
 - Planning
 - Requirements analysis
 - Design
 - Implementation
 - Verification & Validation
 - Maintenance and evolution



Verification & Validation

- Verification:
 - “Are we building the product right? “
 - The software should conform to its specification (such as non-functional requirement).
- Validation:
 - “Are we building the right product? “
 - The software should do what the user really requires.



Verification & Validation

- Is a whole life-cycle process
 - Can be applied during each phase
- Two principal objectives
 - To discover and rectify defects/bugs in a system
 - To assess whether the system is usable in an operational situation
- Mainly focus on 1st V

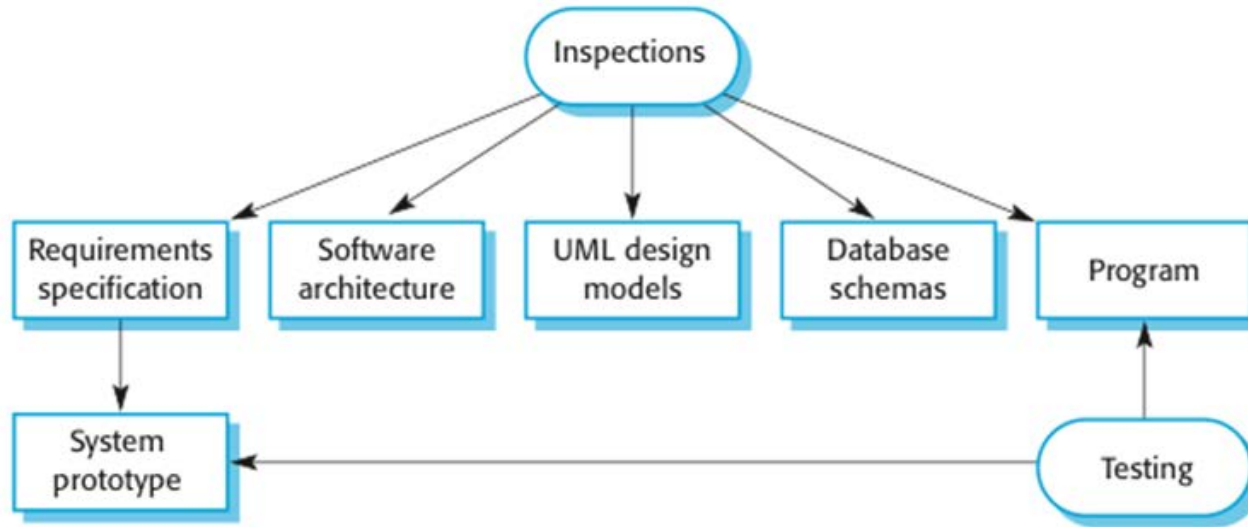


Static and Dynamic Verification

- Software inspections
 - analysis of the static system representation to discover problems (static verification)
- Software testing
 - executing and observing product behaviour (dynamic verification)
 - The system is executed with test data and its operational behaviour is observed



Static and Dynamic Verification



Software inspections

- Designs and code are reviewed by people other than the original developer.
- Inspections do not require execution of a system so may be used before implementation.
 - examine the source representation with the aim of discovering anomalies and defects
 - discovering program errors
- Applied to any representation of the system
 - requirements, design, configuration data, test data, etc.



Software inspections (example)

- Variable and Constant Declaration Defects (VC)
 - naming conventions, non-local variables, constants vs variables
- Function Definition Defects
 - function parameter value checked before being used
- Class Definition Defects
 - constructor and destructor, memory leak, class inheritance hierarchy
- Control Flow Defects
 - loop terminate, nesting of loops and branches
- Input-Output Defects
- Comment Defects
- others



Advantages of inspections

- During testing, errors can hide other errors.
- Incomplete versions of a system can be inspected.
- An inspection can also consider broader quality attributes of a program, such as compliance with standards, portability and maintainability.
- With the reuse domain and programming knowledge, experienced reviewers are likely to have seen the types of error that commonly arise.
 - Knowledge sharing



Advantages of inspections

- Reviews detect (and correct) defects early in development.
 - early detect leads to a cost-effective correction
 - reduce overall development effort
- Inspections vs. Testing
 - They are complementary with each other and not opposing
 - properties hard to test
 - security, exception handling
 - Requirements, architecture, design documents (before testing)
 - not for executing as tests



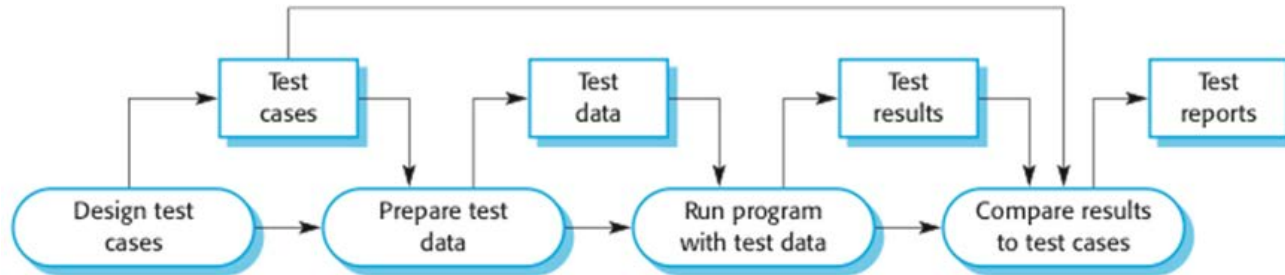
Software Testing

- Executing a program in order to
 - force a program to work incorrectly
 - Find what is wrong
 - demonstrate that the program works correctly
 - fault-based testing: target on certain types of faults
 - for example, divide by zero => the test data would include zero.
- Typically, testing takes about 30–40% of budget for system development



Software Testing

- Testing consists of a set of test cases which are systematically planned and executed



Test Case Design

- Each test case is uniquely identified or created
- Purpose of each test case should be stated
- Expected results are listed in the test case
 - include expected outputs, resultant changes and error messages



A Standard Test Case Matrix

- Case ID
 - Scenario
 - Steps
 - Prerequisites
 - Test Data
 - Expected Results
 - Actual Results
 - Test Status - Pass/Fail
- **Test Case ID:** #BST001
 - **Test Scenario:** To authenticate a successful user login
 - **Test Steps:**
 - The user navigates xxx website.
 - The user enters a registered email address as username
 - The user enters the registered password.
 - The user clicks 'Sign In.'
 - **Prerequisites:**
 - A registered ID with a unique username and password.
 - Browser: Chrome v 86++
 - Device: Samsung Galaxy Tab S7.
 - **Test Data:** Legitimate username and password.
 - **Expected/Intended Results:** Once username and password are entered, the web page redirects to the user's profile page
 - **Actual Results:** As Expected
 - **Test Status - Pass/Fail:** Pass



Who performs testing?

- Testing is conducted by two (or three) groups:
 - the software developer
 - (for large projects) an independent test group
 - the customer



Development testing

- Development testing includes all testing activities that are carried out by the team developing the system
 - Unit testing
 - individual units or object classes are tested
 - focus on testing the functionality of objects or methods
 - System testing as a whole
 - testing component interactions



Unit testing

- Unit testing is the process of testing individual components in isolation
- Units may be:
 - Individual functions or methods within an object
 - Object class as a whole
 - Testing all operations associated with an object
 - Method A * Method B
 - Setting and interrogating all object attributes
 - Exercising the object in all possible states



Unit testing (example)

- Need to define test cases for reportWeather, reportStatus, powerSave, remoteControl, reconfigure, restart and shutdown.
- Using a state model, identify sequences of state transitions to be tested and the event sequences to cause these transitions
 - Shutdown -> Running-> Shutdown
 - Configuring-> Running-> Testing -> Transmitting -> Running

Weather station

state

reportWeather()
reportStatus()
powerSave() remoteControl()
reconfigure()
restart()
shutdown()



Automated unit testing

- Unit testing can/should be automated so that tests are run and checked without manual intervention
 - a test automation framework (such as JUnit or cppUnit)
 - generic test classes (e.g. `cppUnit::TestFixture`) is to be extended to create specific test cases.
 - run all tests, often through some GUI, to report the success



Unit test effectiveness

- Outcome: when used as expected, the component that you are testing does what it is supposed to do.
- Two types of unit test cases:
 - normal operation of a program and so the component works as expected
 - use abnormal inputs to check that these are properly processed and do not crash the component



Testing Techniques

- White box Testing
 - examining the internal workings (i.e. source code) of each module
- Black box Testing
 - without knowing the internal workings of the programs
 - a set of results is derived to ensure that modules produce correct results.



White Box Testing

- White Box tests focus on the program control structure

- Applied to increase logic coverage

- Statement coverage
- Decision (branch) coverage
- Condition coverage
- Path coverage

```
premium = 500;
if (age < 25) && (sex == male) && !married {
    premium += 500;
}
else {
    if (married) || (sex == female)
        premium -= 200;
    if (age > 45) && (age < 65)
        premium -= 100;
}
```



White Box Testing

- Statement coverage
 - each statement is executed at least once
- Decision (branch) coverage
 - each statement ...; each decision takes on all possible outcome at least once
- Condition coverage
 - each statement...; each decision ...; each condition in a decision takes on all possible outputs at least once
- Path coverage
 - each statement ...; all possible combinations of condition outcomes in each decision occur at least once



Black Box Testing

- Without knowing the internal works
- Mainly from inputs and with two techniques:
 - Random (uniform):
 - Pick possible inputs uniformly, treats all inputs as equally valuable
 - **Avoids bias**
 - test designer can make the same logical mistakes and bad assumptions as the program designer (especially if they are the same person)
 - Drawback: impossible to perform testing for each set of test data, especially when there is a large pool of input combinations



Black Box Testing strategies

- Systematic (non-uniform):
 - Try to select important inputs (with specially valuable)
 - Usually by choosing representatives of classes that are apt to fail often or not at all
- Select test cases intelligently from the pool of test-case:
 - Equivalence Partitioning
 - Boundary Value Analysis

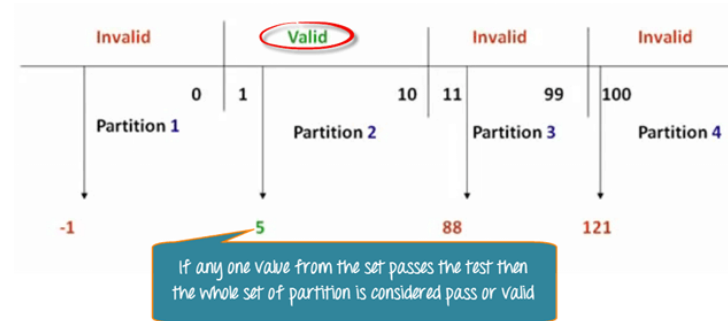


Equivalence Partitioning

- Divides the input data of software into different equivalence data classes.
 - pick only one value from each partition for testing
 - if passes, we assume this partition will also pass.

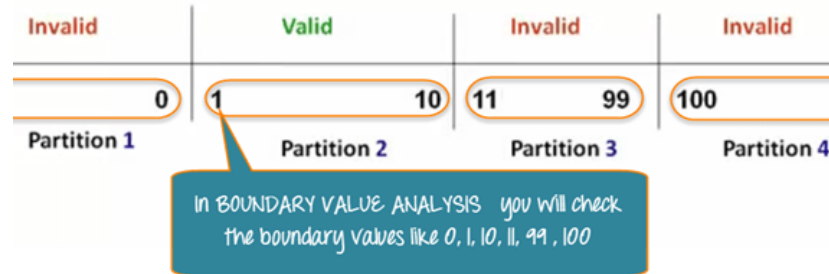
Example:

- Pizza values 1 to 10 is considered valid. A success message is shown.
- While value 11 to 99 are considered invalid for order and an error message will appear, "Only 10 Pizza can be ordered".



Boundary Value Analysis

- Selects test cases at the edges of the set or boundary values.
- Boundary Value Analysis is also called **range checking**
- Complements equivalence partitioning



Integration testing

- Tests on integrated components (subsystems/functions)
- Usually unit testing first, then integration testing will be offered to look at primarily interface faults.
 - Testing effort should focus on interfaces between units rather than their internal details.



Systems Testing

- This test encapsulates the environment as well as the product:
 - Function Testing
 - tests functional requirements of the system
 - Performance Testing
 - tests non-functional requirements, e.g. reliability, availability, etc.
 - Acceptance Testing
 - performs validation testing on the system prior to handover to the customers (client satisfaction)



Test-driven development

- A bridge between testing and coding
- Tests are written before code and ‘passing’ the tests is the critical driver of development.
- Develop code incrementally, along with a test for that increment.
 - don’ t move to the next increment until the current code passes its test.



Test-driven development (steps)

- Start by identifying the new functionality that is required. This should normally be small and implementable in a few lines of code
- Write a test for this functionality and implement this as an automated test
- Run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail.
- Implement the functionality and re-run the test.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.



Benefits of test-driven development

- Code coverage
 - Every code segment that you write has at least one associated test so all code written has at least one test.
- Regression testing
 - developed incrementally to make sure changes have not ‘broken’ previously working code.
- Simplified debugging
 - When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.



Tools

- `cpputest`
- `csUnit` (.Net)
- `Cunit`

- `HTMLUnit`
- `HTTPUnit`
- `JUnit`
- `PHPUnit`

- `PyUnit` (Python)
- `DocTest` (Python)
- `TestOoB` (Python)

- `Test::Unit` (Ruby)
- `VBUnit`



U

O

W

CSIT314

Software Development

Methodologies

Principles and practices of continuous integration and delivery
(CI/CD)



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Continuous Integration (CI)



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

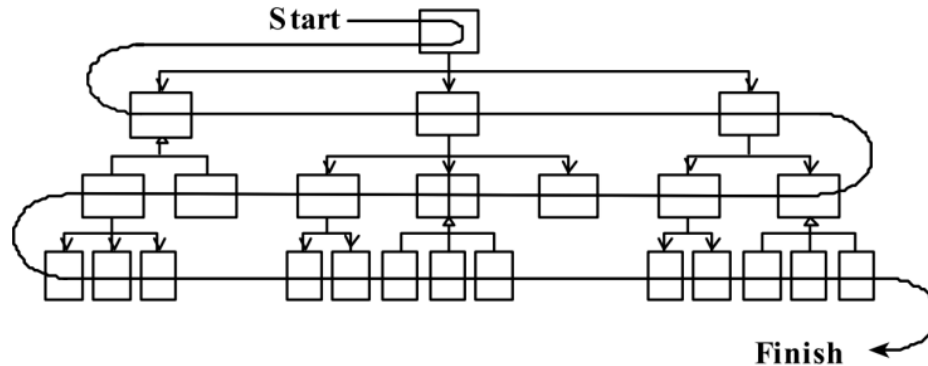
What is integration

- Multiple developers working on the same codebase at the same time (independently):
 - Developer A works on feature 1 while developer B works on feature 2.
 - Developer A works on class 123.java while developer B works on class 456.java
 - Once they have finished, they need to integrate their work into the main codebase.



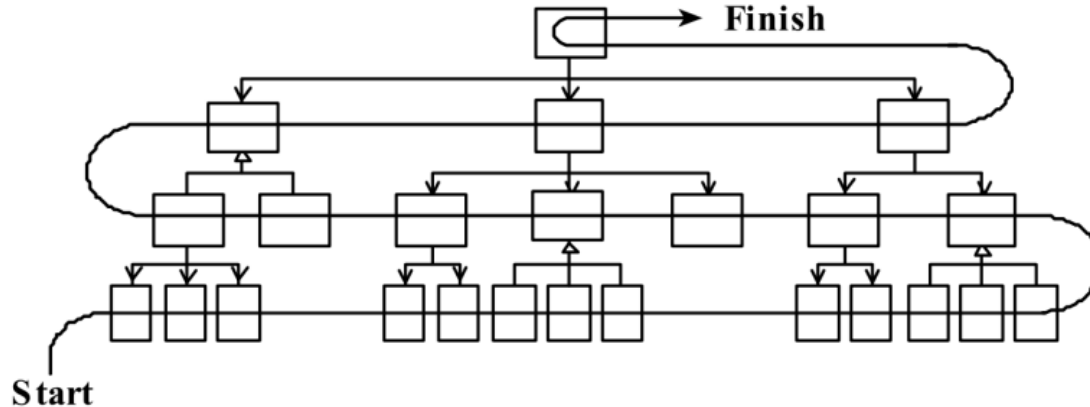
Top-down integration

- Start with top-level UI layers
 - must write (lots of) stub lower layers for UI to interact with
 - allows postponing tough design/debugging decisions



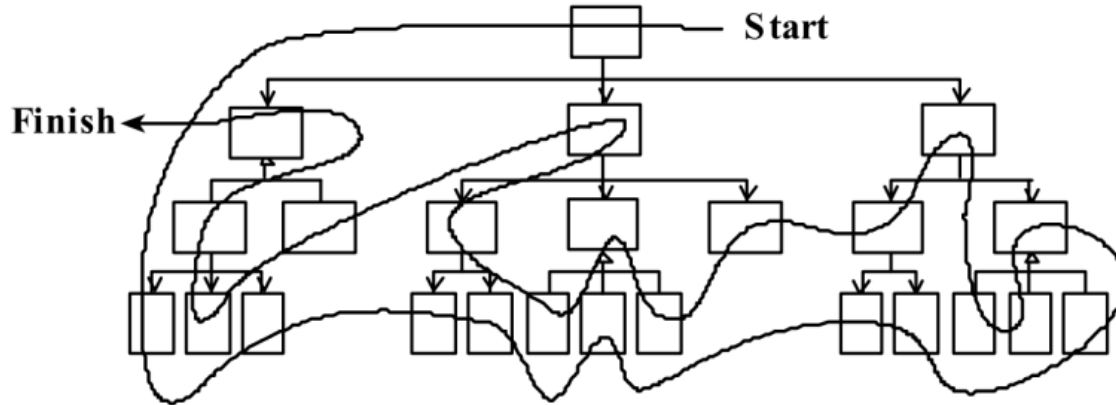
Bottom-up integration

- Start with bottom-level data/logic layers
 - must write test drivers to run these layers
 - won't discover high-level / UI design flaws until late



Sandwich integration

- Combine top-level UI with bottom-level classes
 - add middle layers later as needed
 - more flexible but also problems



Continuous Integration (CI)

- Continuous integration (CI) is a software development practice where developers in a team integrate their work frequently.
 - several times a day
- Each integration can be verified by an automated build:
 - Compile (or build) the code
 - translating human-readable source code into machine-readable instructions, typically performed by a compiler
 - Run automated tests



Build Types

- There are three basic types of builds: private, integration, and release (more on next slide)
 - All builds are automated, script-driven processes that pull source code from the repository and create deployable artifacts
 - These build types differ in several ways
 - Where, when, by whom, and for what reason they are run
 - The code that feeds the process
 - What is done with the output



Build Types

- Private Builds
 - Run by developers in their local environments (IDE)
 - Ensure that code compiles before committing to source code repository
 - Triggered manually
 - Very stripped-down and designed to finish quickly
- Integration Builds
 - Compile in response to recent changes
 - Performed on a dedicated build server either on a scheduled basis (nightly) or continuously
 - Usually drawn from the trunk of the repository tree
 - Fast or slow depending on the number and type of processes included in the build script
- Release Builds
 - Performed codes to production environments
 - Performed by the team' s release manager
 - Run the full suite of automated and manual tests
 - Tend to be slow



Build from command line (example)

- An Android project needs a build.xml to be used by Ant.
 - This file allows your project to be compiled from the command line, making automated builds possible.

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="MainActivity" default="help">
  <property file="ant.properties" />
  <import file="${sdk.dir}/tools/ant/build.xml" />
  <taskdef name="findbugs" classname="edu.umd.cs.findbugs.anttask.FindBugsTask"/>
  <target name="findbugs">
    <findbugs home="${findbugs.home}" output="xml" outputFile="findbugs.xml"
      excludeFilter="findbugs-exclude.xml">
      <auxClasspath path="${android.jar}" />
      <auxClasspath path="${rt.jar}" />
      <auxClasspath path="libs\android-support-v4.jar" />
      <class location="${out.dir}" />
    </findbugs>
  </target>

  <taskdef resource="checkstyletask.properties" classpath="${basedir}/libs/checkstyle-5.6-all.jar"/>
  <checkstyle config="sun_checks.xml" failonviolation="false">
    <fileset dir="src" includes="**/*.java"/>
    <formatter type="plain"/>
    <formatter type="xml" toFile="checkstyle-result.xml"/>
  </checkstyle>
</project>
```

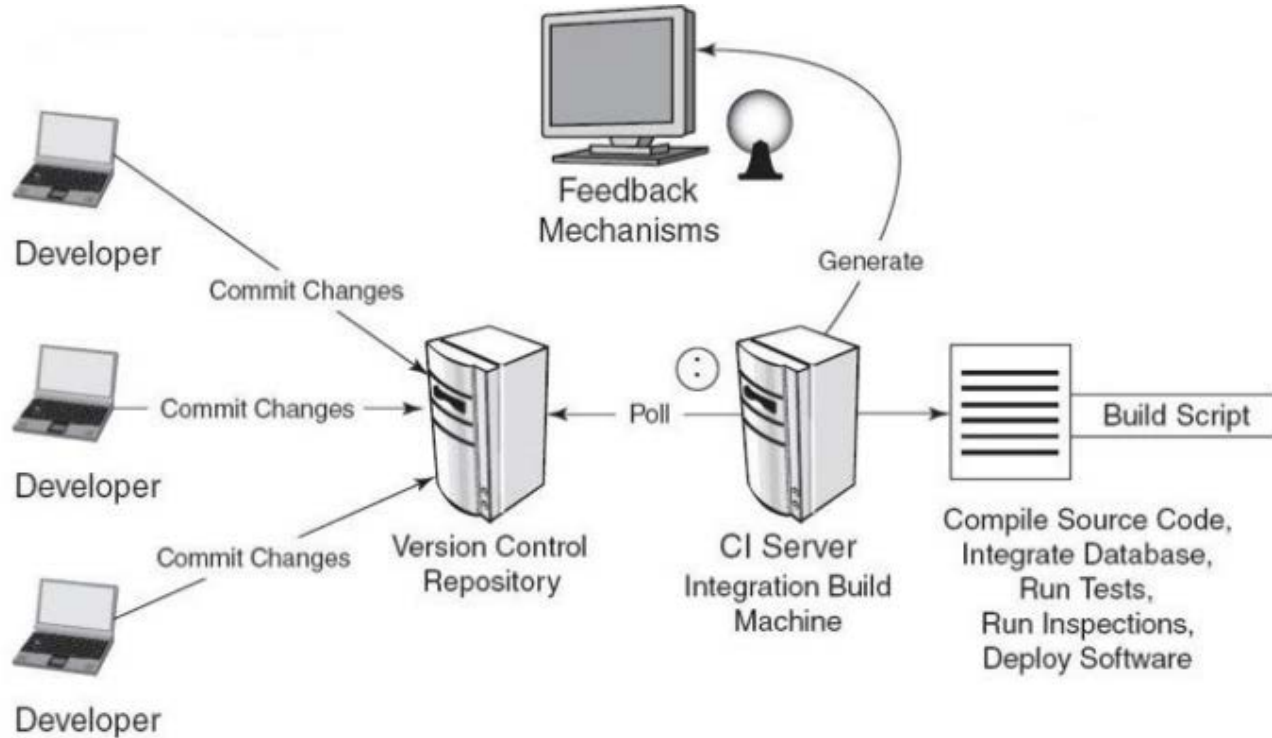


Daily builds

- Daily build: Compile working executable on a daily basis
 - allow test the quality of your integration
 - make sure product “works every day”
 - through an easy script
 - quickly catches/exposes bug that breaks the build
- Server: An external machine that automatically pulls down your latest repo code and fully builds all resources.
 - If anything fails, contacts the team
 - Ensures that the build is never broken for long



Architecture



Artifacts Needed

- A build usually involves more than source code files. Other items may include:
 - Project and third-party components and libraries
 - Configuration files
 - Data files and database scripts
 - Test scripts
 - Properties files
- All items except project and third-party libraries should be stored in the source code repository.



Build System

- completely automated
- usually through the execution of a single script
- a build always begins by retrieving code from the source code repository
- unless terminated prematurely, the end product of a build is always executable code
- notification of build status always occurs through a feedback mechanism



The build cycle

- Basic build cycle consists of these key steps:
 - Developer commits changes to the source code repository
 - Build server executes the master build script, or delegates execution to another server
 - Checks out source code
 - Builds executable version of the application
 - Runs other jobs, such as testing and code inspection
 - Team is notified of build results through a feedback mechanism
 - If alerts are generated, the team takes immediate action to correct problems
 - If a code fix is needed, developer commits the corrected code back to the repository; this action kicks off a new build cycle.



Improving Build Speed

- Three approaches to dealing with build speed:
 - Speed up the build by eliminating bottlenecks or by running the build on a faster machine with plenty of memory.
 - Pick a good time: run complete builds less often or at a time when developers are less likely to be working
 - Employ a “staged/seperate build” approach: the basic build, the 2nd build for another process, the 3rd build for remaining steps.
- A combination of the three approaches is often used.



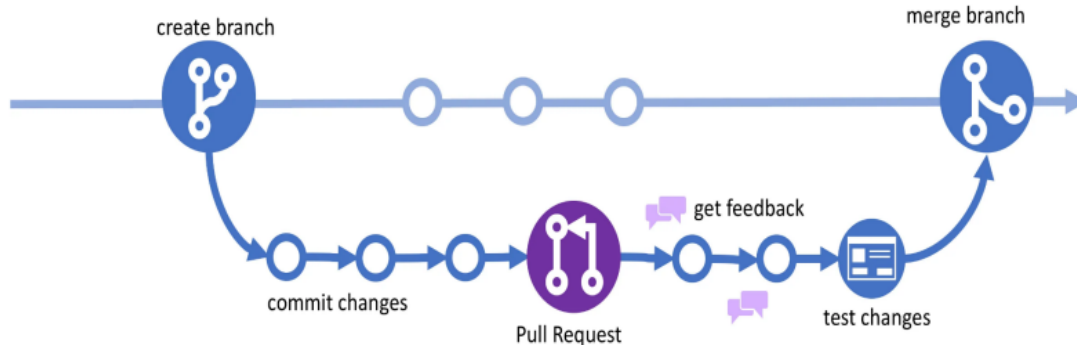
Continuous Integration

- Principles (detail later)
 - maintain a single source repository
 - automate the build
 - make your build self-testing (Automated tests)
 - everyone commits to mainline every day
 - every commit should build mainline on an integration machine
 - keep the build fast
 - test in a clone of the production environment
 - make it easy for anyone to get the latest executable
 - everyone can see what's happening



Practices of Continuous Integration

- Maintain a Single Source Repository
 - Use a version control system such as Git/GitHub
 - **Everything** required to build the software app should be in the repository (code, test scripts, test data, properties files, database schema, third-party libraries, etc.)



Practices of Continuous Integration

- Automate the build
 - Automate the whole process of turning the source code into a running software app
 - Operations: compiling, moving files around, loading schemas into the databases, etc.
 - Issuing a single command and the whole build process will run automatically.



Practices of Continuous Integration

- Make your build self-testing
 - automated tests: e.g. Tests that can be run from the command line on your project code at any time.
 - can be unit tests, coverage, static analysis (inspection)
 - an important component of the CI build cycle (more later)
 - smoke test: A quick set of tests run on the daily build.
 - NOT exhaustive; just sees whether code "smokes" (breaks)
 - run with daily build



Practices of Continuous Integration

- Daily commits
- Submit work to main repo at end of each day.
 - Merge conflicts
 - Maintain the daily commit practice
 - If your code is not ready to submit at end of day, either submit a subset or be flexible about commit schedule



Practices of Continuous Integration

- Every commit should build the mainline (the main codebase) on an integration server/system
 - use continuous integration server/system (e.g. Jenkins, Travis CI, Bamboo, GitLab, etc.)
 - only if this integration build succeeds should the commit be considered to be done
- Test in a clone of the production environment.
 - Testing in a different environment introduces risk when the system is deployed in production.



Practices of Continuous Integration

- Commits should build the mainline (the main codebase) on an integration machine
 - Can be multiple times a day
 - Frequent integration means frequent communication
 - Find out if there is a conflict between the changes
 - Errors, conflicts, bugs, etc. can be detected early and rectified (fixed) quickly.



Practices of Continuous Integration

- Keep the build fast
 - to provide rapid feedback
 - if the build is not fast, developers will commit less often and will be provided feedback on problems less often.
 - also, long builds can be a significant roadblock to wait
- In general, the more processes included in the build, the better the quality of the final product
- However, there are tradeoffs
 - sub-process takes time to execute
 - extra steps result in loss of productivity, as developers wait for build cycles before they continue working



Continuous Database Integration

- Changes to a **database** may occur often
 - as important as early detection of errors in the source code.
 - a build server can be set up to begin a new CI cycle whenever database is changed.
 - code and configuration files that modify the database need also be committed to the repository
 - just as code changes



Infrastructure as Code

- What is IT infrastructure?
 - Physical machines, devices, OS, databases, and any other systems that are used to run a software application.
- Infrastructure as Code
 - the whole IT infrastructure can be treated as if they are software
 - setup, configured and changed automatically through writing code
- Infrastructure as code is the prerequisite for common CI/CD practices



Infrastructure as Code

- IaC model generates the same environment every time it is applied
 - enables software teams to test applications in production-like environments early in the development cycle
 - IaC demo: https://youtu.be/k6_ZTlxI4xk



Continuous Database Integration

- A CI build can:
 - Drop an existing database
 - Inspect database scripts for compliance with project practice
 - Create, configure, and populate data
 - Run automated unit tests to verify that everything is working as expected



Benefits of CI

- Increased productivity
 - Enables shorter feedback cycle when changes are made
 - Code is kept in a “releasable” state
 - Code gets back into the hands of testers quickly
 - Enables more frequent releases with new features
- Improved quality
 - Frequent testing identifies bugs
 - Multi-platform builds help in finding problems
- Reduced Risk
 - Reduces uncertainty greatly because at all times the team knows what works, what does not, and what the major issues are.



Barriers to CI

- Complexity and tradeoff
 - Time
 - Technical knowledge
 - A number of new tools and processes must be mastered
 - Setting up a CI server
 - Automated test
 - A build scripting language and potentially a setup/install platform as well.



Continuous Delivery (CD)



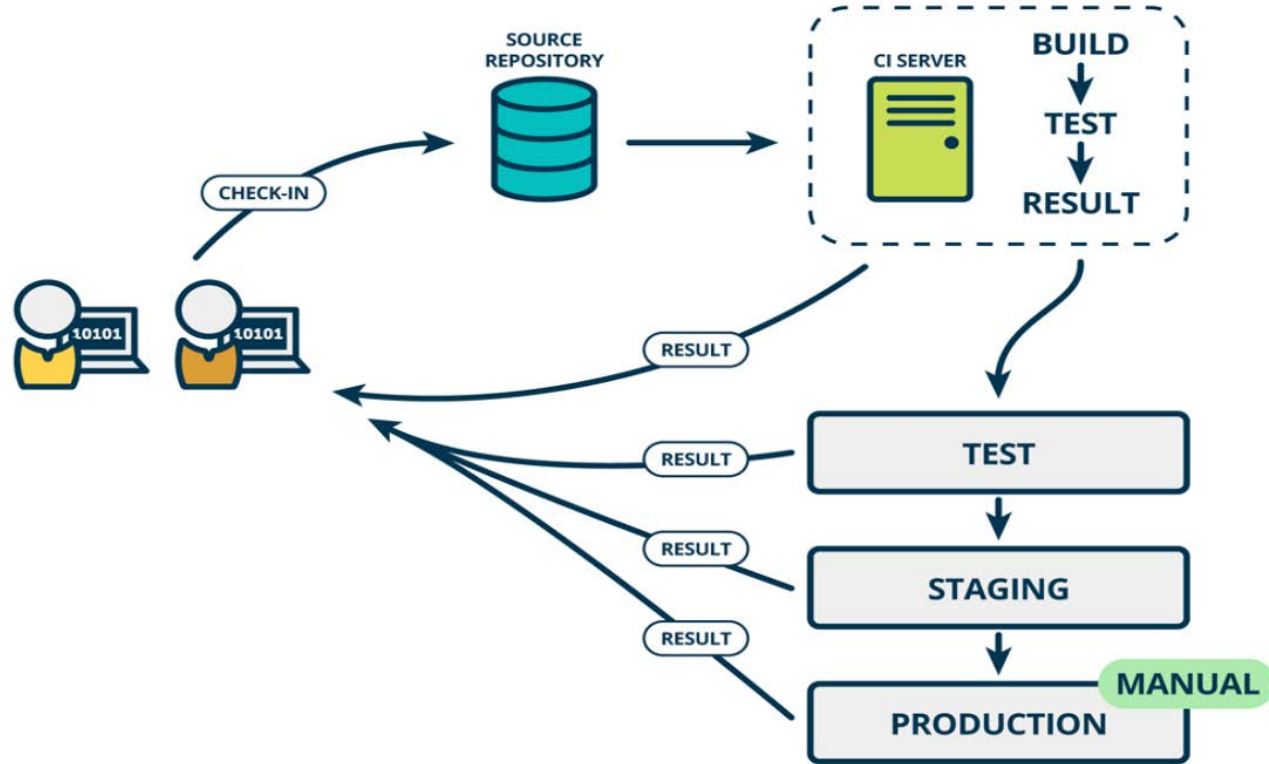
UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Continuous Delivery

- Continuous Delivery is a further step beyond Continuous Integration:
 - Each time changes are pushed to the codebase, the new code is automatically built and tested on environments that are very similar to production (staging environment)
- The staging environment can be used to address non-functional requirements
 - such as security, redundancy, and flexibility
 - note that these may not be covered in the development environments



Continuous Delivery cycle

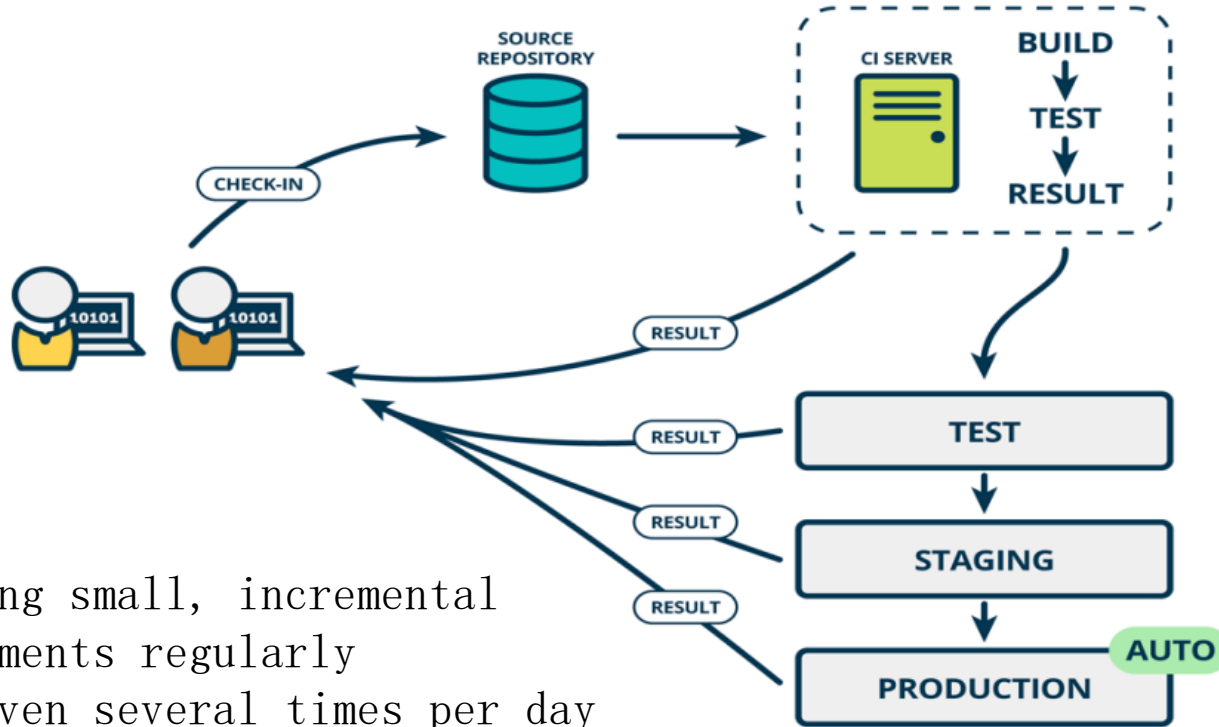


Continuous Deployment

- In Continuous Delivery, the deployment of the software application to a production environment is still done manually.
 - such as the manual testing
- Continuous Deployment is an additional step beyond Continuous Delivery (prototypes):
 - The software application is deployed automatically.
 - Every time code changes are pushed to the codebase, it will be automatically built and tested - and if the tests are successfully, it will automatically go to production.



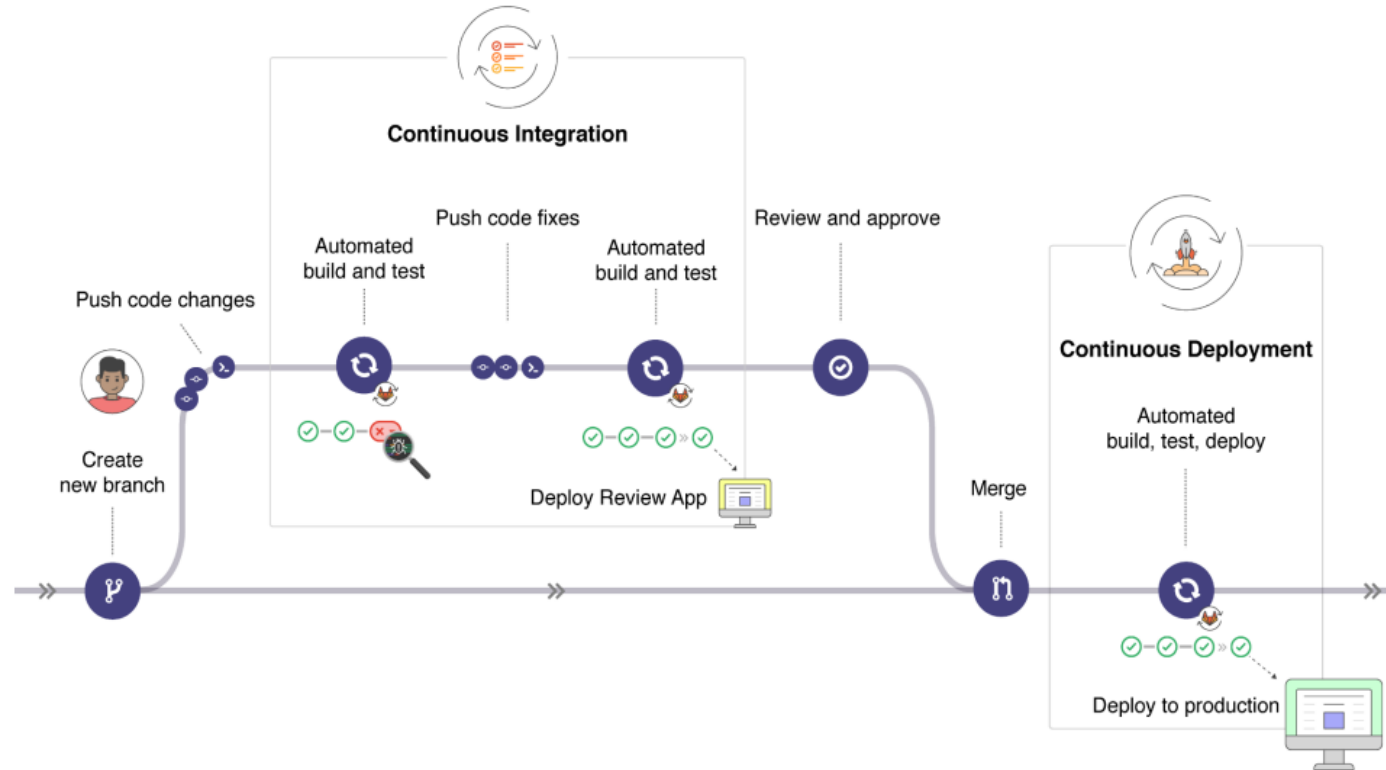
Continuous Deployment cycle



1. releasing small, incremental improvements regularly
2. often even several times per day



CI/CD with GitLab



CI/CD with GitLab (cont.)

- Setup a project on GitLab
- Install GitLab Runner on your computer
 - <https://docs.gitlab.com/runner/#install-gitlab-runner>
- Register GitLab Runner with GitLab
 - <https://docs.gitlab.com/runner/register/>
- Start GitLab Runner



CI/CD with GitLab (cont.)

- Create a CI/CD pipeline on GitLab
 - Create/add a file called “.gitlab-ci.yml” in the root folder of your project.
 - This YAML file defines the structure and order of the CI/CD pipeline and decides:
 - What will be executed using GitLab Runner
 - What to do when some specific conditions are satisfied (e.g. when a process succeeds or fails).
 - See <https://docs.gitlab.com/ee/ci/yaml/> for more details



CI/CD with GitLab (cont.)

- A live demo of how to set up and run CI/CD with GitLab
 - [https://www.youtube.com/watch?v=jUiKi6FWYrg&list=PLhW3qG5bs-L8YSnCiyQ-jD8XfHC2W1NL &index=8](https://www.youtube.com/watch?v=jUiKi6FWYrg&list=PLhW3qG5bs-L8YSnCiyQ-jD8XfHC2W1NL&index=8)
- GitLab CI/CD Examples for different types of applications written in different programming languages
 - <https://docs.gitlab.com/ee/ci/examples/>



U

O

W

CSIT314

Software Development

Methodologies

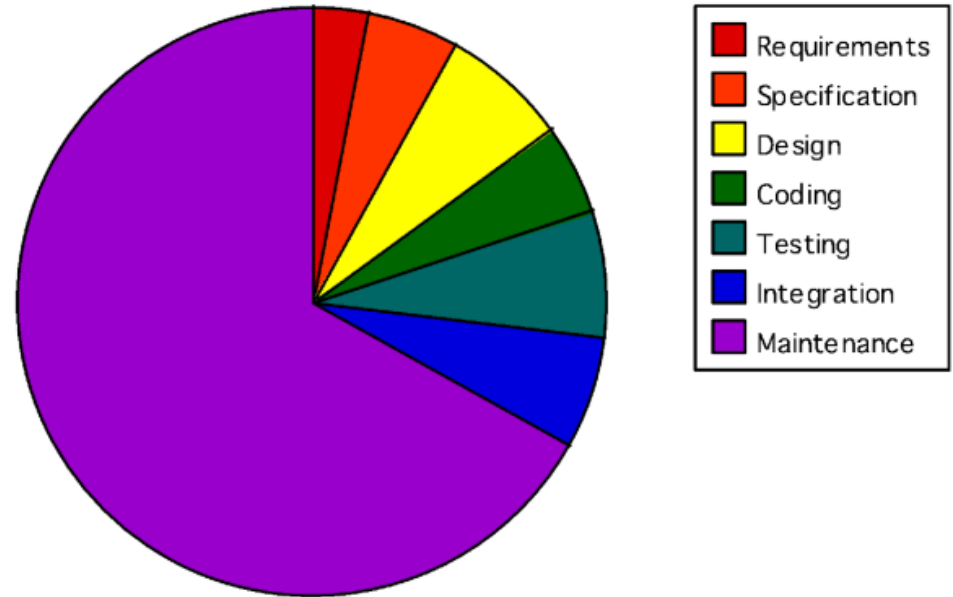
DevOps



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Software Development Activities

- Software Engineering plays a critical part in the software development
 - Planning
 - Requirements analysis
 - Design
 - Implementation
 - Verification & Validation
 - Maintenance and evolution



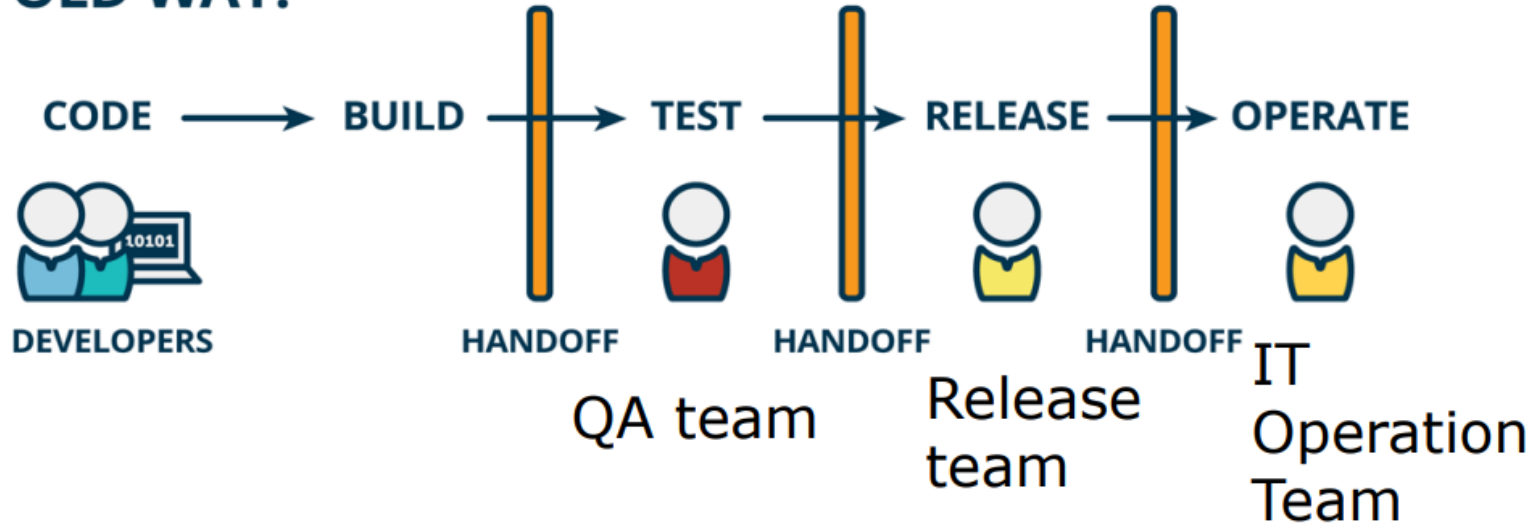
Traditional models

- Development team (DEV)
 - Design + Code + Test
- IT Operations Team (OPS)
 - Deploy + Support, Sys Admin + Maintenance
- Wall of confusion (more details)
 - separate entities within an organization
 - different mindsets/objectives
 - OPS : work in a cost-efficient way
 - DEV : efficiently deliver features
 - different tools & environments



Traditional models

OLD WAY:



Traditional models

- Problem 1: different objectives
 - IT Operations team like the software to be as much stable as possible.
 - might see each change and every new release as a potential danger to stability
 - The Development team is paid for changing and adding new features the software application.
 - Might lead to conflicts between Dev and Ops departments.



Traditional models

- Problem 2: different perspectives
 - Operations team views the application as a black box. They can monitor it with suitable tools (e.g. CPU utilization, I/O load, kernel behaviour)
 - Development team has access to source code and knows the internal working of the application.



Traditional models

- Problem 3: different environments
 - Operations team run the application in the production environment
 - Developers run and test their application in their own computer
 - The two environments may be vastly different in terms of hardware, software, libraries, etc.



Traditional models

- Problem 4: driven by different requirements
 - Operations is driven by non-functional requirements (availability, reliability, system speed ...)
 - Development is driven by functional requirements (features).
- Problem 5: client perspective
 - who is really responsible?



Traditional models

- Problem 6: combining know-how
 - Operations and development have only their own part of knowledge and the required tools to build, test and run the application
 - A smooth operation of applications is only possible with the combined know-how of operations and development.



What is DevOps?

- DevOps is a software engineering methodology which aims to unify software development (Dev) and IT operation (Ops)
- Development and operations teams are no longer separated (i.e. no confusion wall)
 - these two teams can be merged into a single team
 - the DevOps engineers work across the entire application lifecycle, from development and test to deployment to operations.

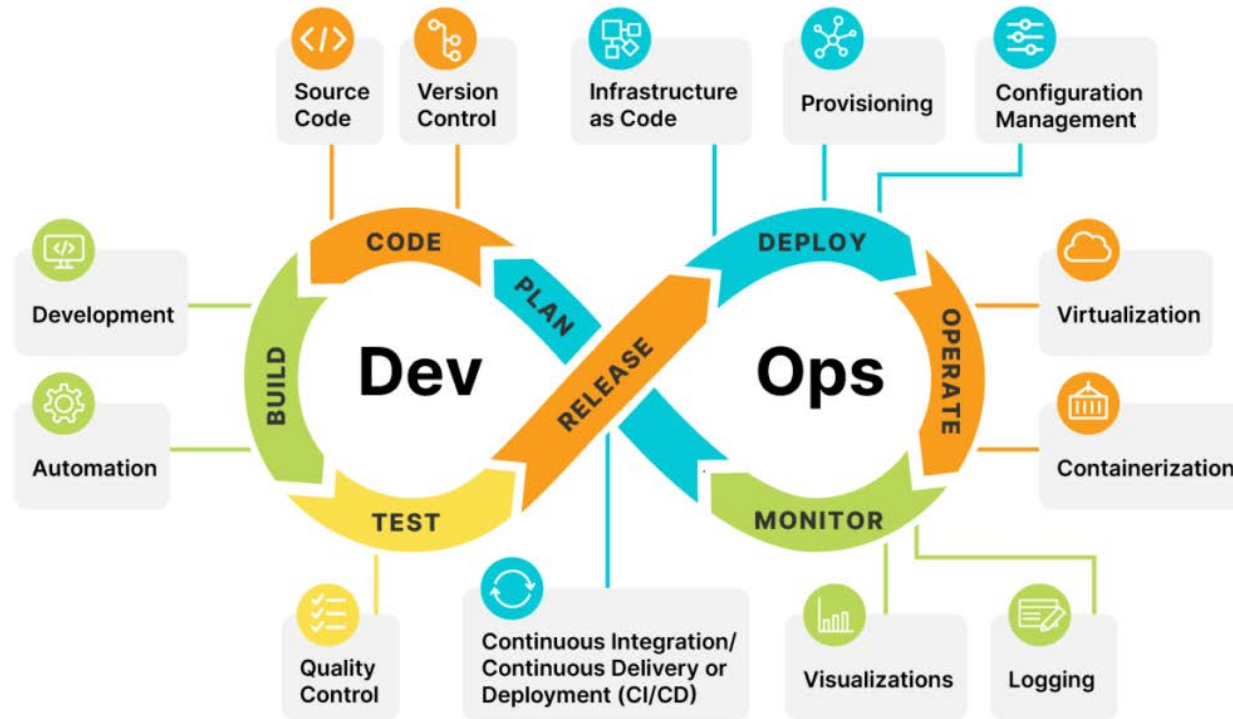


DevOps

- Collaboration, trust, and shared responsibility
- Support autonomous teams
 - better manage unplanned work
 - release faster and work smarter
 - value feedback



DevOps lifecycle



DevOps lifecycle

- Development (plan, code & build)
 - Agile, incremental and iterative development
- Test
 - Test-driven development, acceptance testing
- Integration (release)
 - New functionality is integrated with existing code, and testing takes place
- Deployment (deploy & operate)
 - Deployment process takes place continuously
- Monitor
 - Bugs and performance reporting in real time



DevOps Vs CI/CD

- DevOps facilitates the implementation of CI/CD:
 - Development knows:
 - how the application has to be configured
 - metrics for monitoring
 - Operations knows:
 - the internal structure of the application
 - Feedback from operation can be used directly to optimize further development of the application (since all necessary roles are united in one team)



DevOps Vs Agile

Features	DevOps	Agile
Agility	both Development & Operations	only Development
Processes	processes such as CI, CD, CT, etc	practices such as Agile Scrum, etc.
Key Focus	Timeliness & quality have equal priority	Timeliness is the main priority
Source of Feedback	from self (Monitoring tools)	from customers
Scope of Work	Agility & need for Automation	Agility only



DevOps benefits

- Rapid delivery: innovate and improve your product faster (e.g. release new features and bug fixes quickly) => respond to your customers' needs and build competitive advantage.
- Reliability: ensure the quality of application updates and infrastructure changes.
- Scale: Operate and manage your infrastructure and development processes at scale.



DevOps: A Netflix Casey Study

- Edge Engineering is responsible for the first layer of AWS services (Amazon Web Services) that must be up for Netflix streaming to work.
- Releasing a new feature meant devs coordinating with the ops team on things like metrics, alerts, and capacity considerations, and then handing off code for the ops team to deploy and operate.
- To be effective at running the code and supporting partners, the ops teams needed ongoing training on new features and bug fixes.
- Less developer interrupts when things were going well.

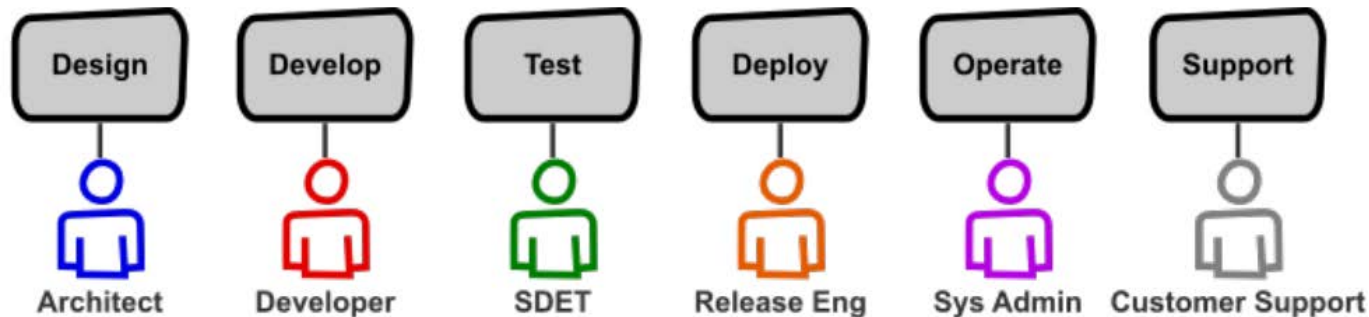
DevOps: A Netflix Casey Study

- When things didn't go well, the costs added up
 - communication and knowledge transfers between devs and ops were messy
 - requiring additional round trips to debug problems or answer partner questions
 - deployment problems had a higher time-to-detect and time-to-resolve due to the ops teams having less direct knowledge of the changes being deployed
 - the gap between code complete and deployed was much longer, with releases happening on the order of weeks rather than days.
 - feedback went from ops, who directly experienced pains such as lack of alerting/monitoring or performance issues and increased latencies, to devs, who were hearing about those problems second-hand.



DevOps: A Netflix Casey Study

- Developing and running a software service involves a full set of responsibilities.
- We had been segmenting these responsibilities. At an extreme, this means each functional area is owned by a different person/role:



DevOps: A Netflix Casey Study

- These specialized roles create efficiencies within each segment while potentially creating inefficiencies across the entire life cycle
 - Specialists develop expertise in a focused area and optimize what's needed for that area
 - Yet, Software requires the entire life cycle to deliver value to customers.
 - Focusing on a slice of the life cycle can create silos that slow down end-to-end progress
 - Grouping differing specialists together into one team may help
 - Yet, communication overhead, introduces bottlenecks, and inhibits the effectiveness of feedback loops



DevOps: A Netflix Casey Study

- Edge Engineering optimize for learning and feedback by breaking down silos and encouraging shared ownership of the full software life cycle (no individual for an area)
- “Operate what you build”
 - having the team that develops a system also be responsible for operating and supporting that system.



DevOps: A Netflix Casey Study

- “Operate what you build”
 - each development team owns deployment issues, performance bugs, capacity planning, alerting gaps, partner support, and so on.
 - direct feedback loops and aligns incentives
 - teams are empowered to make changes to the system design or its underlying code
 - take ownership of the work (be responsible and accountable), rather than relying on external parties or waiting for permission



DevOps: A Netflix Casey Study

- Centralized teams
 - with the mission of developing common tooling and infrastructure (reusable building blocks) to solve problems that every development team has
 - other teams can leverage the pre-built components provided by the centralized teams
 - ensure consistency, standard and quality



DevOps: A Netflix Casey Study

- Empowered with these tools in hand, development teams can
 - decide if their need is important enough for them to solve on their own
 - focus on solving specific problems within their specific product domain
 - centralized teams assess whether the needs are common across multiple dev teams
 - decide if local needs are too specific to warrant centralized investment
- One team, equipped with amazing developer productivity tools, is responsible for the full software life cycle
 - design, development, test, deploy, operate, and support



DevOps: A Netflix Casey Study

- Moving to a full cycle developer model requires a mindset shift
 - Before
 - design+development, and sometimes testing, as the primary way that they create value
 - operations as a distraction, favoring short term fixes to operational and support issues so that they can get back to their “real job”
 - Now
 - use the software development expertise to solve problems across the full life cycle.
 - create software, write test cases, and still other times automate operational aspects



DevOps: A Netflix Casey Study

- With the full cycle model, priority is given to a larger area of ownership and effectiveness in those broader domains
- Breadth requires both interest and aptitude in a diverse range of technologies
 - enjoy and welcome the broader responsibilities
- Developers prefer focusing on becoming world class experts in a narrow field may be uncomfortable
 - support them in finding those roles



DevOps: A Netflix Casey Study

- Breadth increases workload and means a team will balance more priorities every week than if they just focused on one area
- Solution: having an on-call rotation where developers take turns handling the deployment + operations + support responsibilities
 - when done well, that creates space for the others to do the focused, flow-state type work
 - when not done well, teams devolve into everyone jumping in on high-interrupt work like production issues, which can lead to burnout.



DevOps: A Netflix Casey Study

- Conclusion
 - Edge Engineering, whose earlier experiences motivated finding a better model, is actively applying the full cycle developer (design+deploy+support) model
 - Deployments are routine and frequent, canaries take hours instead of days, and developers can quickly research issues and make changes
 - no need to bounce the responsibilities across teams
 - Other groups are seeing similar benefits



DevOps in practice

- Some extra demos of how DevOps is used in major companies:
 - Atlassian: https://youtu.be/neySeor_vAk
 - Netflix: <https://youtu.be/UTKIT6STSVM>
 - Google: <https://youtu.be/0SU8-HJtYYc>

Tools used in DevOps

- Source Code Repository: developers check-in and change code => a major component of continuous integration
 - E.g. Git, Subversion, Cloudforce, Bitbucket and TFS
- Build Server: automatically compiles the code in the source code repository into executable code base
 - E.g. Jenkins, SonarQube and Artifactory
- Configuration Management: defines the configuration of a server or an environment.
 - E.g. Puppet and Chef.
- Virtual Infrastructure: programmatically create new virtual machines - provided by cloud vendors
 - E.g. Amazon Web Services and Microsoft Azure
- Test Automation: automatically perform all tests
 - E.g. Selenium and Water



U

O

W

CSIT314

Software Development

Methodologies

Kanban and XP



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Agile (recap)

- Project planning that places a premium on flexibility (resource constrain) and evolving customer requirements throughout the development process
- Agile refers to a set of methods and practices
 - the collaborative effort of self-organizing cross-functional teams and their customers and users.
 - adaptive planning, evolutionary development, early delivery, and continuous improvement
 - rapid and flexible response to change



Agile

Agile	Traditional (Waterfall)
Working software	Comprehensive documentation
Customer collaboration	Contract negotiation
Responding to change	Following a plan
Individuals and interactions	Process and tools

- Scrum
- Kanban
- Extreme Programming



Kanban - history

- Kanban (meaning “billboard” or “signboard” in Japanese)
- Kanban was originally developed by Toyota for manufacturing
- Adapted to software development in 2010

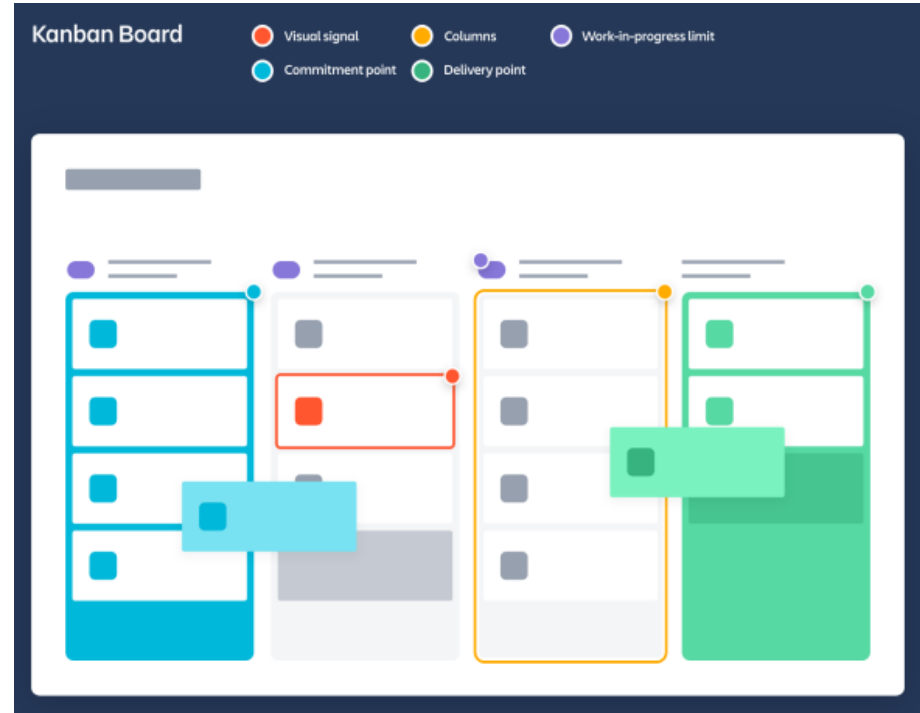
Kanban

- A popular agile software development methodology/framework
- Real-time team communication
- Full transparency of work
- Visually tracking progress



Kanban board

- Represents the overall project
- Has 5 components:
 - Visual signals
 - Columns
 - Work-in-progress limits
 - Commitment point
 - Delivery point



Kanban board (cont.)

- Visual signals:
 - Visual cards representing work items or user stories
 - These cards help all team members quickly see what the team is working on
- Columns:
 - Each column represents different “stage” in the workflow
 - Cards go through each stage in the workflow
 - Workflows can be customized:
 - “To Do” , “In Progress” , “Code Review” , “Done”



Kanban board (cont.)

- Work In Progress (WIP) Limits:
 - Maximum number of cards in one column at any given time
 - When the maximum number of cards is reached, the team needs to focus on the work in those cards to move them to the next stage
 - Once this is resolved, new cards can be moved into that column
 - This is a quick way to identify bottlenecks



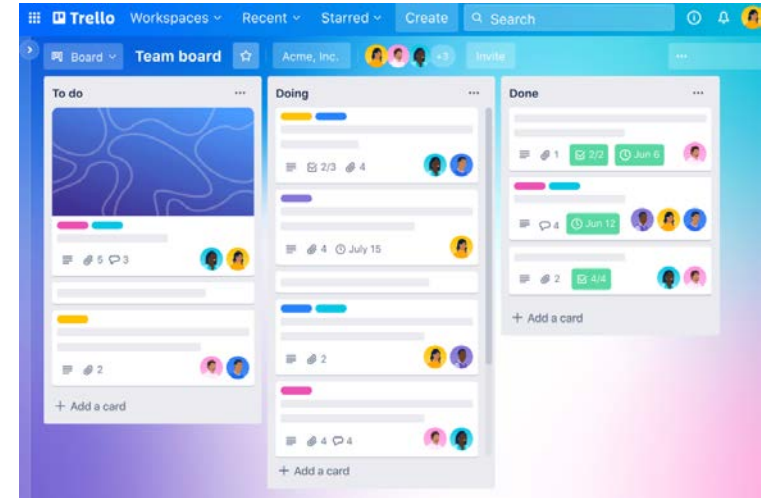
Kanban board (cont.)

- Commitment point:
 - The time point when an item is selected from the backlog and moved into the Kanban board
 - Signifies that work for this item is now started
- Delivery point:
 - The end of a Kanban team's workflow
 - Product is delivered to the clients
 - Lead time (between a request being made and a task being released) = Delivery point - Commitment point



Example

- Physical and virtual



Example

- Trello
 - <https://youtu.be/13F313psqXY>
- Taiga
 - <https://www.youtube.com/watch?v=6VsSvoJHiAk>



Kanban benefits

- Easy to use, simplicity
- Visual interface
- Instantly see what everyone is working on
- Visibility in terms of task progress and bottlenecks (e.g. a specific task is holding up the project)



Kanban vs. Scrum

	Scrum	Kanban
Cadence	Regular fixed length sprints (ie, 1-4 weeks)	Continuous flow
Release methodology	The end of each sprint	Continuous delivery
Roles	Product owner, scrum master, development team	No required roles
Key metrics	Velocity	Lead time, cycle time, WIP
Change philosophy	Teams should not make changes during the sprint.	Change can happen at any time



Case study

- Background
 - Mobile.de, an online search tool for buying and selling vehicles
 - From 1 star rating (2010) to 4 stars rating (2013) in Apple store
 - iPad, Android apps, and a mobile website portal (generate 35% of the overall traffic for the service)
 - With the help of Kanban method and lean thinking, they put the app at the top of the mobile rankings for Germany



Extreme programming

- One several popular agile methods
- Key characteristics:
 - Stress customer satisfaction
 - Empower developers to confidently respond to changing requirements
 - Strong emphasis on teamwork and peers
 - Deliver as early as possible and obtain early feedback from customers



Extreme programming

- XP incorporates the following 5 values
 - Communication: XP focuses on effective communication between the team and the customer
 - Simplicity: XP looks for the simplest ways to get things done. This means doing what is essential
 - Feedback: XP encourages instantaneous feedback
 - Respect: The team must respect each other
 - Courage: XP endorses courage at all levels



The Values of Extreme Programming

- Communication:
 - Everyone is part of the team and we communicate face to face daily
 - We will work together on everything from requirements to code
 - We will create the best solution to our problem that we can together.



The Values of Extreme Programming

- Simplicity
 - We will do what is needed and asked for, but no more
 - We will take small simple steps to our goal and mitigate failures as they happen
 - We will create something we are proud of and maintain it long term for reasonable costs



The Values of Extreme Programming

- Feedback:
 - We will take every iteration commitment seriously by delivering working software
 - We demonstrate our software early and often then listen carefully and make any changes needed
 - We will talk about the project and adapt our process to it, not the other way around



The Values of Extreme Programming

- Respect:
 - Everyone gives and feels the respect they deserve as a valued team member
 - Everyone contributes value even if it's simply enthusiasm
 - Developers respect the expertise of the customers and vice versa
 - Management respects the right to accept responsibility and receive authority over the work



The Values of Extreme Programming

- Courage:
 - We will tell the truth about progress and estimates
 - We don't document excuses for failure because we plan to succeed
 - We don't fear anything because no one ever works alone
 - We will adapt to changes when ever they happen.



XP Practices

Group	Practices
Feedback	✓ Test-Driven Development ✓ The Planning Game ✓ On-site Customer ✓ Pair Programming
Continual Process	✓ Continuous Integration ✓ Code Refactoring ✓ Small Releases
Code understanding	✓ Simple Design ✓ Collective Code Ownership ✓ System Metaphor ✓ Coding Standards
Work conditions	✓ 40-Hour Week

- XP suggests using 12 practices while developing software which can be clustered into four groups.

XP Practices: Feedback

- **Test-Driven Development:** XP teams practice test-driven development technique (TDD) that entails writing an automated unit test before the code itself. Every piece of code must pass the test to be released. TDD allows programmers to use immediate feedback to produce reliable software
- **The Planning Game:** This is a meeting that occurs at the beginning of an iteration cycle. The development team and the customer get together to discuss and approve a product's features
- **On-site Customer:** The customer should be present all the time to answer team questions, set priorities, and resolve disputes if necessary.
- **Pair Programming:** This practice requires two programmers to work jointly on the same code. While the first developer focuses on writing, the other one reviews code, suggests improvements, and fixes mistakes along the way.



XP Practices: Continue Process

- **Continuous Integration:** Developers always keep the system fully integrated. XP teams take iterative development to another level because they commit code multiple times a day
- **Code Refactoring:** Refactoring is about removing redundancy, eliminating unnecessary functions, increasing code coherency, and at the same time decoupling elements. Keep your code clean and simple, so you can easily understand and modify it when required would be the advice of any XP team member
- **Small Releases:** This practice suggests releasing the MVP quickly and further developing the product by making small and incremental updates. Small releases allow developers to frequently receive feedback, detect bugs early, and monitor how the product works in production.



XP Practices: Code Understanding

- **Simple Design:** The best design for software is the simplest one that works. If any complexity is found, it should be removed. The right design should pass all tests, have no duplicate code, and contain the fewest possible methods and classes.
- **Collective Code Ownership:** This practice declares a whole team's responsibility for the design of a system. Each team member can review and update code.
- **System Metaphor:** System metaphor stands for a simple design. The team should be able to start working on it without spending too much time examining specifications.
- **Coding Standards:** A team must have common sets of coding practices, using the same formats and styles for code writing.



XP Practices: Work Condition

- **40-Hour Week:** XP projects require developers to work fast, be efficient, and sustain the product's quality. To adhere to these requirements, they should feel well and rested. Keeping the work-life balance prevents professionals from burnout. In XP, the optimal number of work hours must not exceed 45 hours a week.

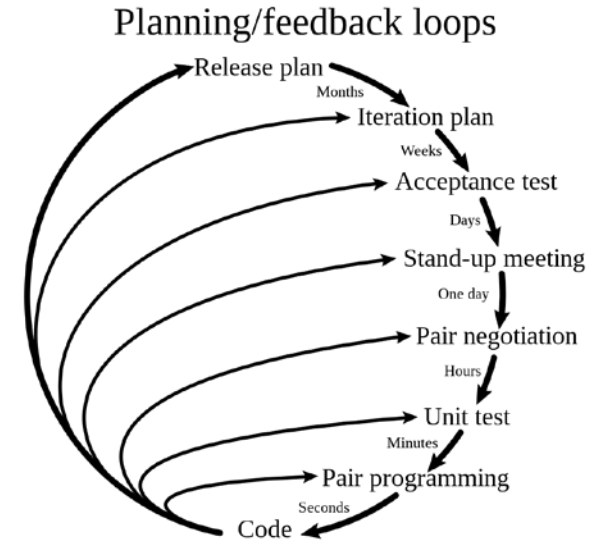
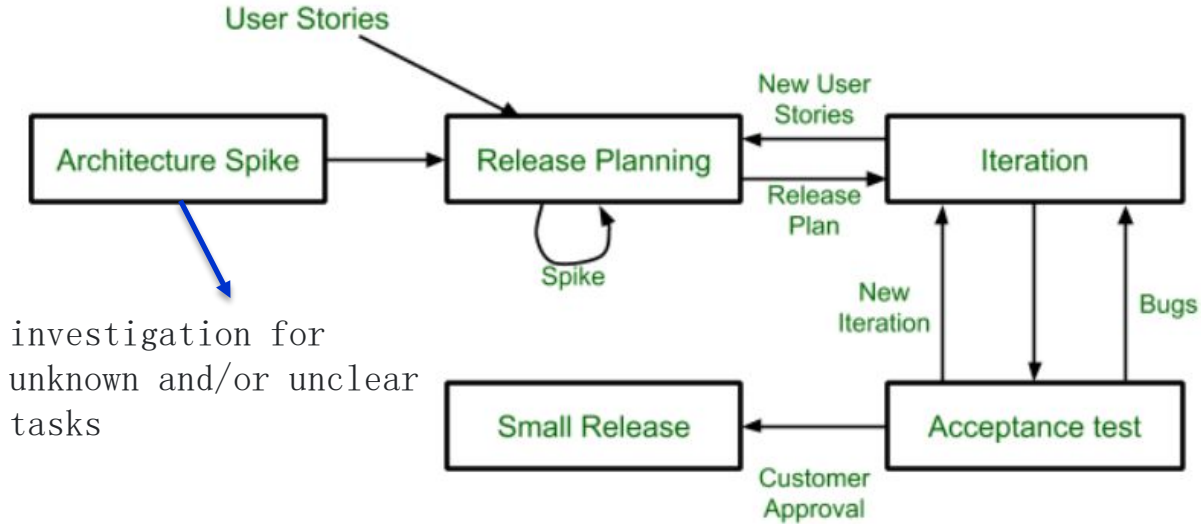


Extreme Programming Participants

- **Customers** are expected to be heavily engaged in the development process by creating user stories, providing continuous feedback, and making all the necessary business decisions related to the project.
- **Developers** are the team members that actually create the product. They are responsible for implementing user stories and conducting user tests.
- **Trackers** link customers and developers. It's not a required role and can be performed by one of the developers. These people organize the meetups, regulate discussions, and keep track of important progress.
- **Coaches** can be included in the teams as mentors to help with understanding the XP practices. It's usually an external consultant who is not involved in the development process, but has used XP before and so can help avoid mistakes.



Extreme Programming Process



Spike

- Create spike solutions to reduce risk
 - spike solution is a very simple program to explore potential solutions
 - build the spike to only addresses the problem under examination and ignore all other concerns
 - Most spikes are not good enough to keep, so expect to throw it away
 - Refactor whenever and wherever possible



Release planning

- User stories are written
- Each user story represents a desired feature and all stories represent the specification of the system
- Customers prioritise user stories based on business value (e.g. essential, highly valuable or good idea)
- Estimate each user story in terms of ideal days

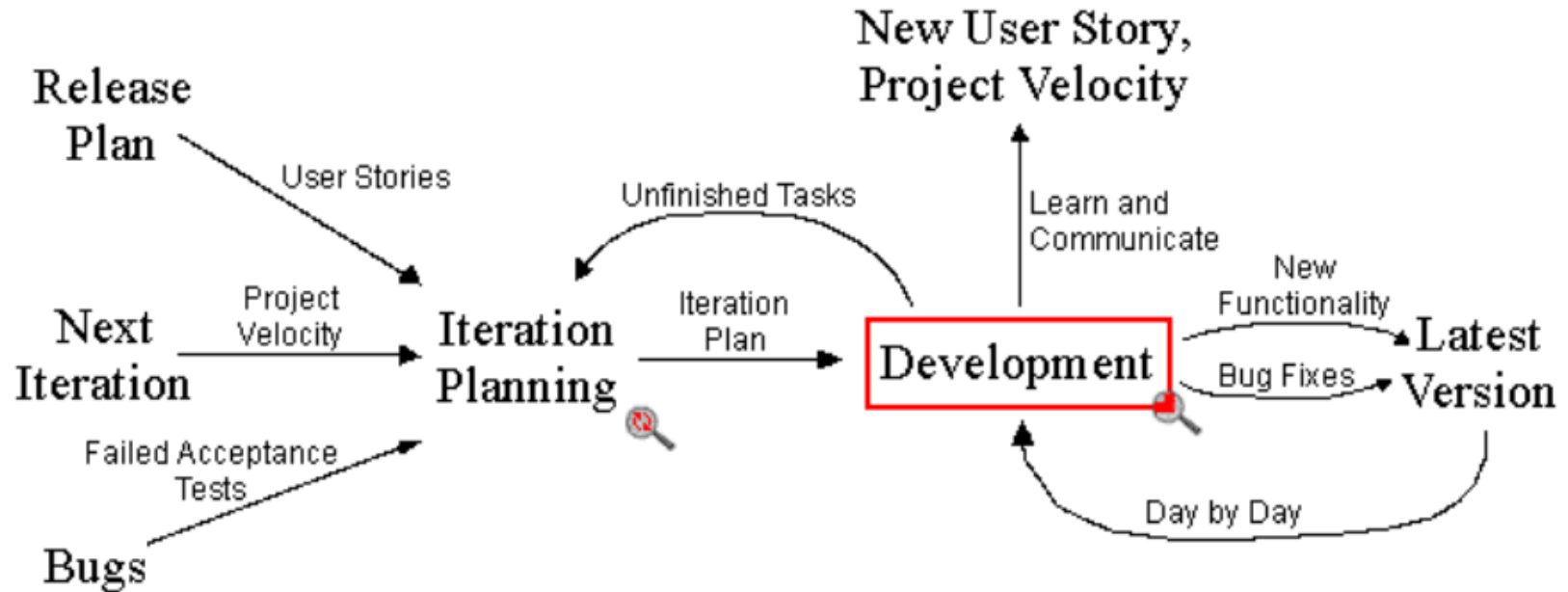


Release planning

- Together developers and customers create a set of stories to be implemented as the first (or next) release
- Make frequent small releases
 - A (re)useable, testable system that makes good business sense delivered early is desired
- You may plan by time or by scope:
 - how many stories can be implemented before a given date (time)
 - how long a set of stories will take to finish (scope)



Iteration



Iteration planning

- Iteration planning takes place before each iteration
- Unimplemented user stories are selected from the iteration plan for the upcoming iteration
- User stories implemented in the previous iterations but did not pass acceptance tests are also selected
- The total user stories selected should have the total estimates up to the project velocity from the last iteration
 - how much work is getting done on your project
 - simply adding up the estimates of the user stories that were finished during the iteration



Designing

- Simplicity in design
 - Testable, Understandable, Browsible, and Explainable
- Use Class Responsibilities and Collaboration (CRC) cards for design sessions.

Class Name	
Responsibilities	Collaborators

Student	
Student number Name Address Phone number Enroll in a seminar Drop a seminar Request transcripts	Seminar



Coding

- Code must be written to agreed coding standards (different languages)
- Code the unit test first (Test Driven Development)
- All production code is pair programmed
- Code review
- Integrate often



Coding (for test)

- All code must have unit tests
- All code must pass all unit tests before it can be released
- When a bug is found, tests are created
- Acceptance tests are run often
 - created from user stories
 - customer specifies scenarios to test when a user story has been correctly implemented
 - one story can have one or many acceptance tests
 - acceptance tests are black box system tests
 - a user story is not considered complete until it has passed its acceptance tests



Managing

- Give the team a dedicated open work space
- Set a sustainable pace
- A stand up meeting starts each day
- Move people around



Advantages and Disadvantages of XP

- Continuous testing and refactoring practices help create stable well performing systems with minimal debugging
- Simplicity value implies creating a clear, concise **code that is easy to read and change** in the future if needed
- The workable results can be **delivered very soon** and only necessary features are built
- **Documentation is reduced** as bulky requirement documents are substituted by user stories
- **Constant communication** provides a high level of visibility and accountability
- **Pair programming** is higher-quality products with fewer bugs
- **Customer engagement** ensures their satisfaction as their participation in the development and testing process



Advantages and Disadvantages of XP

- The customer has **no clear picture** of the end result, which makes it almost unrealistic to accurately estimate scope, cost, and time
- **Regular meetings** with customers often take a great deal of time that could instead be spent on actual code writing
- **Lacking clear requirements** and specifications, leading to project scope creep
- The transition from **traditional methods** to **extreme programming** demands significant cultural and structural changes
- **Pair programming** takes more time and doesn't always work right due to the human factor and character incompatibility
- XP works best with collocated teams and customers present in person to conduct face-to-face meetings, limiting its application with **distributed teams**
- Sometimes customers have **neither the desire, time, nor expertise** to participate in product development
- Over-focusing on **code over design**, lack of quality assurance, code duplication, and poor results with inexperienced developers.



XP vs Scrum

- Scrum is commonly associated with self-organizing teams. It also usually has sprints that are 2 to 4 weeks long, while XP iterations are shorter, taking 1 to 2 weeks
- Besides, XP is much more flexible with possible changes within iterations, while Scrum doesn't allow any modifications after the sprint backlog is set
- in XP the customer prioritizes features and decides on the order of their development, but in Scrum the team itself determines what to work on first
- Scrum's main roles are Product Owner, Scrum Master, and Scrum Team, which are different from those in XP
- Incorporating XP practices and Scrum techniques is considered quite effective with XP focusing on engineering aspects and Scrum organizing the process.



XP vs Kanban

- Kanban puts a lot of focus on visualizing the development process and strictly limits the number of features developed at a time
- It' s also characterised by a continuous workflow while XP has separate iterations, even though both suggest small frequent releases and a high level of flexibility and adaptiveness to the changing requirements
- The roles in Kanban are not strictly defined.



When to use XP

Team's size, structure, and expertise

- **Highly-adaptive development.** development teams adapt to fast-changing requirements
- **Risky projects.** to avoid problems connected with working on a new system, especially when a customer sets strict deadlines for a project
- **Small teams.** XP practices are efficient for teams that don't exceed 12 people
- **Automated testing.** XP requires the developers' ability to create and run unit tests, and availability of the necessary testing tools
- **Readiness to accept new culture and knowledge.** It's important to embrace change, and have an experienced coach
- **Customer participation.** work side-by-side, make sure your client is always available to provide input until a project ends.



U

O

W

CSIT314

Software Development

Methodologies

Rational Unified Process (RUP)



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

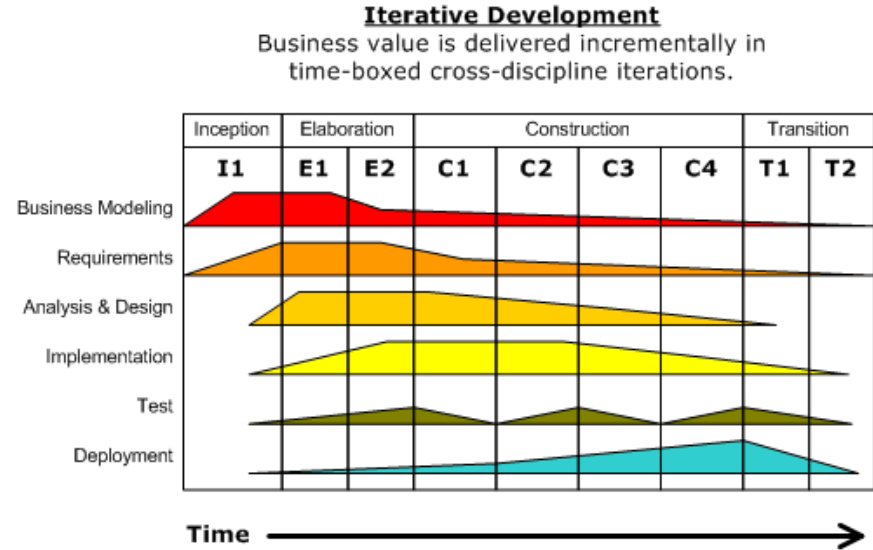
RUP

- Six principles to follow when designing any software project
 - Develop iteratively
 - Manage requirements: keep in mind the requirements set by customers and will be changed by users
 - Use components: test individual components and reusability
 - Model visually: use diagrams to represent all major components, users, and their interaction (UML)
 - Verify quality: always make testing a major part of the project at any point of time
 - Control changes: may be created by many teams, sometimes in various locations, different platforms may be used, etc.
 - make sure that changes made to a system are synchronized and verified constantly.



Rational Unified Process (RUP)

- Two dimensions
 - Horizontal axis represents time and shows the lifecycle aspects of the process as it unfolds
 - plan the scope and acceptance criteria
 - evaluate the resources and create actionable baseline
 - create, write, collaborate and test your software
 - releases the project
 - more later
 - Vertical axis represents core process workflows, which group activities logically by nature



1. Life-Cycle Objective Milestone
2. Life-Cycle Architecture Milestone
3. Initial Operational Capability Milestone
4. Product Release



A model of RUP

- A process describes who is doing what, how and when. The RUP is represented using five primary elements
 - Roles: who
 - Activities: how (also known as tasks)
 - Artifacts: what (also known as work products)
 - Workflows: when
 - Disciplines: container (configuration and change management, project management and environment)



RUP building blocks

- Roles (who) – A Role defines a set of related skills, competencies and responsibilities.
- Work Products (what) – A Work Product represents something resulting from a task, including all the documents and models produced while working through the process.
- Tasks (how) – A Task describes a unit of work assigned to a Role that provides a meaningful result.



Start at Inception

- Establish the project's software scope and acceptance criteria (requirements)
- Establish the main use-case scenarios that define the core functionality of the project.
- Evaluate alternative architectures; create the beginnings of your preferred candidate architecture.
- Estimate the overall cost and schedule for the project.
- Estimate risks
- Produce the “business case” – the business justification and benefit from having the functionality provided.
- Create a work plan for the following elaboration phase



Inception activities

- Formulate the scope of the project: capture the context and the most important requirements and constraints
- Plan and prepare a business case
- Evaluate alternatives:
 - risk management, staffing, project plan, and trade-offs among cost, schedule, and profitability
- Synthesize a candidate architecture



Inception outcomes

- A vision document
 - the core project's requirements, key features, main constraints
- List all use cases and actors already identified
- An initial project glossary (features)
- An initial business case, which includes the following
 - Business context
 - Success criteria (revenue projection, market recognition, and so on)
 - Financial forecast
- An initial risk assessment
- An initial project plan, which shows the phases and iterations

(example later)



Vision document

- What problem are we trying to solve? (Problem Statement)
 - describe the problem, who it affects, how it affects them, and what type of solution would ease the pain (solution can ...)
- Who are the stakeholders
 - persons or organizations. They may be affected by it either directly or indirectly
- Who are the users
- What are their respective needs
- What are the product features
- What are the functional requirements (Use Cases)
- What are the non-functional requirements
- What are the design constraints?



Identify features

- Example:
 - Record statistics
 - Time, Software defects and Source code size
 - Reporting
 - Personal reports and Team reports
 - Viewing
 - Statistics



Sketch high level use cases

- Software Engineer :
 - Create project, Enter data, Count items
 - Use on line help when needed, Create report
- Process administrator:
 - Configure system, Configure project
 - Install tools



Initial Project Plan

- “Iteration plan” for rest of inception phase
 - Artefact, who responsible, when due
- Example:
 - Plan done by 10th Oct
 - Environment set up ...
 - Vision complete by ...
 - Supplementary requirements
 - Initial use cases by ..
 - Risk list
 - Test plan
 - Finish “Inception Iteration”



Inception evaluation

- At the end of the inception phase is the first major project milestone. The evaluation criteria for the inception phase are:
 - Stakeholder concurrence on scope definition and cost and schedule estimates
 - Requirements understanding evident through use cases
 - Credibility of cost & schedule estimates, priorities, risks and development process
 - Depth and breadth of prototype architecture
 - Actual vs planned expenditure



Elaboration

- Requirements (resources) continue to be refined and additional use cases will be identified.
- Elaboration objectives:
 - Define, validate, and baseline the system architecture
 - Baseline the vision
 - Baseline a plan for the construction phase
 - Demonstrate that the baseline architecture will support this vision for a reasonable cost in a reasonable time



Elaboration activities

- The vision is elaborated and fully articulated.
- Develop solid understanding the vision of critical use cases.
- The process, the infrastructure, and the development environment are elaborated.
- Put in place development environment, tools, test automation systems, etc.
- Elaborate the system architecture:
 - Select subsystems that can be purchased
 - Integrate and assess selected components against primary use cases



Elaboration outcomes

- A use-case model (at least 80% complete) in which all use cases have been identified in the use-case model survey, all actors have been identified
 - Most use-case descriptions have been developed
- Supplementary requirements that capture the non-functional requirements
- A software architecture description
- An executable architectural prototype (database design)
- A revised risk list and a revised business case
- A development plan for the overall project
 - major and minor iterations and related deliverables
- A preliminary user manual (optional)



Elaboration evaluation

- Is the product vision stable?
- Is the architecture stable?
- Does the executable demo show that major risk elements have been addressed and resolved?
- Is the construction plan sufficiently detailed?
- Do all stakeholders agree that the current vision can be achieved with the current development plans?



Construction

- Develop remaining components and application features
 - coding, testing, and delivering iterated versions
 - cannot escape project creep
- Integrate all parts into the product
- Test all features thoroughly
- Achieve useful versions (alpha, beta, and other test releases) as rapidly as practical



Construction objectives

- Achieving useful versions as quickly as possible, or system requirement
- Minimized development costs by optimizing resource usage
 - avoiding unnecessary changes, scrap and rework
- Achieving adequate quality as rapidly as is practical



Construction activities

- Complete development and testing against the defined evaluation criteria
- Resource management, resource control, and process optimization
- Assessment of product releases against acceptance criteria for the vision



Construction outcome

- A product ready to put in the hands of its end users. It should consists of the following:
 - The software product integrated on the adequate platforms
 - The user manuals
 - A description of the current release



Construction evaluation

- Decide whether the software, the sites and the users are ready to become operational without exposing to high risks
 - Is the product stable and ready to be exposed to the user environment?
 - Are the stakeholders ready for the product transition into the user environment?
 - Are the actual resource expenditures still acceptable relative to the projected expenditures?



Iterations

- Plan (started during Inception and refined in Elaboration) will have defined a series of iterations
 - Each Iteration
 - List of use-cases that it incorporates
 - Time schedule (time boxing)
 - Leads to a “build” that can be release to others



Iteration releases – a chance to replan

- With each iteration release, you revise your plan
 - Maybe you are lagging
 - need to reduce the scope
 - move some features from next phase to later phase
 - Maybe you have negative feedback from the users/customers
 - Re-analysis, re-design, re-work



Transition

- Move the software product to the users
- Phase starts when system is mature enough to be deployed in the end-user domain
 - Usable subset of the system has been completed to an acceptable level of quality
 - User documentation is available
- Phase includes:
 - Beta testing to validate the new system against users' expectations
 - Parallel operation with any existing system that the project is replacing
 - Training of users and maintainers
 - Rollout of the product to the marketing, distribution, and sales team



Transition objectives

- Achieve user self-supportability
- Achieve stakeholder concurrence that deployment baselines are complete and consistent with the evaluation criteria of the vision
- Achieve final produce baseline as rapidly and cost effectively as practical



Transition activities

- Deployment-specific engineering, i.e., commercial packaging and production, sales rollout and etc.
- Tuning activities, including bug fixing and enhancement for performance and usability
- Assessing the deployment baselines against the vision and the acceptance criteria for the product



Transition evaluation

- You decide whether the objectives were met and whether you should start another development cycle. The primary evaluation criteria are:
 - Is the user satisfied?
 - Are the actual resources expenditures versus planned expenditures still acceptable?



U

O

W

CSIT314

Software Development Methodologies

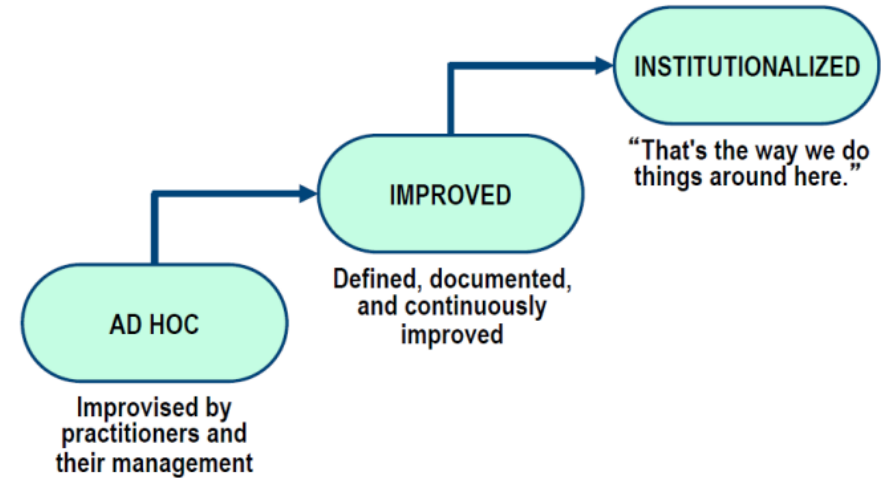
CMMI



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Definition of Process

- A set of interrelated activities, which transform inputs into outputs, to achieve a given purpose.
- The quality of a system is highly influenced by the quality of the (also inline with software design methodology) process used to acquire, develop, and maintain it.



Ad Hoc Processes

- Process descriptions are not carefully followed or enforced
 - Can be very different from people to people
- Understanding of the current status of a project is limited
- Performance is highly dependent on current participants
- XP as an example



Improved Processes

- Process descriptions are consistent with the way work actually is done
- Processes are supported visibly by management and others
- They are well controlled—process fidelity is evaluated and enforced
- There is constructive use of product and process measurement.



Institutionalised Processes

- The organization builds an infrastructure that contains effective, usable, and consistently applied processes
- The organizational culture conveys the process
- Management nurtures the culture
- Culture is conveyed through role models and recognition



Benefits of improving processes

- Better understand of how things occur in an organization
- More predictability
- People working more effective



CMMI

- Capability Maturity Model Integration is a proven industry framework to improve product quality and development efficiency for both hardware and software
 - Sponsored by US Department of Defense in cooperation with Carnegie Mellon University and the Software Engineering Institute (SEI)
 - Many companies have been involved in CMMI definition such as Ericsson
 - CMMI has been established as a model to improve business results
- CMMI uses 5 levels to describe the maturity of the organization
 - Emphasis on business needs, integration and institutionalization

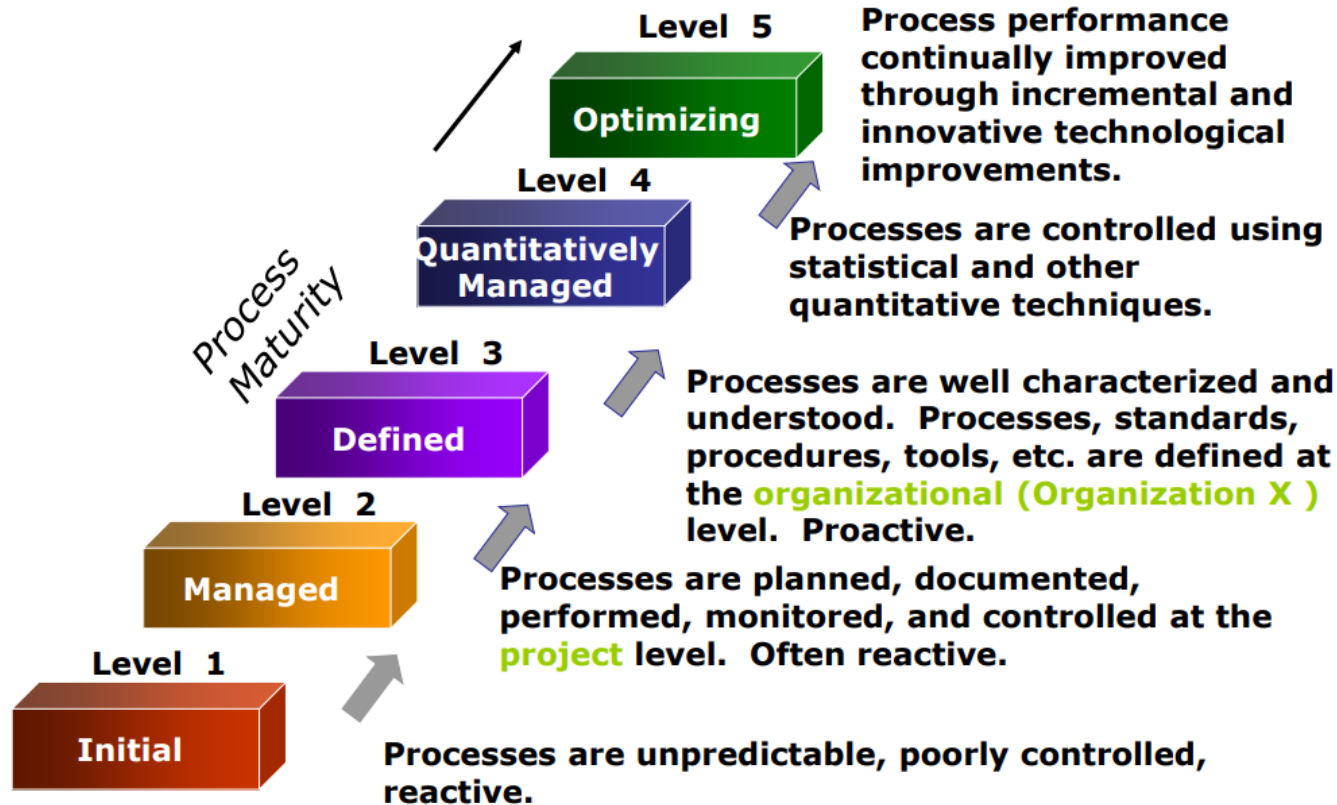


CMMI for Process Improvement

- Use CMMI in process improvement activities as
 - a collection of best practices
 - framework for organising and prioritizing activities
 - support for the coordination of multi-disciplined activities that might be required to successfully build a product
 - emphasize the alignment of the process improvement objectives with organizational business objectives



5 Maturity Levels



Maturity Level 1

- Maturity Level 1 deals with performed processes
 - unpredictable, poorly controlled, reactive
- The process performance may not be stable and may not meet specific objectives such as quality, cost, and schedule, but useful work can be done.



Maturity Level 2

- Maturity Level 2 deals with the Project Level and managed processes
- A managed process is a performed process
 - Planned and executed in accordance with policy
 - Employs skilled people
 - Adequate resources are available
 - Controlled outputs are produced
 - Stakeholders are involved
 - The process is reviewed and evaluated for adherence to requirements
- Processes are planned, documented, performed, monitored, and controlled at the project level
- The managed process comes closer to achieving the specific objectives such as quality, cost, and schedule.



Maturity Level 3

- Maturity Level 3 deals with defined processes (at the Organization Level)
- A defined process is a managed process that
 - Well defined, understood, deployed and executed across the entire organization
 - Project or local tailoring is allowed, however it must be based on the organization' s set of standard processes and defined per the organization' s tailoring guidelines



CMMI Process Areas

- Basic functions that need to be performed (within each of the 5 Maturity Levels)
 - With a set of goals and practices
- CMMI process areas are classified into four groups:
 - Project Management
 - Process Management
 - Engineering
 - Support

Project Management

- Quantitative Project Management
 - *quantitatively* manage the project's defined process to achieve the project's established quality and process-performance objectives
- Integrated Project Management
 - establish and manage the project and the involvement of the relevant stakeholders according to an integrated and defined process that is tailored from the organization's set of standard processes.



Project Management

- Risk Management
 - to identify potential problems before they occur so that risk-handling activities can be planned and invoked as needed across the life of the product or project to mitigate adverse impacts on achieving objectives
- Project Planning
 - establish and maintain plans that define project activities.



Project Management

- Project Monitoring & Control
 - provide an understanding of the project's progress so that appropriate corrections can be taken when the project's performance deviates significantly from the plan.
- Supplier Agreement Management
 - manage the acquisition of products from suppliers for which there exists a formal agreement.



Process management

- Organisational Innovation & Deployment
 - incremental and innovative improvements support the organization's quality and process-performance objectives
- Organisational Process Performance
 - a quantitative understanding of the performance, and to provide the process performance data, baselines, and models to quantitatively manage the organization's projects.



Process management

- Organisational Process Focus
 - plan and implement organizational process based on the current strengths and weaknesses
- Organisational Process Definition
 - establish and maintain a usable set of organizational process assets
- Organizational Training
 - develop the skills and knowledge of people so they can perform their roles effectively and efficiently



Engineering

- Requirements Development
 - to produce and analyze customer, product, and product-component requirements
- Technical Solution
 - design, develop, and implement solutions to requirements.
Solutions, designs, and implementations either singly or in combination as appropriate
- Product Integration
 - assemble the product from the product components, ensure that the product, as integrated, functions properly, and deliver the product.



Engineering

- Verification & Validation
 - ensure that selected work products meet specified requirements
 - fulfills its intended use when placed in its intended environment
- Requirements Management
 - manage the requirements of the project's products and product components and to identify inconsistencies between those requirements and the project's plans and work products



Support

- Causal Analysis & Resolution
 - identify causes of defects and take action to prevent them from occurring in the future
- Decision Analysis & Resolution
 - analyze possible decisions and evaluate alternatives against established criteria



Support

- Measurement & Analysis
 - to support management information needs
- Process & Product Quality Assurance
 - provide objective insight into processes and associated work products
- Configuration Management
 - configuration identification, control, status accounting, and audits



CMMI Process Areas

Maturity Level	Project Management	Engineering	Process Management	Support
5 Optimizing			Organizational Innovation & Deployment	Causal Analysis & Resolution
4 Quantitatively Managed	Quantitative Project Mngt		Organizational Process Performance	
3 Defined	Integrated Project Mngt Risk Management	Requirements Development Technical Solution Product Integration Verification Validation	Organizational Process Focus Organizational Process Definition Organizational Training	Decision Analysis & Resolution
2 Managed	Project Planning Project Monitoring & Control Supplier Agreement Mngt	Requirements Mngt		Measurement & Analysis Process & Product Quality Assurance Configuration Mngt
1 Initial				



CMMI & Agile

- Backlog grooming: common agile technique used by Scrum teams to produce a prioritized backlog of epics and user stories before and during a sprint
 - Project planning
 - Requirement development
 - Requirement management



CMMI & Agile

- User stories
 - Requirement development
 - Requirement management
 - Verification
 - Integrated project management



CMMI & Agile

- Team estimating/Planning : an agile estimation technique that establishes relative sizing using story points and rough order of magnitude estimation
 - Requirement management
 - Risk management
 - Project planning



CMMI & Agile

- Daily standup/Daily Scrum: Daily Standup Meeting is used as a way to identify issues and risks earlier than a traditional project (“fail fast”), and to increase collaboration between agile team members
 - Risk management
 - Project monitoring & control
 - Integrated Project management



CMMI & Agile

- Pair programming: usually increases the initial cost of programing, but it more than pays for itself in increased code quality.
 - Requirement development
 - Verification
- Continuous integration:
 - Verification
 - Product integration
 - Configuration management



CMMI Certification & Appraisal

- An organization gets accredited to certification level i according to CMMI if all process areas are governed that belong to maturity level 1 to i
- The certification process can be complicated and lengthy and is therefore rather suitable for big organizations
- Roughly estimated, around 2-3 years are necessary to go from maturity level i to $i+1$.



U

O

W

CSIT314

Software Development Methodologies

Data-driven Software Development



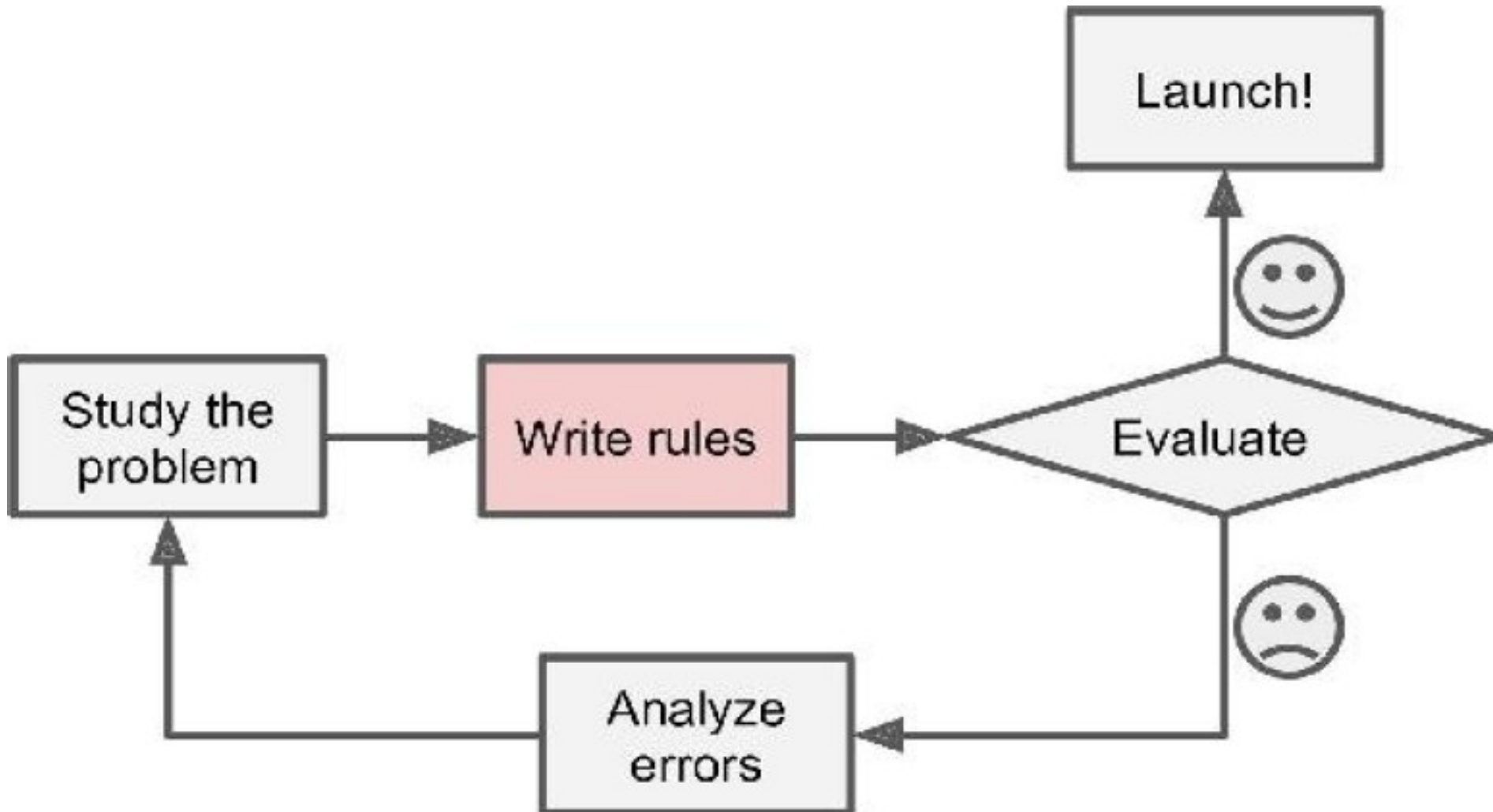
UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Data-driven software development

✓ Two perspectives:

- Developing data-driven software products
 - E.g. Many Artificial Intelligence (AI) applications are data-driven.
 - Also referred to as **AI Engineering**
- Leveraging **software development data** to generate insights and build tool support for business analysts, software developers, project managers, etc.:
 - E.g. Help business analysts identify requirements from app reviews
 - E.g. Help project managers predict delays and risks
 - E.g. Help agile team estimate efforts (story points)
 - E.g. Help software developers locate security vulnerabilities and bugs
 - E.g. Automatically generate code comments, commit messages, test cases, etc.
 - Also referred to as **Software Analytics**
 - Many large organizations (e.g. Microsoft, Google and Facebook) deployed Software Analytics in their software development process.

The traditional software development approach



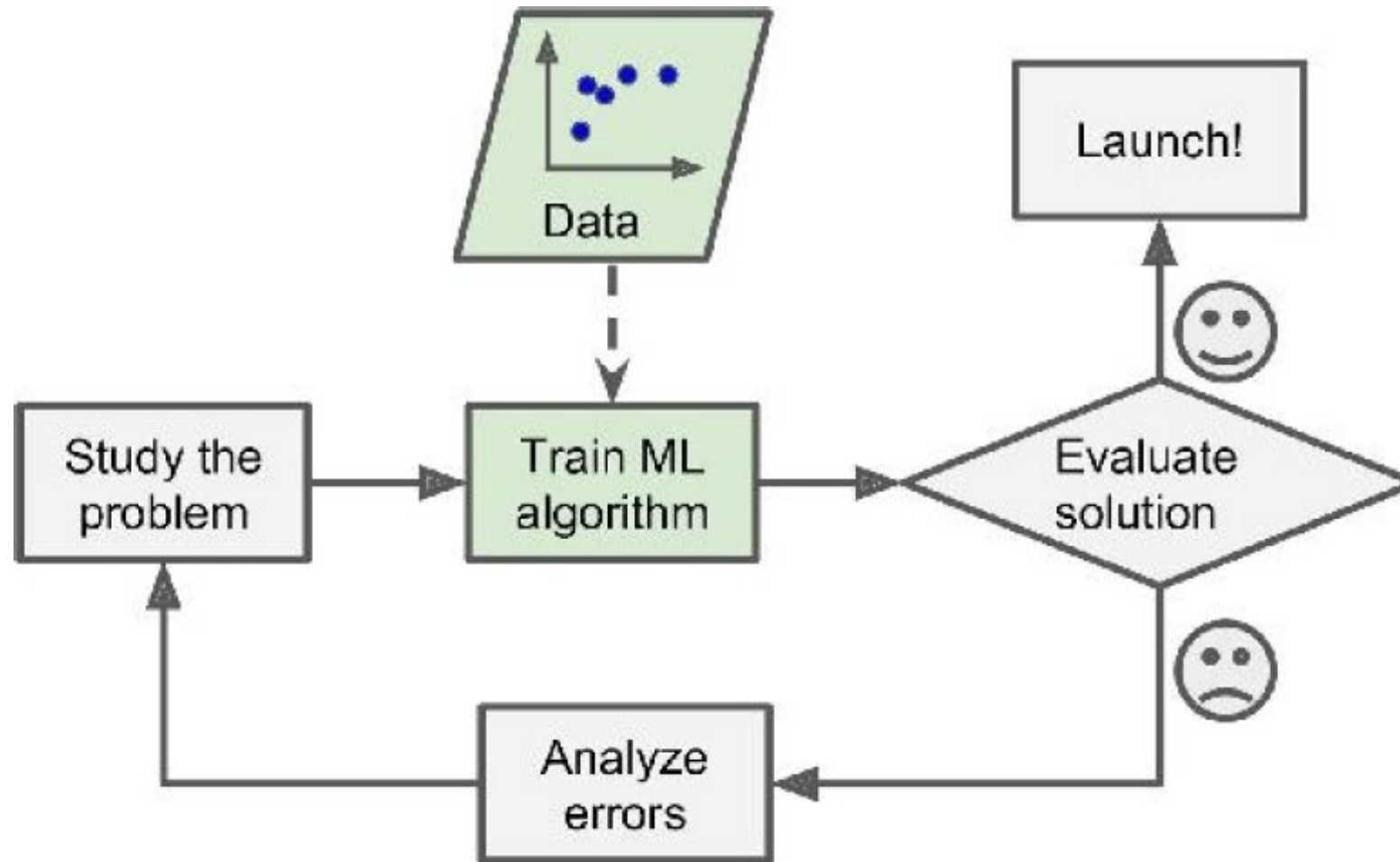
The traditional approach – an example

- Example: the weather problem – condition for playing a cricket game. How can we build an software app which can predict if a cricket game is going to play or not?

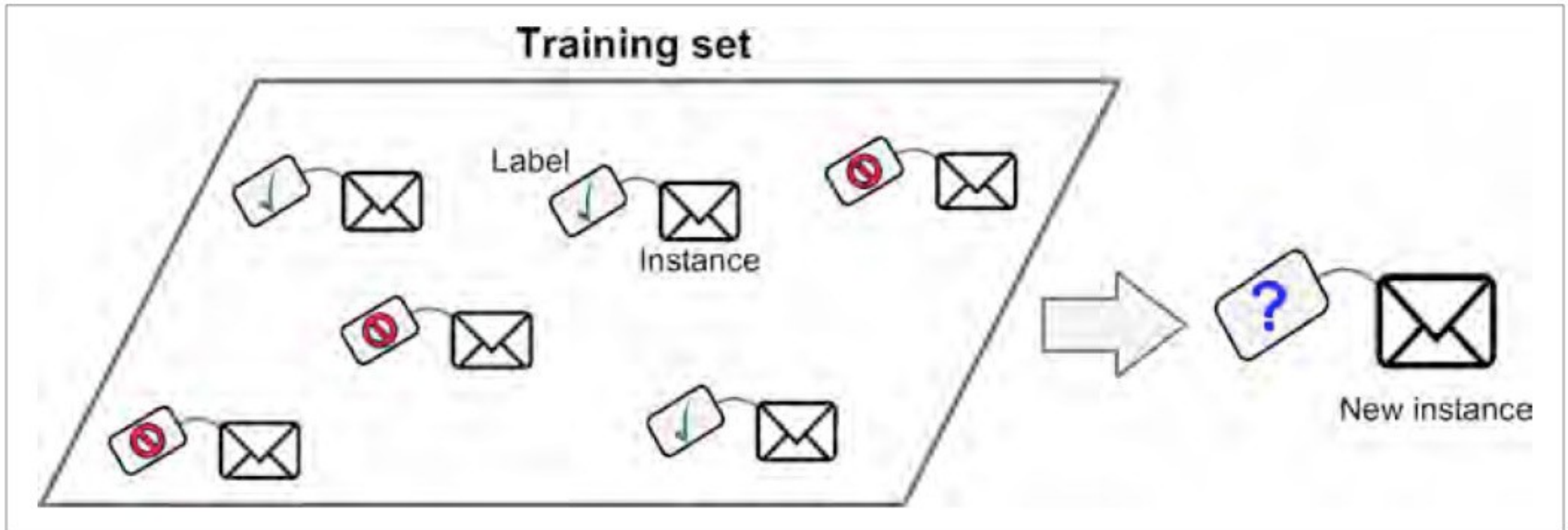
Outlook	Temperature	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	Normal	False	Yes
...	...	...	...	...

```
If outlook = sunny and humidity = high then play = no
If outlook = rainy and windy = true then play = no
If outlook = overcast then play = yes
If humidity = normal then play = yes
If none of the above then play = yes
```


The data-driven approach



Data-driven software systems

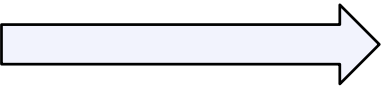


Supervised learning (Machine Learning)

Data-driven software development

Develop data-driven learning models

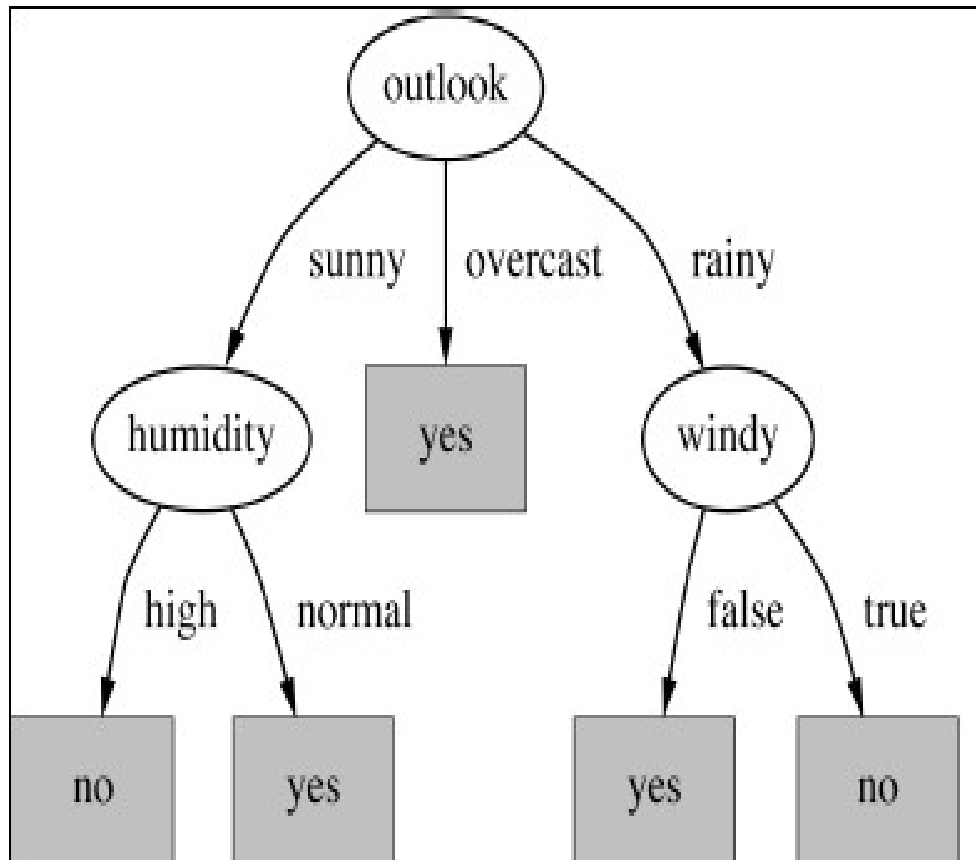
- How do we (automatically) build a model (or an app) for predicting if a game is played?
 - This model is a **function** which can be inferred (learned) from labelled **training data** (aka data driven)
 - $f(\text{outlook}, \text{temperature}, \text{humidity}, \text{windy})$ returns true or false.
 - Features/attributes**: input variable, e.g. outlook, temperature
 - Target/dependent variable**, e.g. play = yes or no
 - Training set** consists of training examples
 - Classification = prediction.



Outlook	Temp	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

The AI/ML approach – an example (cont.)

- Some basic learning models (learners)
 - Decision Trees**



Outlook	Temp	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

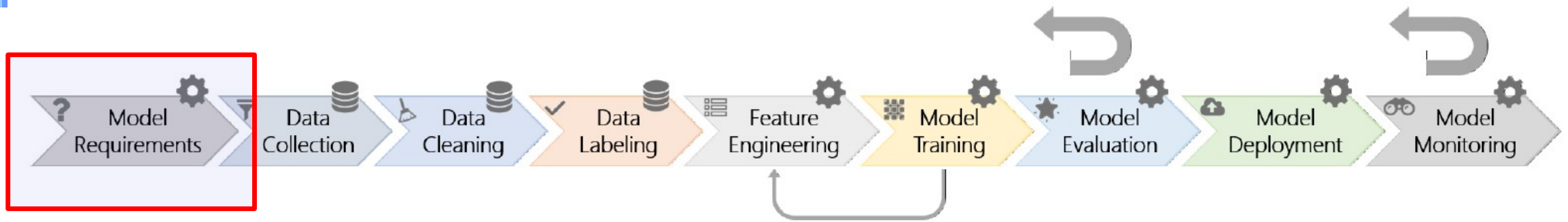
Source: [Data Mining: Practical Machine Learning Tools and Techniques](#), 3rd Edition by Ian H. Witten, Eibe Frank, Mark A. Hall

The AI/ML approach – an example (cont.)

- ✓ Some advanced learning models (learners)
 - Random Forests (RF)
 - An ensemble learning method
 - A significant improvement of the decision tree approach
 - Generating many classification trees, each of which is built with random subset of variables at each node split, and aggregates into the individual results using voting
 - Neural Networks

and many other ML models.

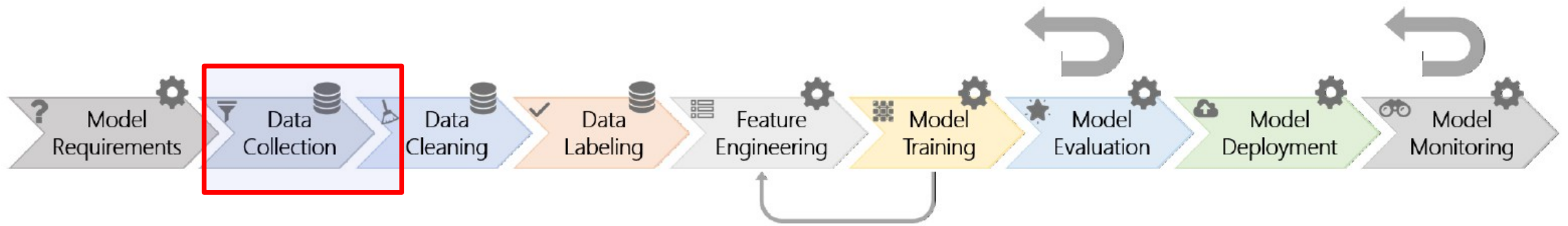
Data-driven software development lifecycle



Model requirements:

- Identify which components of the existing (or new) product are feasible to implement with machine learning/ML, data-driven technology.
- Elicit requirements for these data-driven/ML components
- Determine what types of models (e.g. supervised vs. unsupervised).

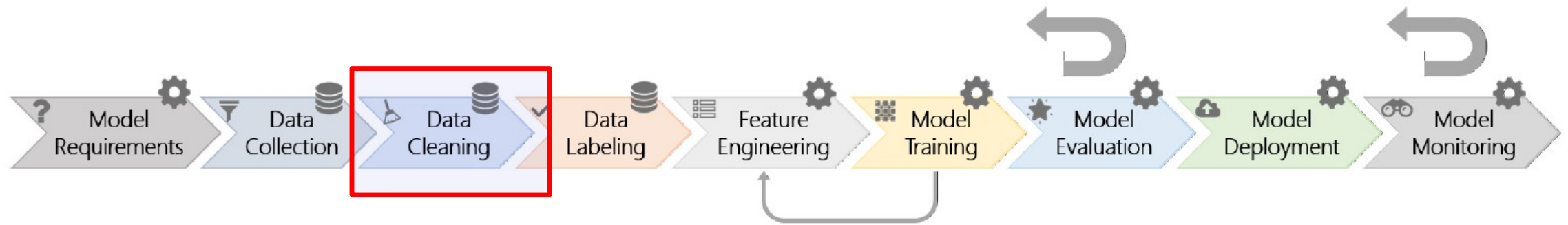
Data-driven software development lifecycle



✓ Data collection:

- Look for available datasets (e.g. internal data, public data, etc.) or build their own datasets.
- May use a mix of datasets (e.g. pre-training on public datasets, and then (post-)training on their own dataset).

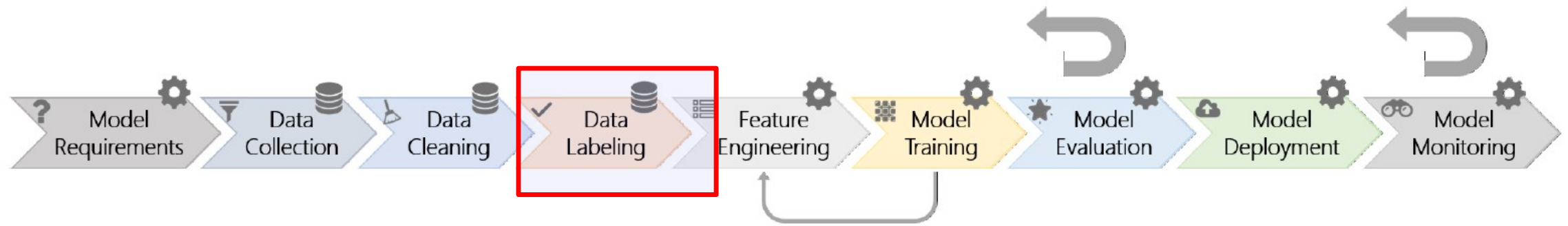
Data-driven software development lifecycle



✓ Data cleaning:

- Remove inaccurate or noisy data records.
- Filling missing data

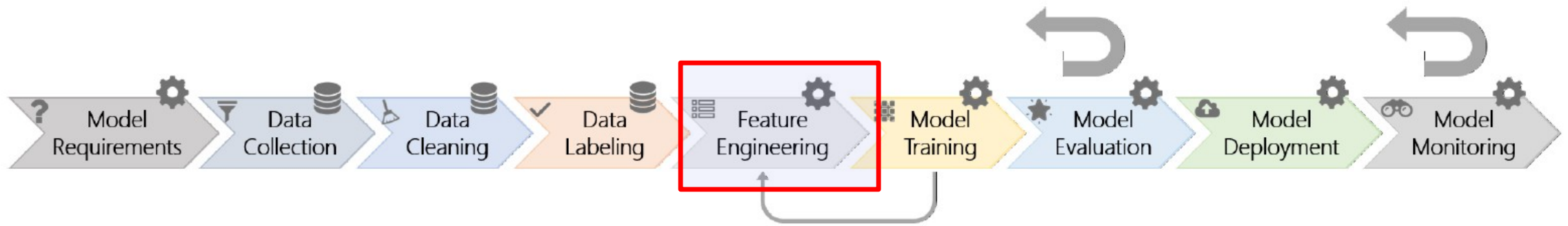
Data-driven software development lifecycle



✓ Data labelling:

- Assign ground-truth labels for each data record
- Can be done by software engineers, domain experts or crowd workers (e.g. Mechanical Turk).

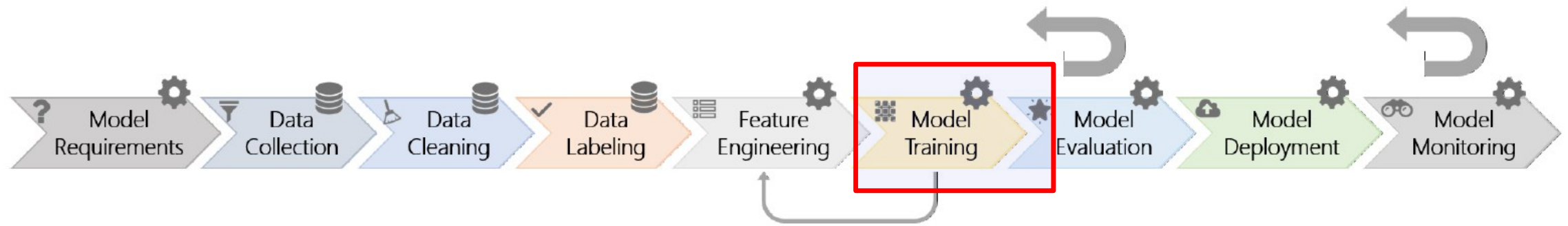
Data-driven software development lifecycle



Feature engineering:

- Extract features from data records – **feature extraction**
- Select informative features (e.g. remove correlated features) – **feature selection**

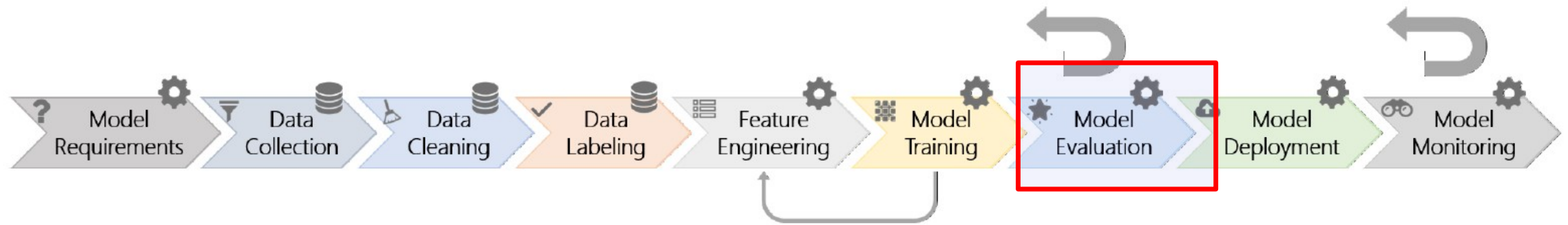
Data-driven software development lifecycle



Model training:

- Split training vs test data
- Choose a model or a set of models (learning algorithms)
- Training the chosen models
- Tuning hyper-parameters

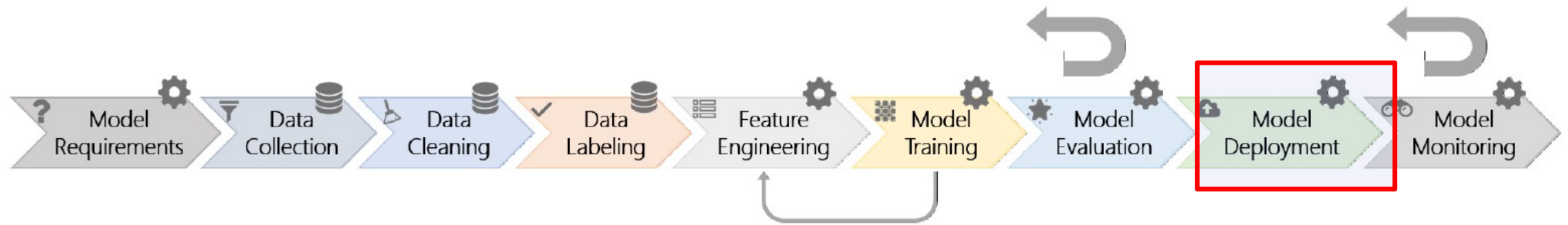
Data-driven software development lifecycle



Model evaluation:

- Assess the model's performance on test data (precision, recall, F-measure, AUC, MAE, etc.)

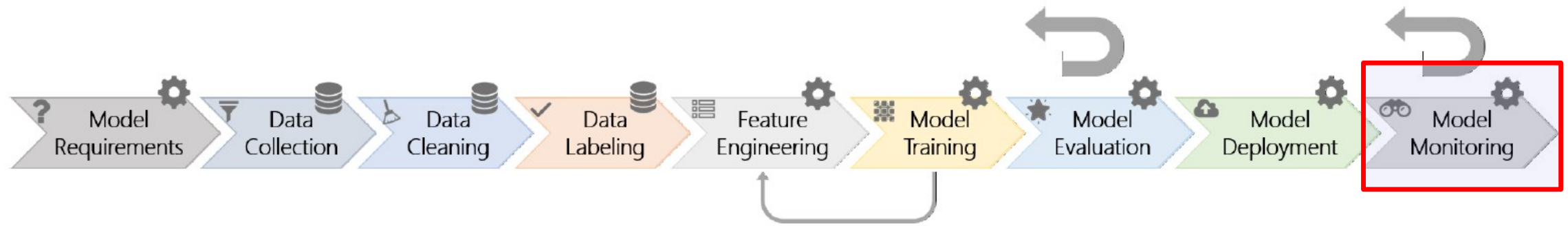
Data-driven software development lifecycle



✓ Model deployment:

- Deploy on the targeted devices

Data-driven software development lifecycle

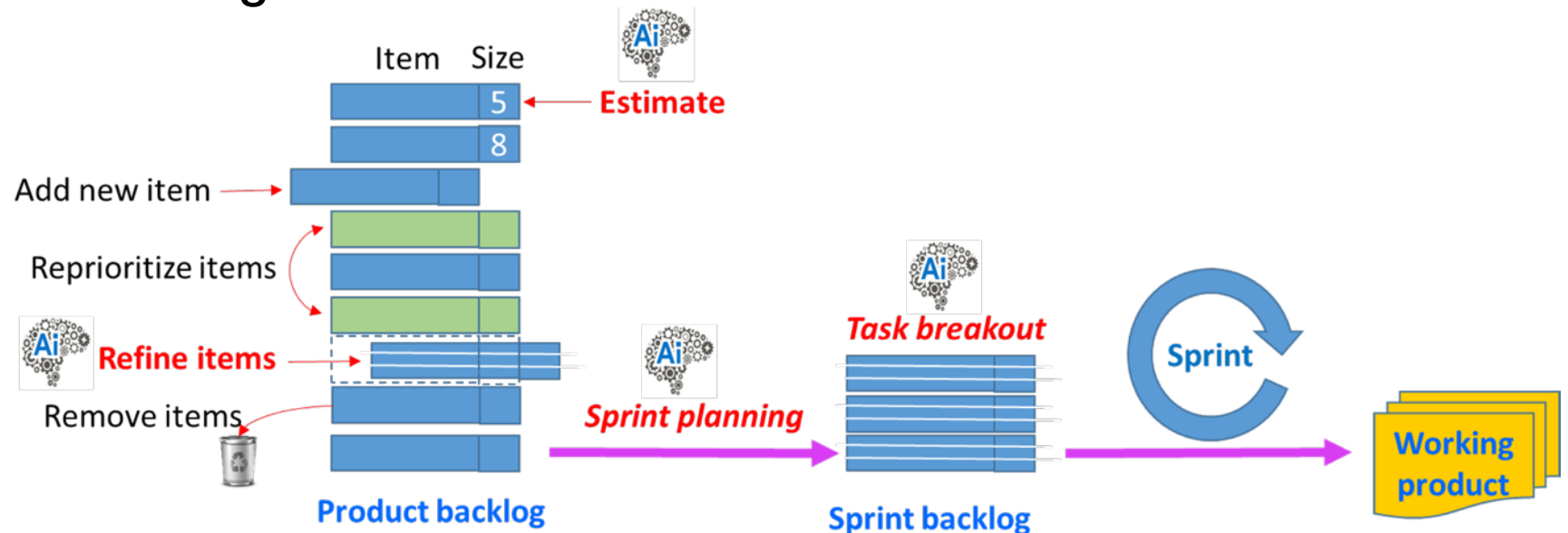


Model monitoring:

- Continuously monitor for performance and errors

Software Engineering

- ✓ Requirements
- ✓ Design
- ✓ Implementation
- ✓ Verification and Validation
- ✓ Maintenance and evolution
- ✓ Software Project Management



AI for Software Engineering (AI4SE)



Project Name: _____		Test Case Template			
Test Case ID: <u>TC_10</u>		Test Designed by: <u><Name></u>			
Test Priority: <u>(Low/Medium/High): Med</u>		Test Designed date: <u><Date></u>			
Module Name: <u>Google login screen</u>		Test Executed by: <u><Name></u>			
Test Title: <u>Verify login with valid username and password</u>		Test Execution date: <u><Date></u>			
Description: <u>Test the Google login page</u>					
Pre-conditions: <u>User has valid username and password</u>					
Dependencies: _____					
Step	Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail) / Notes
1	Navigate to login page	<u>User: admin@gmail.com</u>	<u>User should be able to login</u>	<u>User is redirected to Home</u>	<u>Pass</u>
2	Provide valid email	<u>password: 1234</u>		<u>User is redirected to Home</u>	
3	Click on login button			<u>login</u>	
Post-conditions: _____					
User is validated with database and successfully login to account. The account session details are logged in database.					



Issue, bug reports, user stories

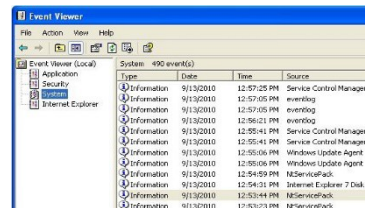
Product backlog

Commit messages

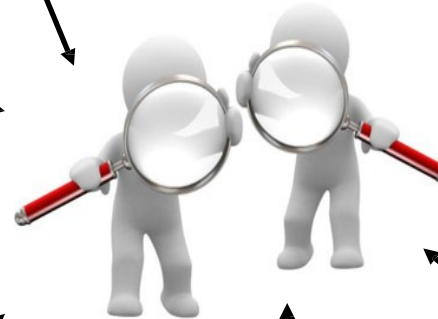
Test cases

Source code

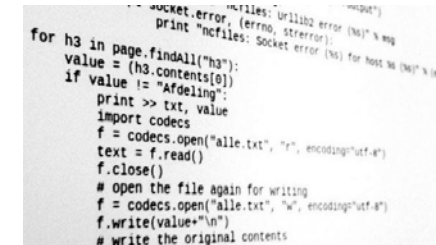
App reviews



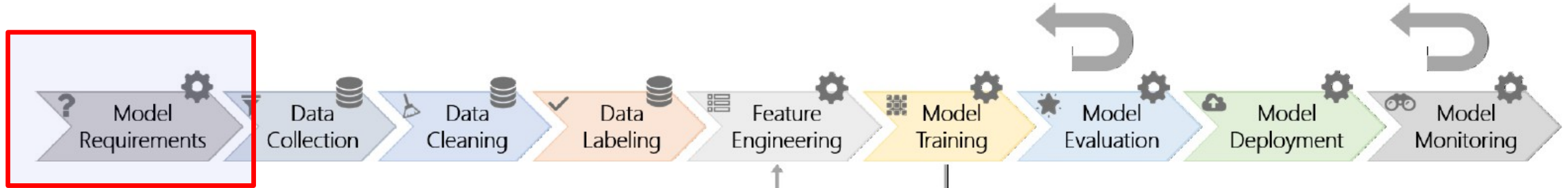
System events



Usage logs



AI4SE: delay prediction in software projects



Model requirements:



A screenshot of a project management dashboard showing a list of projects with their status. The table has columns for project names and their status (e.g., 'ON TRACK', 'CAUTION (LOW)', 'AT RISK').

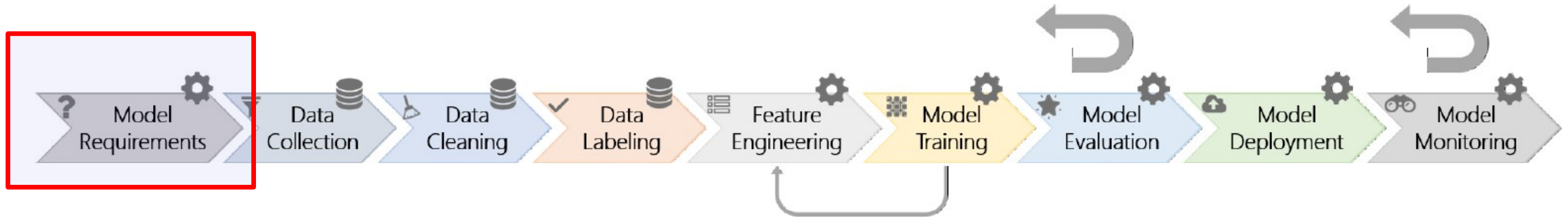
Project Name	Status
Pearson's	ON TRACK
Alphas	ON TRACK
Benson	CAUTION (LOW)
Care-X	ON TRACK
Emerald Cosmetics	ON TRACK
Finch and Lovey	ON TRACK
Hendricks	ON TRACK
Jameway	ON TRACK
Pard	ON TRACK
Sveda	ON TRACK
Ballyhoos	ON TRACK
Castle & Lambert	ON TRACK
CCRx International	ON TRACK
Emerald Gold	ON TRACK
Franti	ON TRACK
HLS, Inc.	ON TRACK
Mises	ON TRACK
POSH	ON TRACK
Univer	ON TRACK



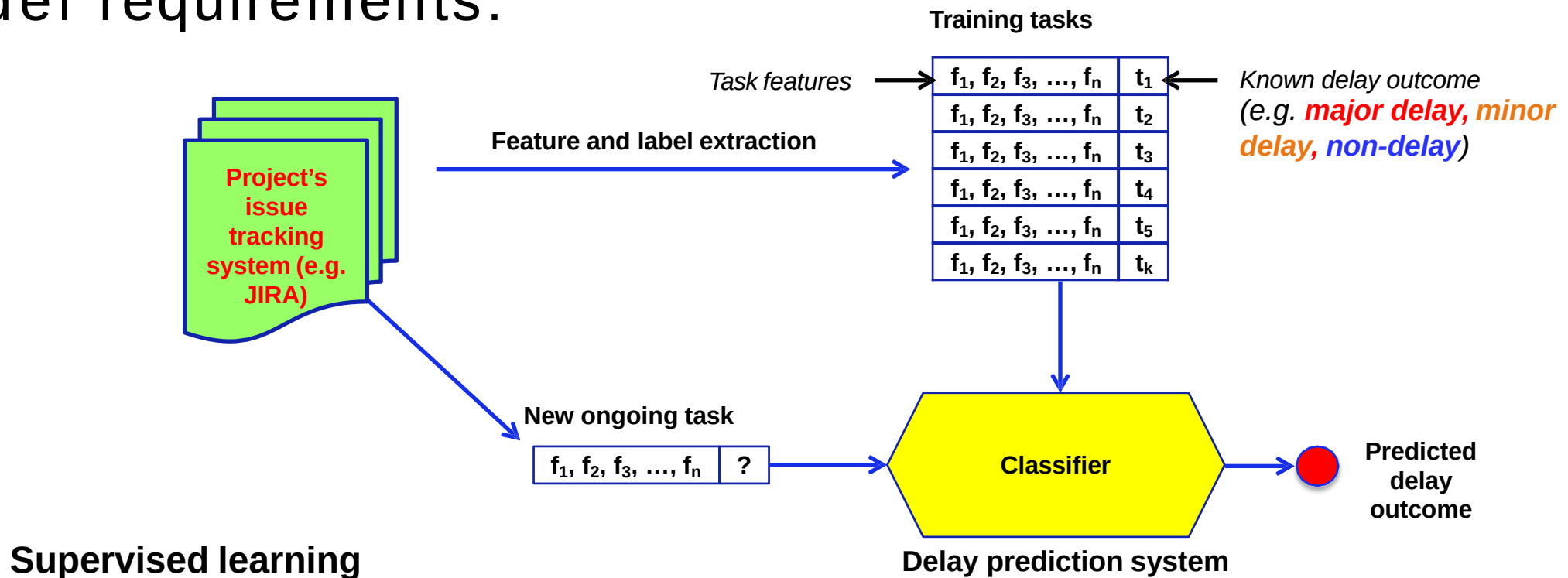
Which of these ongoing tasks will be at risk of being delayed?

Milestone (e.g. release 2.1 on June 2)

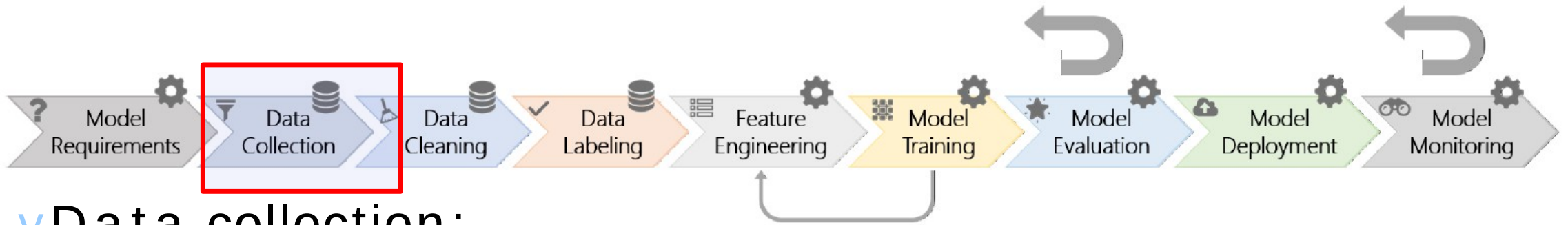
AI Engineering lifecycle - example



Model requirements:



AI Engineering lifecycle



vData collection:

- ✓ Analyze 40,830 past tasks (i.e. issues) in 5 large software projects: Moodle, JBoss, Apache, Duraspace, and Spring.
- ✓ All these tasks are recorded in the JIRA issue tracking system


Application Server 3 4 5 and 6 / JBAS-13

Fix the java5 testsuite errors

Agile Board

Details

Type:

 Task

Priority:

 Major

Affects Version/s:

JBossAS-4.0.0 Final

Component/s:

None

Labels:

None

Environment:

> [starksm@banshee9100 testsuite] \$ /usr/java/jdk1.5.0/bin/java -version java version "1.5.0" Java(TM) 2 Runtime Environment, Standard Edition (build 1.5.0-b64...

Status:

CLOSED (View Workflow)

Resolution:

Done

Fix Version/s:

JBossAS-4.0.1 Final

Description

There are still some failures when running the testsuite under java5 that need to be corrected for the 4.0.1 release. This task contains the subtasks in the java5 testsuite run.

Issue Links

blocks

 [JBAS-7](#) Tasks for the 4.0.1 Release



CLOSED

is blocked by

 [JBACP-4](#) VersionedObjectUnitTestCase fails under java5



CLOSED

 [JBAS-14](#) CompleteUnitTestCase fails under java5



CLOSED

 [JBAS-15](#) ContextCLTestCase fails under java5



CLOSED

 [JBWS-52](#) MarshallDocLiteralTestCase.testEchoDateTimeArray failure under Java5



CLOSED

Activity

All

Comments

Work Log

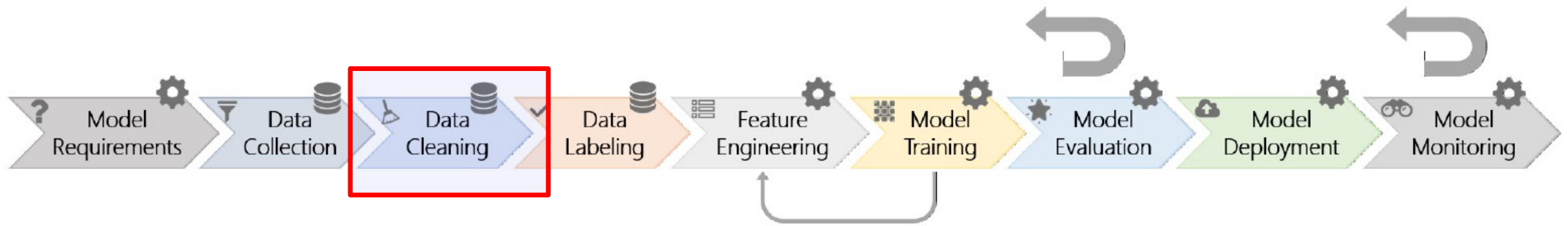
History

Activity

Links Hierarchy

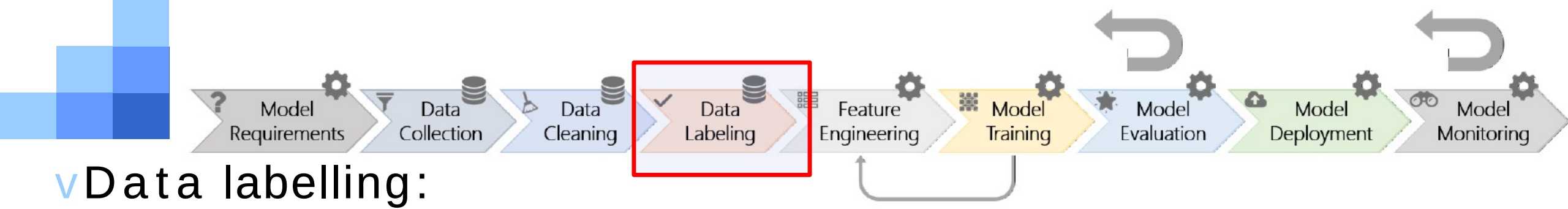
There are no comments yet on this issue.

AI Engineering lifecycle



✓ Data cleaning:

- Remove outliers, e.g. incomplete tasks, issues/tasks long overdue, issues with no due date, etc.



MyFaces Core / MYFACES-2262 197 of 319

implement outcomtarget renderers

Agile Board Export

Details

Type:	☒ Task	Status:	CLOSED
Priority:	Minor	Resolution:	Fixed
Affects Version/s:	2.0.0-alpha	Fix Version/s:	2.0.0-alpha
Component/s:	JSR-314		
Labels:	None		

Description

create link and button renderer implementations for outcome target to support bookmarkable URLs

Issue Links

is part of [MYFACES-2345](#) Add View Parameters feature

People

Assignee: [Leonardo Uribe](#)
Reporter: [Michael Concini](#)
Votes: 0 [Vote for this issue](#)
Watchers: 0 [Start watching this issue](#)

Dates

Due:	10/Jul/09
Created:	29/Jun/09 19:25
Updated:	07/May/10 01:58
Resolved:	10/Sep/09 16:59

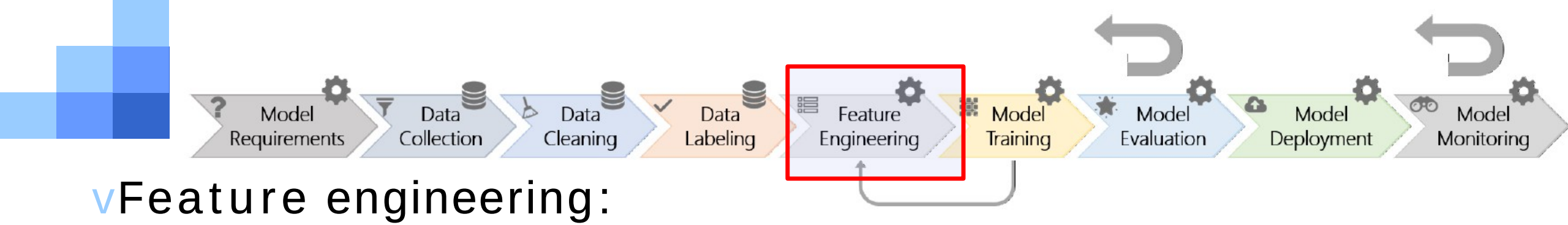
Activity

[View on Board](#)

All Comments Work Log History Activity Transitions

Michael Concini added a comment - 07/Jul/09 14:57
code is ready to be checked in once a new shared trunk has been created.

Leonardo Uribe added a comment - 03/Sep/09 21:05



vFeature engineering:

 MyFaces Core / MYFACES-2262

197 of 319 ▲ ▼

implement outcomtarget renderers

Agile Board

Details

Type: Task

Priority: Minor

Affects Version/s: 2.0.0-alpha

Component/s: JSR-314

Labels: None

Status: CLOSED

Resolution: Fixed

Fix Version/s: 2.0.0-alpha

Description

create link and button renderer implementations for outcome target to support bookmarkable URLs

Issue Links

is part of MYFACES-2345 Add View Parameters feature

Activity

All Comments Work Log History Activity Transitions

Michael Concini added a comment - 07/Jul/09 14:57

code is ready to be checked in once a new shared trunk has been created.

Leonardo Uribe added a comment - 09/Sep/09 21:05

People

Assignee: Leonardo Uribe

Reporter: Michael Concini

Votes: 0 Vote for this issue

Watchers: 0 Start watching this issue

Dates

Due: 10/Jul/09

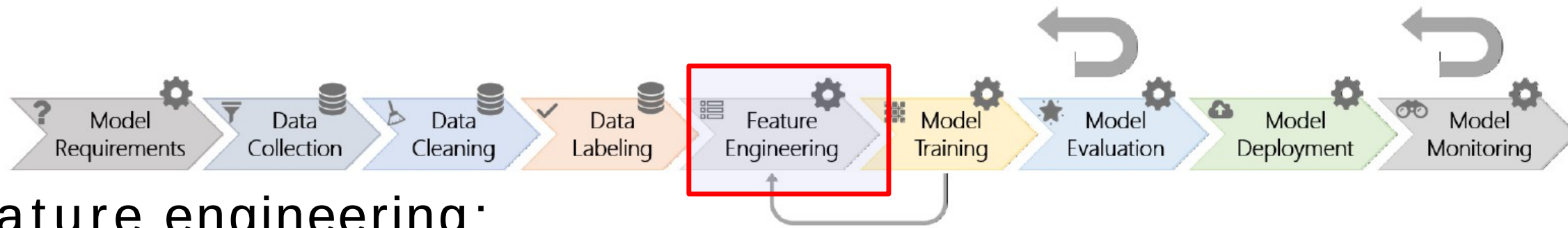
Created: 29/Jun/09 19:25

Updated: 07/May/10 01:58

Resolved: 10/Sep/09 16:59

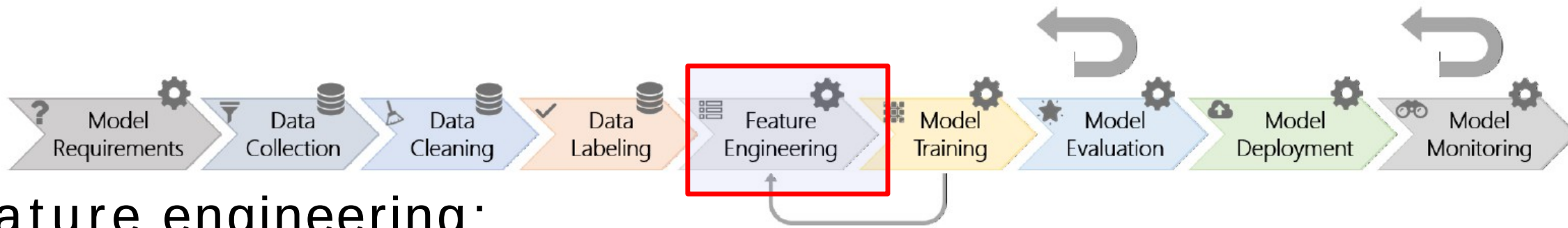
Agile

View on Board



vFeature engineering:

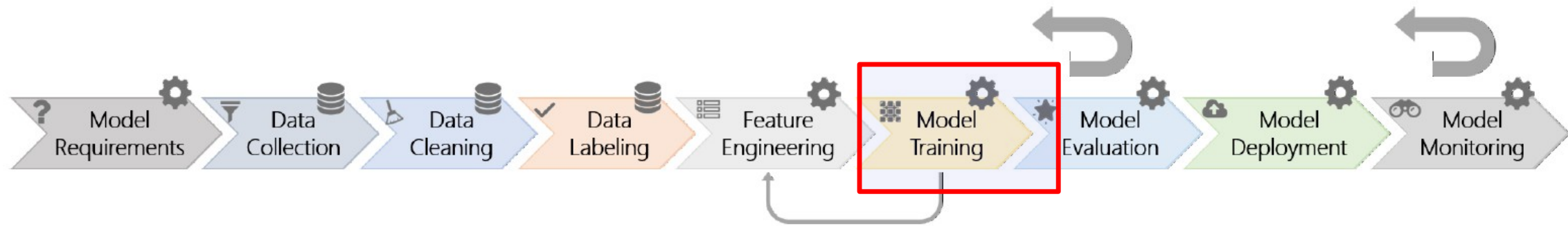
1. Discussion time
2. Waiting time
3. Type
4. Number of times that an issue is reopened
5. Priority
6. Changing of priority
7. Number of comments
8. Number of fix versions
9. Changing of fix versions
10. Number of affect versions
11. Number of issue links
12. Number of issues that are blocked by this issue
13. Number of issues that block this issue
14. Topics of an issue's description (**NLP / LDA**)
15. Changing of description
16. Number of votes
17. Number of watches
18. Reporter reputation
- 19. Developers' workload**
- 20. Percentage of delayed issues that a developer involved with**
21. Task dependencies (e.g. blocking, assigned to the same person, or affecting the same components)



vFeature engineering:

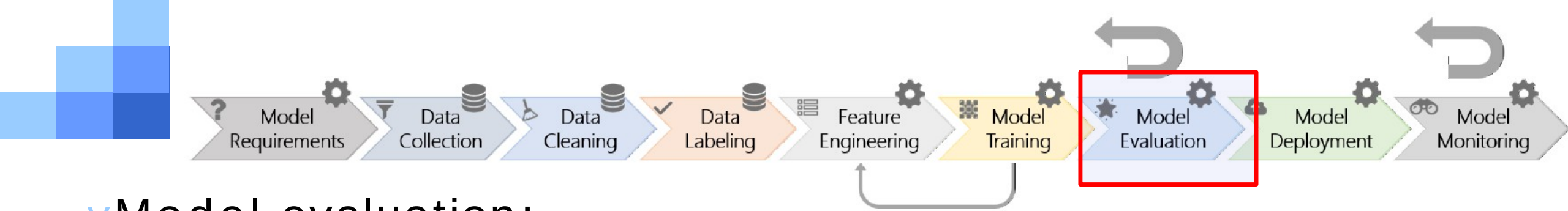
Descriptive ℓ_1 -penalized logistic regression model for risk probability, trained on all tasks collected from the five projects

Feature	Apache	Duraspace	JBoss	Moodle	Spring	All
discussion	1.027	-1.368	-3.153	-21.353	-0.371	-7.274
waiting	0	0	0	-15.307	0	-4.495
type:Bug	0	-3.254	-0.959	-2.933	-4.763	-0.859
type:Code Task	N/A	-0.052	N/A	N/A	N/A	0.216
type:Documentation	0	0.359	1.617	N/A	N/A	1.06
type:Epic	0	1.026	0	0	0	0.925
type:Improvement	0.044	-0.745	N/A	-2.833	-4.704	0.271
type:New Feature	0.106	0.097	0	-3.011	-4.495	0.64
type:Story	0	-0.81	0	0	0	-0.342
type:Sub-task	-1.183	0.267	0.94	-2.796	0.001	0.867
type:Task	-0.37	-0.477	0.724	-2.604	-4.534	0.548
type:Technical task	0	0.35	0	0	0	0.567
repetition	-0.391	4.01	3.11	-0.427	6.358	2.878
perofdelay	3.183	4.202	3.088	2.569	22.091	3.361
workload	0.179	-0.911	1.367	-1.104	0.241	1.621

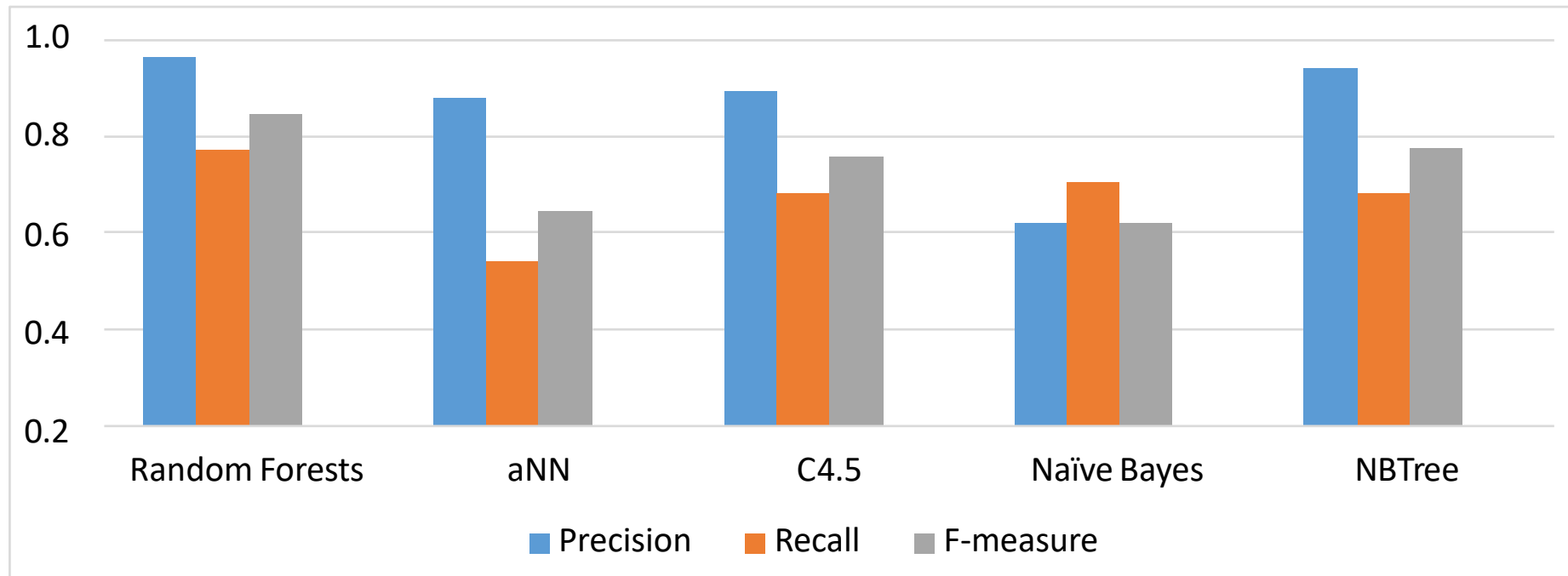


Model training:

- Data (e.g. 40,830 past tasks) split into training set and test set
- Use a number of classifiers: Random Forests, Neural Networks, Decision Tree (C4.5), Naïve Bayes and NBTree.
- Training set is used to train these classifiers.

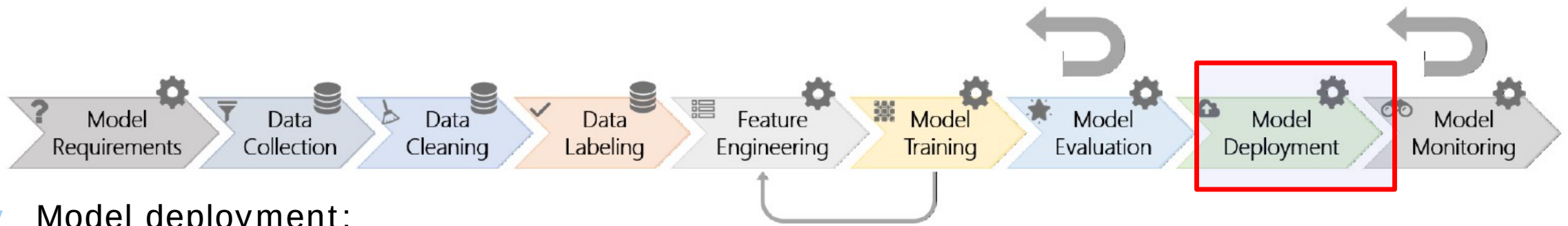


vModel evaluation:



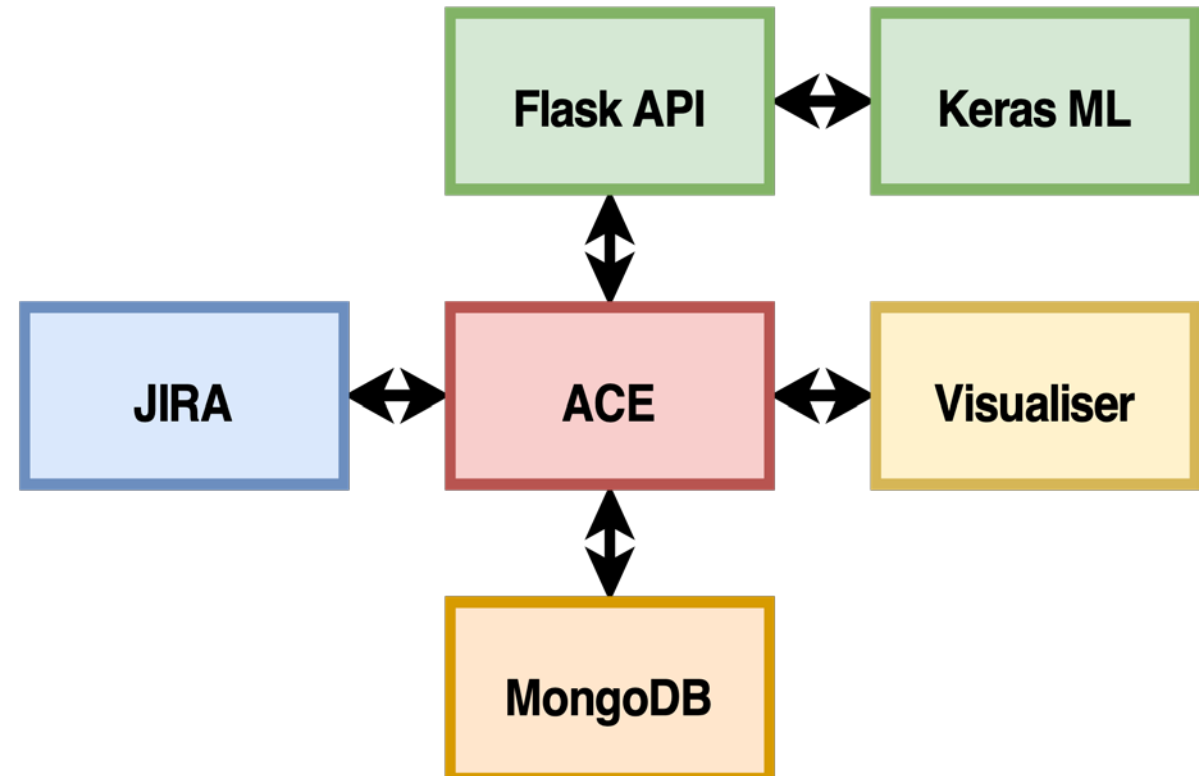
- Morakot Choetkiertikul, Hoa Khanh Dam, Truyen Tran and Aditya Ghose, *Characterization and prediction of issue-related risks in software projects*, Proceedings of **12th International Conference on Mining Software Repositories (MSR 2015)**, co-located with ICSE 2015, pages 280 - 291, IEEE (**ACM SIGSOFT Distinguished Paper Award**)
- Morakot Choetkiertikul, Hoa Khanh Dam, Truyen Tran and Aditya Ghose, *Predicting the delay of issues with due dates in software projects*, **Empirical Software Engineering** journal, Volume 22, Issue 3, pages 1223-1263, Springer.

AI Engineering lifecycle

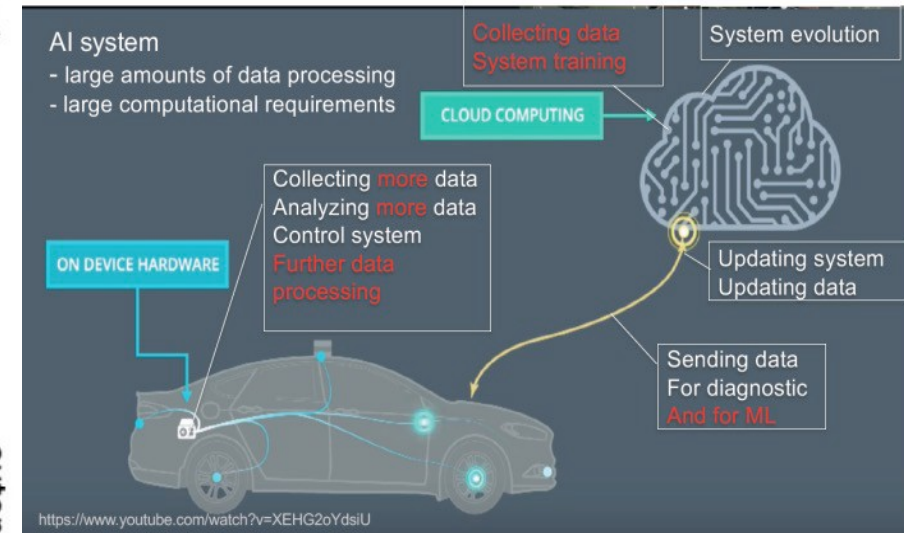
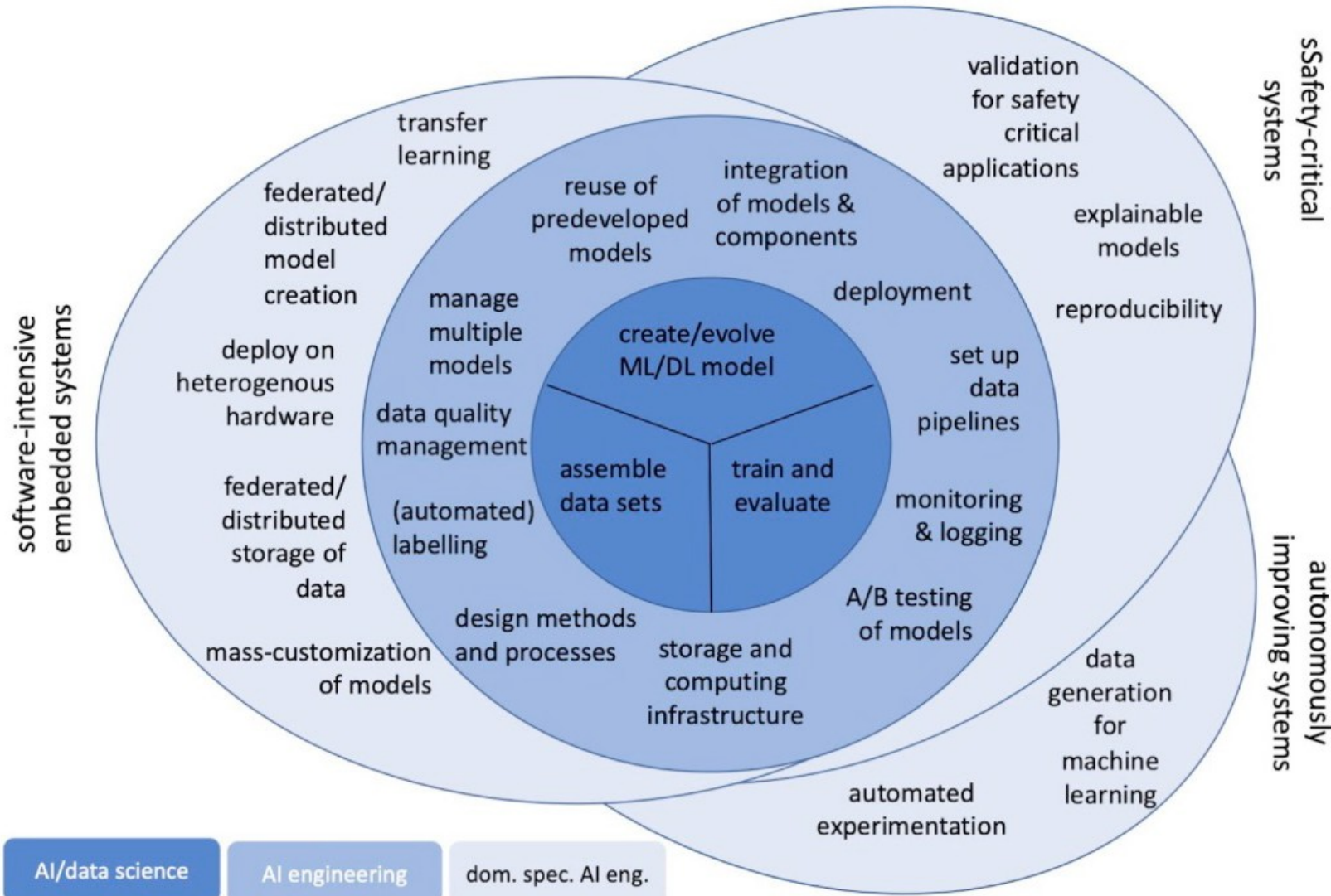


v Model deployment:

- AI-powered plugin for JIRA issue tracking system
 - Recommending story points, labels, priority, type and components for each issue
 - Visualization of issue dependency



Conceptualization of AI Engineering



U

O

W

CSIT314

Software Development

Methodologies

Review



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

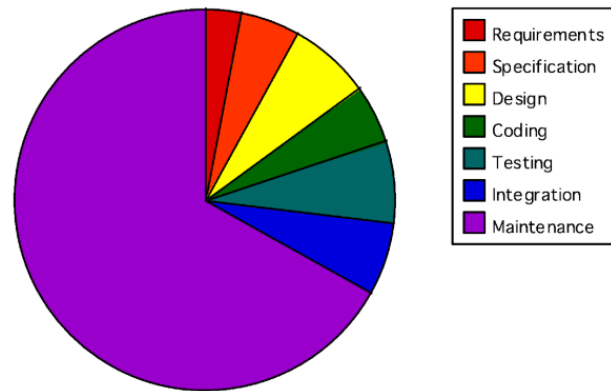
Subject review



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Software Development

- Introduction to Software Development and its Lifecycle
 - Software Engineering
 - Software Development Activities
 - Change propagation
 - Eight laws of software evolution



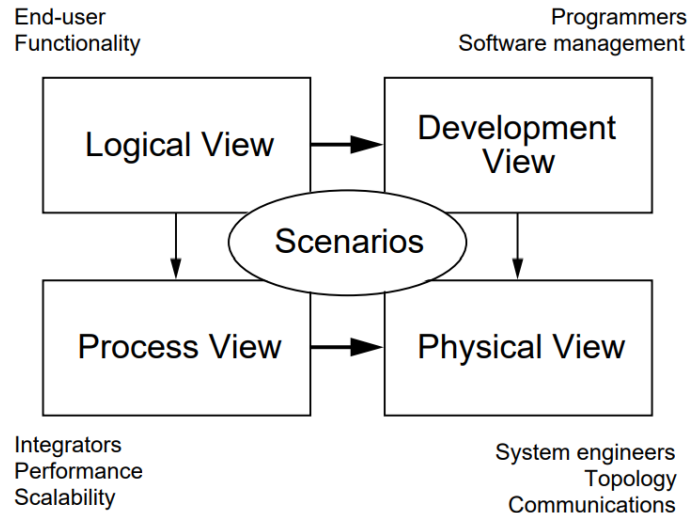
Models

- Software process models
 - Waterfall model
 - Prototyping model
 - Iterative/incremental model
 - Spiral model
 - Rational Unified Process
 - Agile methods



UML

- Unified Modelling Language



- “4+1” View
 - 1 is for user case view, or known as scenario, (user requirement)
- Conceptual and Physical
- Each view for several diagrams



2V

- Verification & Validation
 - “build the product right “ VS “build the right product”
 - Static and Dynamic Verification
 - Software inspections / (unit, white, black-box) testing
- Test-driven development



CI & CD

- Continuous Integration
 - 9 principles and practices
- Continuous Delivery
- CI/CD with practical tools



DevOps

- Unify software development (Dev) and IT operation (Ops)
- DevOps Vs CI/CD, Agile



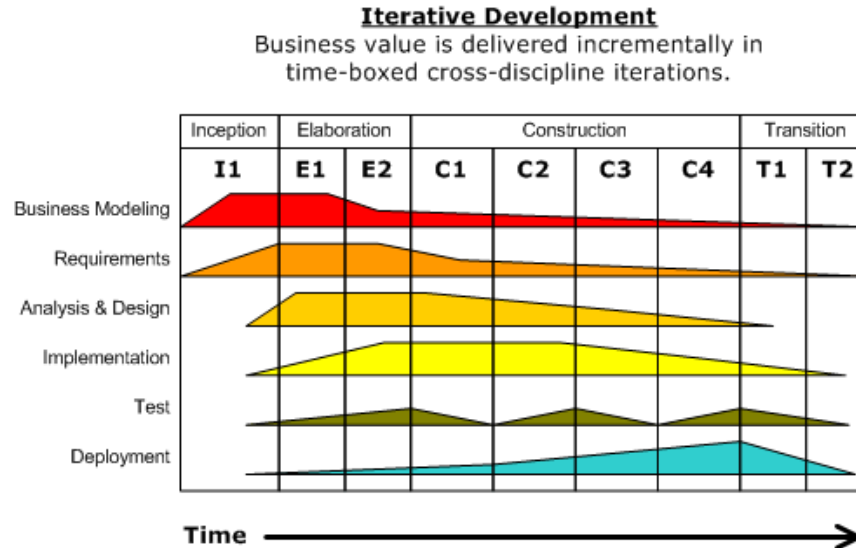
Kanban & XP

- Kanban
 - Represents the overall project (visual interface)
 - 5 components
- Extreme programming
 - 12 practices
 - Extreme Programming Process



RUP

- Rational Unified Process
 - Two ways:
 - horizontal/vertical
 - Building blocks



CMMI

- Capability Maturity Model Integration
 - 5 Maturity Levels
 - Process Areas

Maturity Level	Project Management	Engineering	Process Management	Support
5 Optimizing			Organizational Innovation & Deployment	Causal Analysis & Resolution
4 Quantitatively Managed	Quantitative Project Mngt		Organizational Process Performance	
3 Defined	Integrated Project Mngt Risk Management	Requirements Development Technical Solution Product Integration Verification Validation	Organizational Process Focus Organizational Process Definition Organizational Training	Decision Analysis & Resolution
2 Managed	Project Planning Project Monitoring & Control Supplier Agreement Mngt	Requirements Mngt		Measurement & Analysis Process & Product Quality Assurance Configuration Mngt
1 Initial				



Data-driven

- Data-driven Software Development
 - Model requirements/deployment/monitoring
 - Data collection, cleaning, labelling and feature engineering
 - Model training, evaluation

